

Part II

Low-Level

Engine Systems



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

6

Engine Support Systems

Every game engine requires some low-level support systems that manage mundane but crucial tasks, such as starting up and shutting down the engine, configuring engine and game features, managing the engine's memory usage, handling access to file system(s), providing access to the wide range of heterogeneous asset types used by the game (meshes, textures, animations, audio, etc.), and providing debugging tools for use by the game development team. This chapter will focus on the lowest-level support systems found in most game engines. In the chapters that follow, we will explore some of the larger core systems, including resource management, human interface devices and in-game debugging tools.

6.1 Subsystem Start-Up and Shut-Down

A game engine is a complex piece of software consisting of many interacting subsystems. When the engine first starts up, each subsystem must be configured and initialized in a specific order. Interdependencies between subsystems implicitly define the order in which they must be started—i.e., if subsystem B depends on subsystem A, then A will need to be started up before B can be initialized. Shut-down typically occurs in the reverse order, so B would shut down first, followed by A.

6.1.1 C++ Static Initialization Order (or Lack Thereof)

Since the programming language used in most modern game engines is C++, we should briefly consider whether C++'s native start-up and shut-down semantics can be leveraged in order to start up and shut down our engine's subsystems. In C++, global and static objects are constructed before the program's entry point (`main()`, or `WinMain()` under Windows) is called. However, these constructors are called in a totally unpredictable order. The destructors of global and static class instances are called after `main()` (or `WinMain()`) returns, and once again they are called in an unpredictable order. Clearly this behavior is not desirable for initializing and shutting down the subsystems of a game engine, or indeed any software system that has interdependencies between its global objects.

This is somewhat unfortunate, because a common design pattern for implementing major subsystems such as the ones that make up a game engine is to define a *singleton class* (often called a *manager*) for each subsystem. If C++ gave us more control over the order in which global and static class instances were constructed and destroyed, we could define our singleton instances as globals, without the need for dynamic memory allocation. For example, we could write:

```
class RenderManager
{
public:
    RenderManager()
    {
        // start up the manager...
    }

    ~RenderManager()
    {
        // shut down the manager...
    }

    // ...
};

// singleton instance
static RenderManager gRenderManager;
```

Alas, with no way to directly control construction and destruction order, this approach won't work.

6.I.I.I Construct On Demand

There is one C++ “trick” we can leverage here. A static variable that is declared within a function will not be constructed before `main()` is called, but rather on the first invocation of that function. So if our global singleton is function-static, we can control the order of construction for our global singletons.

```
class RenderManager
{
public:
    // Get the one and only instance.
    static RenderManager& get()
    {
        // This function-static will be constructed on the
        // first call to this function.
        static RenderManager sSingleton;
        return sSingleton;
    }

    RenderManager()
    {
        // Start up other managers we depend on, by
        // calling their get() functions first...
        VideoManager::get();
        TextureManager::get();

        // Now start up the render manager.
        // ...
    }

    ~RenderManager()
    {
        // Shut down the manager.
        // ...
    }
};
```

You’ll find that many software engineering textbooks suggest this design or a variant that involves dynamic allocation of the singleton as shown below.

```
static RenderManager& get()
{
    static RenderManager* gpSingleton = nullptr;
    if (gpSingleton == nullptr)
    {
        gpSingleton = new RenderManager;
```

```

    }
    ASSERT(gpSingleton);
    return *gpSingleton;
}

```

Unfortunately, this still gives us no way to control destruction order. It is possible that C++ will destroy one of the managers upon which the `RenderManager` depends for its shut-down procedure, prior to the `RenderManager`'s destructor being called. In addition, it's difficult to predict exactly when the `RenderManager` singleton will be constructed, because the construction will happen on the first call to `RenderManager::get()`—and who knows when that might be? Moreover, the programmers using the class may not be expecting an innocuous-looking `get()` function to do something expensive, like allocating and initializing a heavyweight singleton. This is an unpredictable and dangerous design. Therefore, we are prompted to resort to a more direct approach that gives us greater control.

6.1.2 A Simple Approach That Works

Let's presume that we want to stick with the idea of singleton managers for our subsystems. In this case, the simplest "brute-force" approach is to define explicit start-up and shut-down functions for each singleton manager class. These functions take the place of the constructor and destructor, and in fact we should arrange for the constructor and destructor to do *absolutely nothing*. That way, the start-up and shut-down functions can be explicitly called *in the required order* from within `main()` (or from some overarching singleton object that manages the engine as a whole). For example:

```

class RenderManager
{
public:
    RenderManager()
    {
        // do nothing
    }

    ~RenderManager()
    {
        // do nothing
    }

    void startUp()
    {
        // start up the manager...
    }
}

```

```

    }

    void shutDown()
    {
        // shut down the manager...
    }

    // ...
};

class PhysicsManager    { /* similar... */ };
class AnimationManager { /* similar... */ };
class MemoryManager     { /* similar... */ };
class FileSystemManager { /* similar... */ };

// ...

RenderManager      gRenderManager;
PhysicsManager     gPhysicsManager;
AnimationManager   gAnimationManager;
TextureManager     gTextureManager;
VideoManager       gVideoManager;
MemoryManager      gMemoryManager;
FileSystemManager  gFileSystemManager;
// ...

int main(int argc, const char* argv)
{
    // Start up engine systems in the correct order.
    gMemoryManager.startUp();
    gFileSystemManager.startUp();
    gVideoManager.startUp();
    gTextureManager.startUp();
    gRenderManager.startUp();
    gAnimationManager.startUp();
    gPhysicsManager.startUp();
    // ...

    // Run the game.
    gSimulationManager.run();

    // Shut everything down, in reverse order.
    // ...
    gPhysicsManager.shutDown();

```

```
gAnimationManager.shutdown();  
gRenderManager.shutdown();  
gFileManager.shutdown();  
gMemoryManager.shutdown();  
  
return 0;  
}
```

There are “more elegant” ways to accomplish this. For example, you could have each manager register itself into a global priority queue and then walk this queue to start up all the managers in the proper order. You could define the manager-to-manager dependency graph by having each manager explicitly list the other managers upon which it depends and then write some code to calculate the optimal start-up order given their interdependencies. You could use the construct-on-demand approach outlined above. In my experience, the brute-force approach always wins out, because of the following:

- It’s simple and easy to implement.
- It’s explicit. You can see and understand the start-up order immediately by just looking at the code.
- It’s easy to debug and maintain. If something isn’t starting early enough, or is starting too early, you can just move one line of code.

One minor disadvantage to the brute-force manual start-up and shut-down method is that you might accidentally shut things down in an order that isn’t strictly the reverse of the start-up order. But I wouldn’t lose any sleep over it. As long as you can start up and shut down your engine’s subsystems successfully, you’re golden.

6.1.3 Some Examples from Real Engines

Let’s take a brief look at some examples of engine start-up and shut-down taken from real game engines.

6.1.3.1 OGRE

OGRE is by its authors’ admission a rendering engine, not a game engine per se. But by necessity it provides many of the low-level features found in full-fledged game engines, including a simple and elegant start-up and shut-down mechanism. Everything in OGRE is controlled by the singleton object `Ogre::Root`. It contains pointers to every other subsystem in OGRE and manages their creation and destruction. This makes it very easy for a programmer to start up OGRE—just new an instance of `Ogre::Root` and you’re done.

Here are a few excerpts from the OGRE source code so we can see what it's doing:

OgreRoot.h

```
class _OgreExport Root : public Singleton<Root>
{
    // <some code omitted...>

    // Singletons
    LogManager* mLogManager;
    ControllerManager* mControllerManager;
    SceneManagerEnumerator* mSceneManagerEnum;
    SceneManager* mCurrentSceneManager;
    DynLibManager* mDynLibManager;
    ArchiveManager* mArchiveManager;
    MaterialManager* mMaterialManager;
    MeshManager* mMeshManager;
    ParticleSystemManager* mParticleManager;
    SkeletonManager* mSkeletonManager;
    OverlayElementFactory* mPanelFactory;
    OverlayElementFactory* mBorderPanelFactory;
    OverlayElementFactory* mTextAreaFactory;
    OverlayManager* mOverlayManager;
    FontManager* mFontManager;
    ArchiveFactory *mZipArchiveFactory;
    ArchiveFactory *mFileSystemArchiveFactory;
    ResourceGroupManager* mResourceGroupManager;
    ResourceBackgroundQueue* mResourceBackgroundQueue;
    ShadowTextureManager* mShadowTextureManager;

    // etc.
};
```

OgreRoot.cpp

```
Root::Root(const String& pluginFileName,
           const String& configFileFileName,
           const String& logFileName) :
    mLogManager(0),
    mCurrentFrame(0),
    mFrameSmoothingTime(0.0f),
    mNextMovableObjectTypeFlag(1),
    mIsInitialised(false)
{
    // superclass will do singleton checking
    String msg;

    // Init
    mActiveRenderer = 0;
```

```

mVersion
    = StringConverter::toString(OGRE_VERSION_MAJOR)
    + "."
    + StringConverter::toString(OGRE_VERSION_MINOR)
    + "."
    + StringConverter::toString(OGRE_VERSION_PATCH)
    + OGRE_VERSION_SUFFIX + " "
    + "(" + OGRE_VERSION_NAME + ")";
mConfigFileName = configFileName;

// create log manager and default log file if there
// is no log manager yet
if(LogManager::getSingletonPtr() == 0)
{
    mLogManager = new LogManager();
    mLogManager->createLog(logFileName, true, true);
}

// dynamic library manager
mDynLibManager = new DynLibManager();
mArchiveManager = new ArchiveManager();

// ResourceGroupManager
mResourceGroupManager = new ResourceGroupManager();

// ResourceBackgroundQueue
mResourceBackgroundQueue
    = new ResourceBackgroundQueue();

// and so on...
}

```

OGRE provides a templated `Ogre::Singleton` base class from which all of its singleton (manager) classes derive. If you look at its implementation, you'll see that `Ogre::Singleton` does not use deferred construction but instead relies on `Ogre::Root` to explicitly new each singleton. As we discussed above, this is done to ensure that the singletons are created and destroyed in a well-defined order.

6.1.3.2 Naughty Dog's *Uncharted* and *The Last of Us* Series

The engine created by Naughty Dog, Inc. for its *Uncharted* and *The Last of Us* series of games uses a similar explicit technique for starting up its subsystems. You'll notice by looking at the following code that engine start-up is not always a simple sequence of allocating singleton instances. A wide range of operating system services, third-party libraries and so on must all be started

up during engine initialization. Also, dynamic memory allocation is avoided wherever possible, so many of the singletons are statically allocated objects (e.g., `g_fileSystem`, `g_languageMgr`, etc.) It's not always pretty, but it gets the job done.

```
Err BigInit()
{
    init_exception_handler();

    U8* pPhysicsHeap = new(kAllocGlobal, kAlign16)
        U8[ALLOCATION_GLOBAL_PHYS_HEAP];
    PhysicsAllocatorInit(pPhysicsHeap,
        ALLOCATION_GLOBAL_PHYS_HEAP);

    g_textDb.Init();
    g_textSubDb.Init();
    g_spuMgr.Init();

    g_drawScript.InitPlatform();

    PlatformUpdate();

    thread_t init_thr;
    thread_create(&init_thr, threadInit, 0, 30,
        64*1024, 0, "Init");

    char masterConfigFileName[256];
    snprintf(masterConfigFileName,
        sizeof(masterConfigFileName),
        MASTER_CFG_PATH);
    {
        Err err = ReadConfigFromFile(
            masterConfigFileName);
        if (err.Failed())
        {
            MsgErr("Config file not found (%s).\n",
                masterConfigFileName);
        }
    }

    memset(&g_discInfo, 0, sizeof(BootDiscInfo));
    int err1 = GetBootDiscInfo(&g_discInfo);
    Msg("GetBootDiscInfo() : 0x%x\n", err1);
    if(err1 == BOOTDISCINFO_RET_OK)
    {
        printf("titleId      : [%s]\n",
            g_discInfo.titleId);
    }
}
```

```

        printf("parentalLevel : [%d]\n",
               g_discInfo.parentalLevel);
    }

    g_fileSystem.Init(g_gameInfo.m_onDisc);

    g_languageMgr.Init();
    if (g_shouldQuit) return Err::kOK;

    // and so on...

```

6.2 Memory Management

As game developers, we are always trying to make our code run more quickly. The performance of any piece of software is dictated not only by the algorithms it employs, or the efficiency with which those algorithms are coded, but also by how the program *utilizes memory* (RAM). Memory affects performance in two ways:

1. *Dynamic memory allocation* via `malloc()` or C++'s global operator `new` is a very slow operation. We can improve the performance of our code by either avoiding dynamic allocation altogether or by making use of custom memory allocators that greatly reduce allocation costs.
2. On modern CPUs, the performance of a piece of software is often dominated by its *memory access patterns*. As we'll see, data that is located in small, *contiguous* blocks of memory can be operated on much more efficiently by the CPU than if that same data were to be spread out across a wide range of memory addresses. Even the most efficient algorithm, coded with the utmost care, can be brought to its knees if the data upon which it operates is not laid out efficiently in memory.

In this section, we'll learn how to optimize our code's memory utilization along these two axes.

6.2.1 Optimizing Dynamic Memory Allocation

Dynamic memory allocation via `malloc()` and `free()` or C++'s global `new` and `delete` operators—also known as *heap allocation*—is typically very slow. The high cost can be attributed to two main factors. First, a heap allocator is a general-purpose facility, so it must be written to handle any allocation size, from one byte to one gigabyte. This requires a lot of management overhead, making the `malloc()` and `free()` functions inherently slow. Second,

on most operating systems a call to `malloc()` or `free()` must first context-switch from user mode into kernel mode, process the request and then context-switch back to the program. These context switches can be extraordinarily expensive. One rule of thumb often followed in game development is:

Keep heap allocations to a minimum, and never allocate from the heap within a tight loop.

Of course, no game engine can entirely avoid dynamic memory allocation, so most game engines implement one or more custom allocators. A custom allocator can have better performance characteristics than the operating system's heap allocator for two reasons. First, a custom allocator can satisfy requests from a preallocated memory block (itself allocated using `malloc()` or `new`, or declared as a global variable). This allows it to run in user mode and entirely avoid the cost of context-switching into the operating system. Second, by making various assumptions about its usage patterns, a custom allocator can be much more efficient than a general-purpose heap allocator.

In the following sections, we'll take a look at some common kinds of custom allocators. For additional information on this topic, see Christian Gyrling's excellent blog post, <http://www.swedishcoding.com/2008/08/31/are-we-out-of-memory>.

6.2.1.1 Stack-Based Allocators

Many games allocate memory in a stack-like fashion. Whenever a new game level is loaded, memory is allocated for it. Once the level has been loaded, little or no dynamic memory allocation takes place. At the conclusion of the level, its data is unloaded and all of its memory can be freed. It makes a lot of sense to use a stack-like data structure for these kinds of memory allocations.

A *stack allocator* is very easy to implement. We simply allocate a large contiguous block of memory using `malloc()` or global `new`, or by declaring a global array of bytes (in which case the memory is effectively allocated out of the executable's BSS segment). A pointer to the top of the stack is maintained. All memory addresses below this pointer are considered to be in use, and all addresses above it are considered to be free. The top pointer is initialized to the lowest memory address in the stack. Each allocation request simply moves the pointer up by the requested number of bytes. The most recently allocated block can be freed by simply moving the top pointer back down by the size of the block.

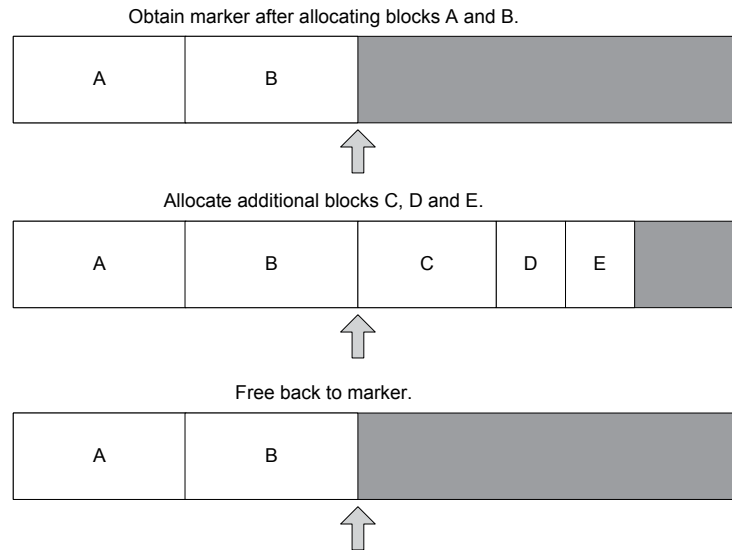


Figure 6.1. Stack allocation and freeing back to a marker.

It is important to realize that with a stack allocator, memory cannot be freed in an arbitrary order. All frees must be performed in an order opposite to that in which they were allocated. One simple way to enforce these restrictions is to disallow individual blocks from being freed at all. Instead, we can provide a function that rolls the stack top back to a previously marked location, thereby freeing all blocks between the current top and the roll-back point.

It's important to always roll the top pointer back to a point that lies at the boundary between two allocated blocks, because otherwise new allocations would overwrite the tail end of the top-most block. To ensure that this is done properly, a stack allocator often provides a function that returns a *marker* representing the current top of the stack. The roll-back function then takes one of these markers as its argument. This is depicted in Figure 6.1. The interface of a stack allocator often looks something like this.

```
class StackAllocator
{
public:
    // Stack marker: Represents the current top of the
    // stack. You can only roll back to a marker, not to
    // arbitrary locations within the stack.
    typedef U32 Marker;
```

```

    // Constructs a stack allocator with the given total
    // size.
    explicit StackAllocator(U32 stackSize_bytes);

    // Allocates a new block of the given size from stack
    // top.
    void* alloc(U32 size_bytes);

    // Returns a marker to the current stack top.
    Marker getMarker();

    // Rolls the stack back to a previous marker.
    void freeToMarker(Marker marker);

    // Clears the entire stack (rolls the stack back to
    // zero).
    void clear();

private:
    // ...
};

```

Double-Ended Stack Allocators

A single memory block can actually contain two stack allocators—one that allocates up from the bottom of the block and one that allocates down from the top of the block. A double-ended stack allocator is useful because it uses memory more efficiently by allowing a trade-off to occur between the memory usage of the bottom stack and the memory usage of the top stack. In some situations, both stacks may use roughly the same amount of memory and meet in the middle of the block. In other situations, one of the two stacks may eat up a lot more memory than the other stack, but all allocation requests can still be satisfied as long as the total amount of memory requested is not larger than the block shared by the two stacks. This is depicted in Figure 6.2.

In Midway’s *Hydro Thunder* arcade game, all memory allocations are made from a single large block of memory managed by a double-ended stack allocator. The bottom stack is used for loading and unloading levels (race tracks), while the top stack is used for temporary memory blocks that are allocated and freed every frame. This allocation scheme worked extremely well and ensured that *Hydro Thunder* never suffered from memory fragmentation problems (see Section 6.2.1.4). Steve Ranck, *Hydro Thunder*’s lead engineer, describes this allocation technique in depth in [8, Section 1.9].



Figure 6.2. A double-ended stack allocator.

6.2.1.2 Pool Allocators

It's quite common in game engine programming (and software engineering in general) to allocate lots of small blocks of memory, each of which are the same size. For example, we might want to allocate and free matrices, or iterators, or links in a linked list, or renderable mesh instances. For this type of memory allocation pattern, a *pool allocator* is often the perfect choice.

A pool allocator works by preallocating a large block of memory whose size is an exact multiple of the size of the elements that will be allocated. For example, a pool of 4×4 matrices would be an exact multiple of 64 bytes—that's 16 elements per matrix times four bytes per element (assuming each element is a 32-bit `float`). Each element within the pool is added to a linked list of free elements; when the pool is first initialized, the free list contains all of the elements. Whenever an allocation request is made, we simply grab the next free element off the free list and return it. When an element is freed, we simply tack it back onto the free list. Both allocations and frees are $O(1)$ operations, since each involves only a couple of pointer manipulations, no matter how many elements are currently free. (The notation $O(1)$ is an example of “big O” notation. In this case it means that the execution times of both allocations and frees are roughly constant and do not depend on things like the number of elements currently in the pool. See Section 6.3.3 for an explanation of “big O” notation.)

The linked list of free elements can be a singly-linked list, meaning that we need a single pointer (four bytes on 32-bit machines or eight bytes on 64-bit machines) for each free element. Where should we obtain the memory for all these pointers? Certainly they could be stored in a separate preallocated memory block, occupying `(sizeof(void*) * numElementsInPool)` bytes. However, this is unduly wasteful. The key is to realize that the memory blocks residing on the free list are, by definition, free memory blocks. So why not store each free list “next” pointer *within the free block itself*? This little “trick” works as long as `elementSize >= sizeof(void*)`. We don't waste any memory, because our free list pointers all reside inside the free memory blocks—in memory that wasn't being used for anything anyway!

If each element is smaller than a pointer, then we can use pool element in-

dices instead of pointers to implement our linked list. For example, if our pool contains 16-bit integers, then we can use 16-bit indices as the “next pointers” in our linked list. This works as long as the pool doesn’t contain more than $2^{16} = 65,536$ elements.

6.2.1.3 Aligned Allocations

As we saw in Section 3.3.7.1, every variable and data object has an alignment requirement. An 8-bit integer variable can be aligned to any address, but a 32-bit integer or floating-point variable must be 4-byte aligned, meaning its address can only end in the nibbles 0x0, 0x4, 0x8 or 0xC. A 128-bit SIMD vector value generally has a 16-byte alignment requirement, meaning that its memory address can end only in the nibble 0x0. On the PS3, memory blocks that are to be transferred to an SPU via the direct memory access (DMA) controller should be 128-byte aligned for maximum DMA throughput, meaning they can only end in the bytes 0x00 or 0x80.

All memory allocators must be capable of returning aligned memory blocks. This is relatively straightforward to implement. We simply allocate a little bit more memory than was actually requested, shift the address of the memory block upward slightly so that it is aligned properly, and then return the shifted address. Because we allocated a bit more memory than was requested, the returned block will still be large enough, even with the slight upward shift.

In most implementations, the number of additional bytes allocated is equal to the alignment minus one, which is the worst-case alignment shift we can make. For example, if we want a 16-byte aligned memory block, the worst case would be to get back an unaligned pointer that ends in 0x1, because it would require us to apply a shift of 15 bytes to bring it to a 16-byte boundary.

Here’s one possible implementation of an aligned memory allocator:

```
// Shift the given address upwards if/as necessary to
// ensure it is aligned to the given number of bytes.
inline uintptr_t AlignAddress(uintptr_t addr, size_t align)
{
    const size_t mask = align - 1;
    assert((align & mask) == 0); // pwr of 2
    return (addr + mask) & ~mask;
}

// Shift the given pointer upwards if/as necessary to
// ensure it is aligned to the given number of bytes.
template<typename T>
inline T* AlignPointer(T* ptr, size_t align)
```

```

{
    const uintptr_t addr = reinterpret_cast<uintptr_t>(ptr);
    const uintptr_t addrAligned = AlignAddress(addr, align);
    return reinterpret_cast<T*>(addrAligned);
}

// Aligned allocation function. IMPORTANT: 'align'
// must be a power of 2 (typically 4, 8 or 16).
void* AllocAligned(size_t bytes, size_t align)
{
    // Determine worst case number of bytes we'll need.
    size_t worstCaseBytes = bytes + align - 1;

    // Allocate unaligned block.
    U8* pRawMem = new U8[worstCaseBytes];

    // Align the block.
    return AlignPointer(pRawMem, align);
}

```

The alignment “magic” is performed by the function `AlignAddress()`. Here’s how it works: Given an address and a desired alignment L , we can align that address to an L -byte boundary by first adding $L - 1$ to it, and then stripping off the N least-significant bits of the resulting address, where $N = \log_2(L)$. For example, to align any address to a 16-byte boundary, we shift it up by 15 bytes and then mask off the $N = \log_2(16) = 4$ least-significant bits.

To strip off these bits, we need a *mask* that we can apply to the address using the bitwise AND operator. Because L is always a power of two, $L - 1$ will be a mask with binary 1s in the N least-significant bits and binary 0s everywhere else. So all we need to do is *invert* this mask and then AND it with the address (`addr & ~mask`).

Freeing Aligned Blocks

When an aligned block is later freed, we will be passed the *shifted* address, not the original address that we allocated. But in order to free the memory, we need to free the address that was actually returned by `new`. How can we convert an aligned address back into its original, unaligned address?

One simple approach is to store the shift (i.e., the difference between the aligned address and the original address) some place where our free function will be able to find it. Recall that we actually allocate `align - 1` extra bytes in `AllocAligned()`, in order to give us some room to align the pointer. Those

extra bytes are a perfect place to store the shift value. The smallest shift we'll ever make is one byte, so that's the minimum space we'll have to store the offset. Therefore, given an aligned pointer p , we can simply store the shift as a one-byte value at address $p - 1$.

However, there's one problem: It's possible that the raw address returned by `new` will already be aligned. In that case, the code we presented above would not shift the raw address at all, meaning there'd be no extra bytes into which to store the offset. To overcome this, we simply allocate L extra bytes, instead of $L - 1$, and then we *always* shift the raw pointer up to the next L -byte boundary, even if it was already aligned. Now the maximum shift will be L bytes, and the minimum shift will be 1 byte. So there will always be at least one extra byte into which we can store our shift value.

Storing the shift in a single byte works for alignments up to and including 128. We never ever shift a pointer by zero bytes, so we can make this scheme work for up to 256-byte alignment by interpreting the impossible shift value of zero as a 256-byte shift. (For larger alignments, we'd have to allocate even more bytes, and shift the pointer even further up, to make room for a wider "header".)

Here's how the modified `AllocAligned()` function and its corresponding `FreeAligned()` function could be implemented. The process of allocating and freeing aligned blocks is illustrated in Figure 6.3.

```
// Aligned allocation function. IMPORTANT: 'align'
// must be a power of 2 (typically 4, 8 or 16).
void* AllocAligned(size_t bytes, size_t align)
{
    // Allocate 'align' more bytes than we need.
    size_t actualBytes = bytes + align;

    // Allocate unaligned block.
    U8* pRawMem = new U8[actualBytes];

    // Align the block. If no alignment occurred,
    // shift it up the full 'align' bytes so we
    // always have room to store the shift.
    U8* pAlignedMem = AlignPointer(pRawMem, align);
    if (pAlignedMem == pRawMem)
        pAlignedMem += align;

    // Determine the shift, and store it.
    // (This works for up to 256-byte alignment.)
    ptrdiff_t shift = pAlignedMem - pRawMem;
    assert(shift > 0 && shift <= 256);
```

```

        pAlignedMem[-1] = static_cast<U8>(shift & 0xFF);

        return pAlignedMem;
    }

    void FreeAligned(void* pMem)
    {
        if (pMem)
        {
            // Convert to U8 pointer.
            U8* pAlignedMem = reinterpret_cast<U8*>(pMem);

            // Extract the shift.
            ptrdiff_t shift = pAlignedMem[-1];
            if (shift == 0)
                shift = 256;

            // Back up to the actual allocated address,
            // and array-delete it.
            U8* pRawMem = pAlignedMem - shift;
            delete[] pRawMem;
        }
    }

```

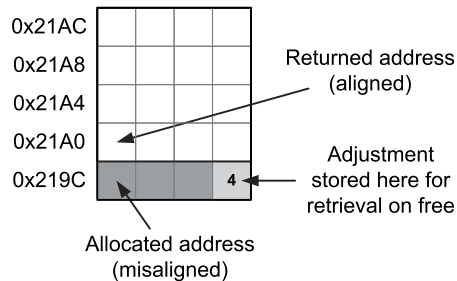


Figure 6.3. Aligned memory allocation with a 16-byte alignment requirement. The difference between the allocated memory address and the adjusted (aligned) address is stored in the byte immediately preceding the adjusted address, so that it may be retrieved during free.

6.2.1.4 Single-Frame and Double-Buffered Memory Allocators

Virtually all game engines allocate at least some temporary data during the game loop. This data is either discarded at the end of each iteration of the loop or used on the next frame and then discarded. This allocation pattern is so common that many engines support *single-frame* and *double-buffered* allocators.

Single-Frame Allocators

A single-frame allocator is implemented by reserving a block of memory and managing it with a simple stack allocator as described above. At the beginning of each frame, the stack's "top" pointer is cleared to the bottom of the memory block. Allocations made during the frame grow toward the top of the block. Rinse and repeat.

```
StackAllocator g_singleFrameAllocator;

// Main Game Loop
while (true)
{
    // Clear the single-frame allocator's buffer every
    // frame.
    g_singleFrameAllocator.clear();

    // ...

    // Allocate from the single-frame buffer. We never
    // need to free this data! Just be sure to use it
    // only this frame.
    void* p = g_singleFrameAllocator.alloc(nBytes);

    // ...
}
```

One of the primary benefits of a single-frame allocator is that allocated memory needn't ever be freed—we can rely on the fact that the allocator will be cleared at the start of every frame. Single-frame allocators are also blindingly fast. The one big negative is that using a single-frame allocator requires a reasonable level of discipline on the part of the programmer. You need to realize that a memory block allocated out of the single-frame buffer will only be valid during the current frame. Programmers must *never* cache a pointer to a single-frame memory block across the frame boundary!

Double-Buffered Allocators

A double-buffered allocator allows a block of memory allocated on frame i to be used on frame $(i + 1)$. To accomplish this, we create two single-frame stack allocators of equal size and then ping-pong between them every frame.

```
class DoubleBufferedAllocator
{
    U32          m_curStack;
```

```

        StackAllocator m_stack[2];

public:

    void swapBuffers()
    {
        m_curStack = (U32)!m_curStack;
    }

    void clearCurrentBuffer()
    {
        m_stack[m_curStack].clear();
    }

    void* alloc(U32 nBytes)
    {
        return m_stack[m_curStack].alloc(nBytes);
    }

    // ...
};

// ...

DoubleBufferedAllocator g_doubleBufAllocator;

// Main Game Loop
while (true)
{
    // Clear the single-frame allocator every frame as
    // before.
    g_singleFrameAllocator.clear();

    // Swap the active and inactive buffers of the double-
    // buffered allocator.
    g_doubleBufAllocator.swapBuffers();

    // Now clear the newly active buffer, leaving last
    // frame's buffer intact.
    g_doubleBufAllocator.clearCurrentBuffer();

    // ...

    // Allocate out of the current buffer, without
    // disturbing last frame's data. Only use this data
    // this frame or next frame. Again, this memory never

```

```
// needs to be freed.  
void* p = g_doubleBufAllocator.alloc(nBytes);  
  
// ...  
}
```

This kind of allocator is extremely useful for caching the results of asynchronous processing on a multicore game console like the Xbox 360, Xbox One, PlayStation 3 or PlayStation 4. On frame i , we can kick off an asynchronous job on one of the PS4's cores, for example, handing it the address of a destination buffer that has been allocated from our double-buffered allocator. The job runs and produces its results some time before the end of frame i , storing them into the buffer we provided. On frame $(i + 1)$, the buffers are swapped. The results of the job are now in the inactive buffer, so they will not be overwritten by any double-buffered allocations that might be made during this frame. As long as we use the results of the job before frame $(i + 2)$, our data won't be overwritten.

6.2.2 Memory Fragmentation

Another problem with dynamic heap allocations is that memory can become *fragmented* over time. When a program first runs, its heap memory is entirely free. When a block is allocated, a contiguous region of heap memory of the appropriate size is marked as “in use,” and the remainder of the heap remains free. When a block is freed, it is marked as such, and adjacent free blocks are merged into a single, larger free block. Over time, as allocations and deallocations of various sizes occur in random order, the heap memory begins to look like a patchwork of free and used blocks. We can think of the free regions as “holes” in the fabric of used memory. When the number of holes becomes large, and/or the holes are all relatively small, we say the memory has become *fragmented*. This is illustrated in Figure 6.4.

The problem with memory fragmentation is that allocations may fail even when there are enough free bytes to satisfy the request. The crux of the problem is that allocated memory blocks must always be *contiguous*. For example, in order to satisfy a request of 128 KiB, there must exist a free “hole” that is 128 KiB or larger. If there are two holes, each of which is 64 KiB in size, then enough bytes are available but the allocation fails because they are not *contiguous bytes*.

Memory fragmentation is not as much of a problem on operating systems that support *virtual memory*. A virtual memory system maps discontinuous blocks of physical memory known as *pages* into a *virtual address space*, in

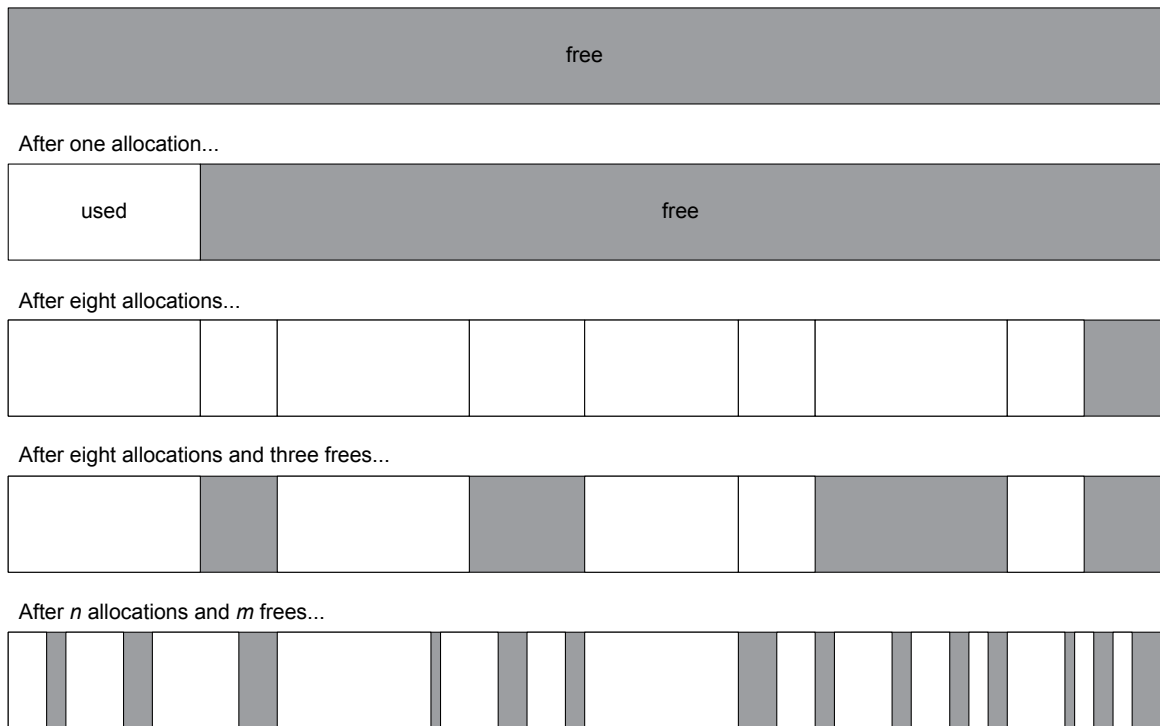


Figure 6.4. Memory fragmentation.

which the pages appear to the application to be contiguous. Stale pages can be swapped to the hard disk when physical memory is in short supply and reloaded from disk when they are needed. For a detailed discussion of how virtual memory works, see http://en.wikipedia.org/wiki/Virtual_memory. Most embedded systems cannot afford to implement a virtual memory system. While some modern consoles do technically support it, most console game engines still do not make use of virtual memory due to the inherent performance overhead.

6.2.2.1 Avoiding Fragmentation with Stack and Pool Allocators

The detrimental effects of memory fragmentation can be avoided by using stack and/or pool allocators.

- A stack allocator is impervious to fragmentation because allocations are always contiguous, and blocks must be freed in an order opposite to that in which they were allocated. This is illustrated in Figure 6.5.

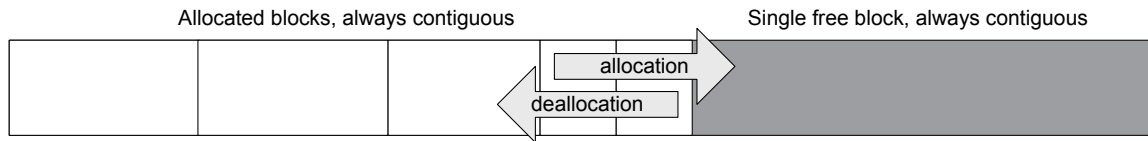


Figure 6.5. A stack allocator is free from fragmentation problems.

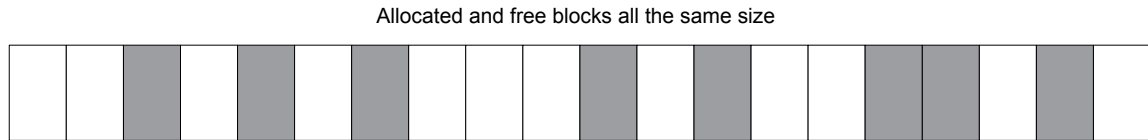


Figure 6.6. A pool allocator is not degraded by fragmentation.

- A pool allocator is also free from fragmentation problems. Pools *do* become fragmented, but the fragmentation never causes premature out-of-memory conditions as it does in a general-purpose heap. Pool allocation requests can never fail due to a lack of a large enough contiguous free block, because all of the blocks are exactly the same size. This is shown in Figure 6.6.

6.2.2.2 Defragmentation and Relocation

When differently sized objects are being allocated and freed in a random order, neither a stack-based allocator nor a pool-based allocator can be used. In such cases, fragmentation can be avoided by periodically *defragmenting* the heap. Defragmentation involves coalescing all of the free “holes” in the heap by shifting allocated blocks from higher memory addresses down to lower addresses (thereby shifting the holes up to higher addresses). One simple algorithm is to search for the first “hole” and then take the allocated block immediately above the hole and shift it down to the start of the hole. This has the effect of “bubbling up” the hole to a higher memory address. If this process is repeated, eventually all the allocated blocks will occupy a contiguous region of memory at the low end of the heap’s address space, and all the holes will have bubbled up into one big hole at the high end of the heap. This is illustrated in Figure 6.7.

The shifting of memory blocks described above is not particularly tricky to implement. What *is* tricky is accounting for the fact that we’re moving *allocated* blocks of memory around. If anyone has a *pointer* into one of these allocated blocks, then moving the block will invalidate the pointer.

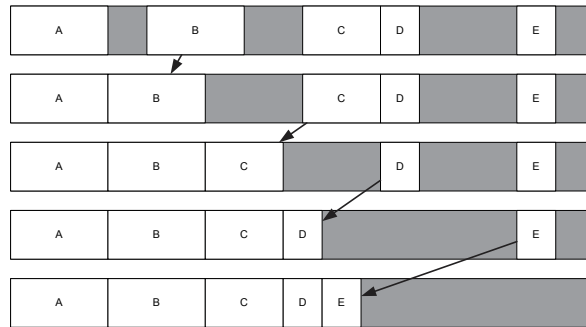


Figure 6.7. Defragmentation by shifting allocated blocks to lower addresses.

The solution to this problem is to patch any and all pointers into a shifted memory block so that they point to the correct new address after the shift. This procedure is known as *pointer relocation*. Unfortunately, there is no general-purpose way to *find* all the pointers that point into a particular region of memory. So if we are going to support memory defragmentation in our game engine, programmers must either carefully keep track of all the pointers manually so they can be relocated, or pointers must be abandoned in favor of something inherently more amenable to relocation, such as *smart pointers* or *handles*.

A smart pointer is a small class that contains a pointer and acts like a pointer for most intents and purposes. But because a smart pointer is a class, it can be coded to handle memory relocation properly. One approach is to arrange for all smart pointers to add themselves to a global linked list. Whenever a block of memory is shifted within the heap, the linked list of all smart pointers can be scanned, and each pointer that points into the shifted block of memory can be adjusted appropriately.

A handle is usually implemented as an index into a non-relocatable table, which itself contains the pointers. When an allocated block is shifted in memory, the handle table can be scanned and all relevant pointers found and updated automatically. Because the handles are just indices into the pointer table, their values never change no matter how the memory blocks are shifted, so the objects that use the handles are never affected by memory relocation.

Another problem with relocation arises when certain memory blocks cannot be relocated. For example, if you are using a third-party library that does not use smart pointers or handles, it's possible that any pointers into its data structures will not be relocatable. The best way around this problem is usually to arrange for the library in question to allocate its memory from a special buffer outside of the relocatable memory area. The other option is to simply

accept that some blocks will not be relocatable. If the number and size of the non-relocatable blocks are both small, a relocation system will still perform quite well.

It is interesting to note that all of Naughty Dog's engines have supported defragmentation. Handles are used wherever possible to avoid the need to relocate pointers. However, in some cases raw pointers cannot be avoided. These pointers are carefully tracked and relocated manually whenever a memory block is shifted due to defragmentation. A few of Naughty Dog's game object classes are not relocatable for various reasons. However, as mentioned above, this doesn't pose any practical problems, because the number of such objects is always very small, and their sizes are tiny when compared to the overall size of the relocatable memory area.

Amortizing Defragmentation Costs

Defragmentation can be a slow operation because it involves copying memory blocks. However, we needn't fully defragment the heap all at once. Instead, the cost can be amortized over many frames. We can allow up to N allocated blocks to be shifted each frame, for some small value of N like 8 or 16. If our game is running at 30 frames per second, then each frame lasts $1/30$ of a second (33 ms). So, the heap can usually be completely defragmented in less than one second without having any noticeable effect on the game's frame rate. As long as allocations and deallocations aren't happening at a faster rate than the defragmentation shifts, the heap will remain mostly defragmented at all times.

This approach is only valid when the size of each block is relatively small, so that the time required to move a single block does not exceed the time allotted to relocation each frame. If very large blocks need to be relocated, we can often break them up into two or more subblocks, each of which can be relocated independently. This hasn't proved to be a problem in Naughty Dog's engine, because relocation is only used for dynamic game objects, and they are never larger than a few kibibytes—and usually much smaller.

6.3 Containers

Game programmers employ a wide variety of collection-oriented data structures, also known as *containers* or *collections*. The job of a container is always the same—to house and manage zero or more data elements; however, the details of how they do this vary greatly, and each type of container has its pros and cons. Common container data types include, but are certainly not limited

to, the following.

- *Array*. An ordered, contiguous collection of elements accessed by index. The length of the array is usually statically defined at compile time. It may be multidimensional. C and C++ support these natively (e.g., `int a[5]`).
- *Dynamic array*. An array whose length can change dynamically at runtime (e.g., the C++ standard library's `std::vector`).
- *Linked list*. An ordered collection of elements not stored contiguously in memory but rather linked to one another via pointers (e.g., the C++ standard library's `std::list`).
- *Stack*. A container that supports the last-in-first-out (LIFO) model for adding and removing elements, also known as push/pop (e.g., `std::stack`).
- *Queue*. A container that supports the first-in-first-out (FIFO) model for adding and removing elements (e.g., `std::queue`).
- *Deque*. A double-ended queue—supports efficient insertion and removal at both ends of the array (e.g., `std::deque`).
- *Tree*. A container in which elements are grouped hierarchically. Each element (node) has zero or one parent and zero or more children. A tree is a special case of a DAG (see below).
- *Binary search tree (BST)*. A tree in which each node has at most two children, with an order property to keep the nodes sorted by some well-defined criteria. There are various kinds of binary search trees, including red-black trees, splay trees, AVL trees, etc.
- *Binary heap*. A binary tree that maintains itself in sorted order, much like a binary search tree, via two rules: the *shape property*, which specifies that the tree must be fully filled and that the last row of the tree is filled from left to right; and the *heap property*, which states that every node is, by some user-defined criterion, “greater than” or “equal to” all of its children.
- *Priority queue*. A container that permits elements to be added in any order and then removed in an order defined by some property of the elements themselves (i.e., their *priority*). A priority queue is typically implemented as a *heap* (e.g., `std::priority_queue`), but other implementations are possible. A priority queue is a bit like a list that stays sorted at all times, except that a priority queue only supports retrieval of the highest-priority element, and it is rarely implemented as a list under the hood.

- *Dictionary*. A table of key-value pairs. A value can be “looked up” efficiently given the corresponding key. A dictionary is also known as a *map* or *hash table*, although technically a hash table is just one possible implementation of a dictionary (e.g., `std::map`, `std::hash_map`).
- *Set*. A container that guarantees that all elements are unique according to some criteria. A set acts like a dictionary with only keys, but no values.
- *Graph*. A collection of nodes connected to one another by unidirectional or bidirectional pathways in an arbitrary pattern.
- *Directed acyclic graph (DAG)*. A collection of nodes with unidirectional (i.e., *directed*) interconnections, with no *cycles* (i.e., there is no nonempty path that starts and ends on the same node).

6.3.1 Container Operations

Game engines that make use of container classes inevitably make use of various commonplace algorithms as well. Some examples include:

- *Insert*. Add a new element to the container. The new element might be placed at the beginning of the list, or the end, or in some other location; or the container might not have a notion of ordering at all.
- *Remove*. Remove an element from the container; this may require a find operation (see below). However, if an iterator is available that refers to the desired element, it may be more efficient to remove the element using the iterator.
- *Sequential access (iteration)*. Accessing each element of the container in some “natural” predefined order.
- *Random access*. Accessing elements in the container in an arbitrary order.
- *Find*. Search a container for an element that meets a given criterion. There are all sorts of variants on the find operation, including finding in reverse, finding multiple elements, etc. In addition, different types of data structures and different situations call for different algorithms (see http://en.wikipedia.org/wiki/Search_algorithm).
- *Sort*. Sort the contents of a container according to some given criteria. There are many different sorting algorithms, including bubble sort, selection sort, insertion sort, quicksort and so on. (See http://en.wikipedia.org/wiki/Sorting_algorithm for details.)

6.3.2 Iterators

An iterator is a little class that “knows” how to efficiently visit the elements in a particular kind of container. It acts like an array index or pointer—it refers to one element in the container at a time, it can be advanced to the next element, and it provides some sort of mechanism for testing whether or not all elements in the container have been visited. As an example, the first of the following two code snippets iterates over a C-style array using a pointer, while the second iterates over a linked list using almost identical syntax.

```
void processArray(int container[], int numElements)
{
    int* pBegin = &container[0];
    int* pEnd = &container[numElements];

    for (int* p = pBegin; p != pEnd; p++)
    {
        int element = *p;
        // process element...
    }
}

void processList(std::list<int>& container)
{
    std::list<int>::iterator pBegin = container.begin();
    std::list<int>::iterator pEnd = container.end();

    for (auto p = pBegin; p != pEnd; ++p)
    {
        int element = *p;
        // process element...
    }
}
```

The key benefits to using an iterator over attempting to access the container’s elements directly are as follows:

- Direct access would break the container class’ encapsulation. An iterator, on the other hand, is typically a *friend* of the container class, and as such it can iterate efficiently without exposing any implementation details to the outside world. (In fact, most good container classes hide their internal details and cannot be iterated over *without* an iterator.)
- An iterator can simplify the process of iterating. Most iterators act like array indices or pointers, so a simple loop can be written in which the

iterator is incremented and compared against a terminating condition—even when the underlying data structure is arbitrarily complex. For example, an iterator can make an in-order depth-first tree traversal look no more complex than a simple array iteration.

6.3.2.1 Preincrement versus Postincrement

Notice in the `processArray()` example that we are using C++’s *postincrement* operator, `p++`, rather than the *preincrement* operator, `++p`. This is a subtle but sometimes important optimization. The preincrement operator increments the contents of the variable *before* its (now modified) value is used in the expression. The postincrement operator increments the contents of the variable *after* it has been used. This means that writing `++p` introduces a *data dependency* into your code—the CPU must wait for the increment operation to be completed before its value can be used in the expression. On a deeply pipelined CPU, this introduces a *stall*. On the other hand, with `p++` there is no data dependency. The value of the variable can be used immediately, and the increment operation can happen later or in parallel with its use. Either way, no stall is introduced into the pipeline.

Of course, within the “update” expression of a `for` loop, there should be no difference between pre- and postincrement. This is because any good compiler will recognize that the *value* of the variable isn’t used in `update_expr`. But in cases where the value *is* used, postincrement is preferable because it doesn’t introduce a stall in the CPU’s pipeline.

It can be wise to make an exception to this little rule of thumb for classes with *overloaded* increment operators, as is common practice in *iterator* classes. By definition, the postincrement operator must return an unmodified *copy* of the object on which it is called. Depending on the size and complexity of the data members of the class, the added cost of copying the iterator can tip the scales toward a preference for preincrement when using such classes in performance-critical loops. (Preincrement isn’t necessarily better than postincrement in such a simple example as the function `processList()` shown above, but I’ve implemented it with preincrement to highlight the difference.)

6.3.3 Algorithmic Complexity

The choice of which container type to use for a given application depends upon the performance and memory characteristics of the container being considered. For each container type, we can determine the theoretical performance of common operations such as insertion, removal, find and sort.

We usually indicate the amount of time T that an operation is expected to

take as a function of the number of elements n in the container:

$$T = f(n).$$

Rather than try to find the exact function f , we concern ourselves only with finding the overall *order* of the function. For example, if the actual theoretical function were any of the following,

$$T = 5n^2 + 17,$$

$$T = 102n^2 + 50n + 12,$$

$$T = \frac{1}{2}n^2,$$

we would, in all cases, simplify the expression down to its most relevant term—in this case n^2 . To indicate that we are only stating the *order* of the function, not its exact equation, we use “big O” notation and write

$$T = O(n^2).$$

The order of an algorithm can usually be determined via an inspection of the pseudocode. If the algorithm’s execution time is not dependent upon the number of elements in the container at all, we say it is $O(1)$ (i.e., it completes in *constant time*). If the algorithm performs a *loop* over the elements in the container and visits each element once, such as in a linear search of an unsorted list, we say the algorithm is $O(n)$. If two loops are nested, each of which potentially visits each node once, then we say the algorithm is $O(n^2)$. If a divide-and-conquer approach is used, as in a *binary search* (where half of the list is eliminated at each step), then we would expect that only $\lfloor \log_2(n) + 1 \rfloor$ elements will actually be visited by the algorithm in the worst case, and hence we refer to it as an $O(\log n)$ operation. If an algorithm executes a subalgorithm n times, and the subalgorithm is $O(\log n)$, then the resulting algorithm would be $O(n \log n)$.

To select an appropriate container class, we should look at the operations that we expect to be most common, then select the container whose performance characteristics for those operations are most favorable. The most common orders you’ll encounter are listed here from fastest to slowest: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^k)$ for $k > 2$.

We should also take the memory layout and usage characteristics of our containers into account. For example, an array (e.g., `int a[5]` or `std::vector`) stores its elements *contiguously* in memory and requires *no overhead storage* for anything other than the elements themselves. (Note that

a *dynamic* array does require a small fixed overhead.) On the other hand, a linked list (e.g., `std::list`) wraps each element in a “link” data structure that contains a pointer to the next element and possibly also a pointer to the previous element, for a total of up to 16 bytes of overhead per element on a 64-bit machine. Also, the elements in a linked list need not be contiguous in memory and often aren’t. A contiguous block of memory is usually much more cache-friendly than a set of disparate memory blocks. Hence, for high-speed algorithms, arrays are usually better than linked lists in terms of cache performance (unless the nodes of the linked list are themselves allocated from a small, contiguous block of memory). But a linked list is better for situations in which speed of inserting and removing elements is of prime importance.

6.3.4 Building Custom Container Classes

Many game engines provide their own custom implementations of the common container data structures. This practice is especially prevalent in console game engines and games targeted at mobile phone and PDA platforms. The reasons for building these classes yourself include:

- *Total control.* You control the data structure’s memory requirements, the algorithms used, when and how memory is allocated, etc.
- *Opportunities for optimization.* You can optimize your data structures and algorithms to take advantage of hardware features specific to the console(s) you are targeting; or fine-tune them for a particular application within your engine.
- *Customizability.* You can provide custom algorithms not prevalent in the C++ standard library or third-party libraries like Boost (for example, searching for the n most relevant elements in a container, instead of just the single most relevant).
- *Elimination of external dependencies.* Since you built the software yourself, you are not beholden to any other company or team to maintain it. If problems arise, they can be debugged and fixed immediately rather than waiting until the next release of the library (which might not be until after you have shipped your game!)
- *Control over concurrent data structures.* When you write your own container classes, you have full control over the means by which they are protected against concurrent access on a multithreaded or multicore system. For example, on the PS4, Naughty Dog uses lightweight “spin lock” mutexes for the majority of our concurrent data structures, because they work well with our fiber-based job scheduling system. A third-party container library might not have given us this kind of flexibility.

We cannot cover all possible data structures here, but let's look at a few common ways in which game engine programmers tend to tackle containers.

6.3.4.1 To Build or Not to Build

We will not discuss the details of how to implement all of these data types and algorithms here—a plethora of books and online resources are available for that purpose. However, we *will* concern ourselves with the question of where to obtain implementations of the types and algorithms that you need. As game engine designers, we have basically three choices:

1. Build the needed data structures manually.
2. Make use of the STL-style containers provided by the C++ standard library.
3. Rely on a third-party library such as Boost (<http://www.boost.org>).

The C++ standard library and third-party libraries like Boost are attractive options, because they provide a rich and powerful set of container classes covering pretty much every type of data structure imaginable. In addition, these packages provide a powerful suite of template-based *generic algorithms*—implementations of common algorithms, such as finding an element in a container, which can be applied to virtually any type of data object. However, these implementations may not be appropriate for some kinds of game engines. Let's take a moment to investigate some of the pros and cons of each approach.

The C++ Standard Library

The benefits of the C++ standard library's STL-style container classes include:

- They offer a rich set of features.
- Their implementations are robust and fully portable.

However, these container classes also have some drawbacks, including:

- The header files are cryptic and difficult to understand (although the documentation is quite good).
- General-purpose container classes are often slower than a data structure that has been crafted specifically to solve a particular problem.
- A generic container may consume more memory than a custom-designed data structure.
- The C++ standard library does a lot of dynamic memory allocation, and it's sometimes challenging to control its appetite for memory in a way that is suitable for high-performance, memory-limited console games.

- The templated allocator system provided by the standard C++ library isn't flexible enough to allow these containers to be used with certain kinds of memory allocators, such as stack-based allocators (see Section 6.2.1.1).

The *Medal of Honor: Pacific Assault* engine for the PC made heavy use of what was known at the time as the standard template library (STL). And while *MOHPA* did have its share of frame rate problems, the team was able to work around the performance problems caused by STL (primarily by carefully limiting and controlling its use). OGRE, the popular object-oriented rendering library that we use for some of the examples in this book, also makes heavy use of STL-style containers. However, at Naughty Dog we prohibit the use of STL containers in runtime game code (although we do permit their use in offline tools code). Your mileage may vary: Using the STL-style containers provided by the C++ standard library on a game engine project is certainly feasible, but it should be used with care.

Boost

The Boost project was started by members of the C++ Standards Committee Library Working Group, but it is now an open source project with many contributors from across the globe. The aim of the project is to produce libraries that extend and work together with the standard C++ library, for both commercial and non-commercial use. Many of the Boost libraries have already been included in the C++ standard library as of C++11, and more components are included in the Standards Committee's Library Technical Report (TR2), which is a step toward becoming part of a future C++ standard. Here is a brief summary of what Boost brings to the table:

- Boost provides a lot of useful facilities not available in the C++ standard library.
- In some cases, Boost provides alternatives or work-arounds for problems with the design or implementation of some classes in the C++ standard library.
- Boost does a great job of handling some very complex problems, like smart pointers. (Bear in mind that smart pointers are complex beasts, and they can be performance hogs. Handles are usually preferable; see Section 16.5 for details.)
- The Boost libraries' documentation is usually excellent. Not only does the documentation explain what each library does and how to use it, but

in most cases it also provides an excellent in-depth discussion of the design decisions, constraints and requirements that went into constructing the library. As such, reading the Boost documentation is a great way to learn about the principles of software design.

If you are already using the C++ standard library, then Boost can serve as an excellent extension and/or alternative to many of its features. However, be aware of the following caveats:

- Most of the core Boost classes are templates, so all that one needs in order to use them is the appropriate set of header files. However, some of the Boost libraries build into rather large .lib files and may not be feasible for use in very small-scale game projects.
- While the worldwide Boost community is an excellent support network, the Boost libraries come with no guarantees. If you encounter a bug, it will ultimately be your team's responsibility to work around it or fix it.
- The Boost libraries are distributed under the Boost Software License. Read the license information (http://www.boost.org/more/license_info.html) carefully to be sure it is right for your engine.

Folly

Folly is an open source library developed by Andrei Alexandrescu and the engineers at Facebook. Its goal is to extend the standard C++ library and the Boost library (rather than to compete with these libraries), with an emphasis on ease of use and the development of high-performance software. You can read about it by searching online for the article entitled "Folly: The Facebook Open Source Library" which is hosted at <https://www.facebook.com/>. The library itself is available on GitHub here: <https://github.com/facebook/folly>.

Loki

There is a rather esoteric branch of C++ programming known as *template meta-programming*. The core idea is to use the compiler to do a lot of the work that would otherwise have to be done at runtime by exploiting the template feature of C++ and in effect "tricking" the compiler into doing things it wasn't originally designed to do. This can lead to some startlingly powerful and useful programming tools.

By far the most well-known and probably most powerful template meta-programming library for C++ is Loki, a library designed and written by Andrei

Alexandrescu (whose home page is at <http://www.erdani.org>). The library can be obtained from SourceForge at <http://loki-lib.sourceforge.net>.

Loki is extremely powerful; it is a fascinating body of code to study and learn from. However, its two big weaknesses are of a practical nature: (a) its code can be daunting to read and use, much less truly understand, and (b) some of its components are dependent upon exploiting “side-effect” behaviors of the compiler that require careful customization in order to be made to work on new compilers. So Loki can be somewhat tough to use, and it is not as portable as some of its “less-extreme” counterparts. Loki is not for the faint of heart. That said, some of Loki’s concepts such as *policy-based programming* can be applied to any C++ project, even if you don’t use the Loki library per se. I highly recommend that all software engineers read Andrei’s ground-breaking book, *Modern C++ Design* [3], from which the Loki library was born.

6.3.5 Dynamic Arrays and Chunky Allocation

Fixed size C-style arrays are used quite a lot in game programming, because they require no memory allocation, are contiguous and hence cache-friendly, and support many common operations such as appending data and searching very efficiently.

When the size of an array cannot be determined a priori, programmers tend to turn either to *linked lists* or *dynamic arrays*. If we wish to maintain the performance and memory characteristics of fixed-length arrays, then the dynamic array is often the data structure of choice.

The easiest way to implement a dynamic array is to allocate an n -element buffer initially and then *grow* the list only if an attempt is made to add more than n elements to it. This gives us the favorable characteristics of a fixed size array but with no upper bound. Growing is implemented by allocating a new larger buffer, copying the data from the original buffer into the new buffer, and then freeing the original buffer. The size of the buffer is increased in some orderly manner, such as adding n to it on each grow, or doubling it on each grow. Most of the implementations I’ve encountered never shrink the array, only grow it (with the notable exception of clearing the array to zero size, which might or might not free the buffer). Hence the size of the array becomes a sort of “high water mark.” The `std::vector` class works in this manner.

Of course, if you can establish a high water mark for your data, then you’re probably better off just allocating a single buffer of that size when the engine starts up. Growing a dynamic array can be incredibly costly due to reallocation and data copying costs. The impact of these things depends on the sizes of the

buffers involved. Growing can also lead to fragmentation when discarded buffers are freed. So, as with all data structures that allocate memory, caution must be exercised when working with dynamic arrays. Dynamic arrays are probably best used during development, when you are as yet unsure of the buffer sizes you'll require. They can always be converted into fixed size arrays once suitable memory budgets have been established.

6.3.6 Dictionaries and Hash Tables

A dictionary is a table of key-value pairs. A value in the dictionary can be looked up quickly, given its key. The keys and values can be of any data type. This kind of data structure is usually implemented either as a binary search tree or as a hash table.

In a binary tree implementation, the key-value pairs are stored in the nodes of the binary tree, and the tree is maintained in key-sorted order. Looking up a value by key involves performing an $O(\log n)$ binary search.

In a hash table implementation, the values are stored in a fixed size table, where each slot in the table represents one or more keys. To insert a key-value pair into a hash table, the key is first converted into integer form via a process known as *hashing* (if it is not already an integer). Then an *index* into the hash table is calculated by taking the hashed key *modulo* the size of the table. Finally, the key-value pair is stored in the slot corresponding to that index. Recall that the *modulo* operator (`%` in C/C++) finds the remainder of dividing the integer key by the table size. So if the hash table has five slots, then a key of 3 would be stored at index 3 ($3 \% 5 == 3$), while a key of 6 would be stored at index 1 ($6 \% 5 == 1$). Finding a key-value pair is an $O(1)$ operation in the absence of collisions.

6.3.6.1 Collisions: Open and Closed Hash Tables

Sometimes two or more keys end up occupying the same slot in the hash table. This is known as a *collision*. There are two basic ways to *resolve* a collision, giving rise to two different kinds of hash tables:

- *Open*. In an open hash table (see Figure 6.8), collisions are resolved by simply storing more than one key-value pair at each index, usually in the form of a linked list. This approach is easy to implement and imposes no upper bound on the number of key-value pairs that can be stored. However, it does require memory to be allocated dynamically whenever a new key-value pair is added to the table.
- *Closed*. In a closed hash table (see Figure 6.9), collisions are resolved via a process of *probing* until a vacant slot is found. ("Probing" means ap-

plying a well-defined algorithm to search for a free slot.) This approach is a bit more difficult to implement, and it imposes an upper limit on the number of key-value pairs that can reside in the table (because each slot can hold only one key-value pair). But the main benefit of this kind of hash table is that it uses up a fixed amount of memory and requires no dynamic memory allocation. Therefore, it is often a good choice in a console engine.

Confusingly, closed hash tables are sometimes said to use *open addressing*, while open hash tables are said to use an addressing method known as *chaining*, so named due to the linked lists at each slot in the table.

6.3.6.2 Hashing

Hashing is the process of turning a key of some arbitrary data type into an integer, which can be used modulo the table size as an index into the table. Mathematically, given a key k , we want to generate an integer hash value h using the hash function H and then find the index i into the table as follows:

$$h = H(k),$$

$$i = h \bmod N,$$

where N is the number of slots in the table, and the symbol *mod* represents the *modulo* operation, i.e., finding the remainder of the quotient h/N .

If the keys are unique integers, the hash function can be the identity function, $H(k) = k$. If the keys are unique 32-bit floating-point numbers, a hash function might simply reinterpret the bit pattern of the 32-bit float as if it were a 32-bit integer.

```
U32 hashFloat(float f)
{
```

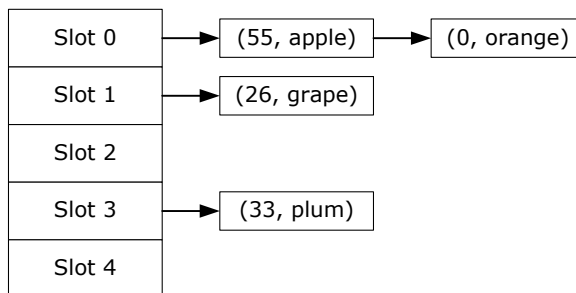


Figure 6.8. An open hash table.

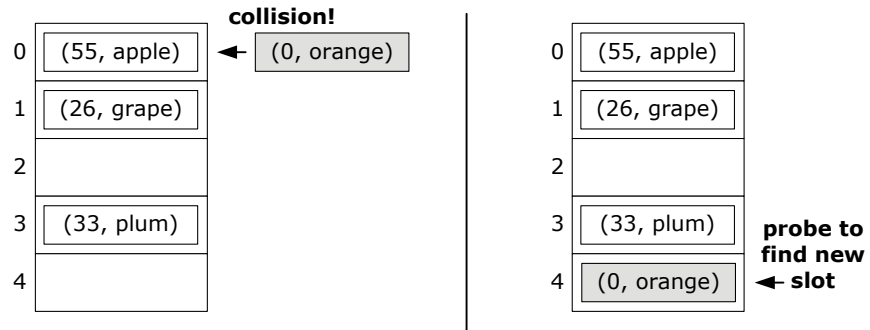


Figure 6.9. A closed hash table.

```

union
{
    float m_asFloat;
    U32   m_asU32;
} u;

u.m_asFloat = f;
return u.m_asU32;
}

```

If the key is a string, we can employ a *string hashing function*, which combines the ASCII or UTF codes of all the characters in the string into a single 32-bit integer value.

The *quality* of the hashing function $H(k)$ is crucial to the efficiency of the hash table. A “good” hashing function is one that distributes the set of all valid keys evenly across the table, thereby minimizing the likelihood of collisions. A hash function must also be reasonably quick to calculate, and *deterministic* in the sense that it must produce the exact same output every time it is called with an identical input.

Strings are probably the most prevalent type of key you’ll encounter, so it’s particularly helpful to know a “good” string hashing function. Table 6.1 lists a number of well-known hashing algorithms, their throughput ratings (based on benchmark measurements and then converted into a rating of Low, Medium or High) and their score on the SMHasher test (<https://github.com/aappleby/smhasher>). Please note that the relative throughputs listed in the table are for rough comparison purposes only. Many factors contribute to the throughput of a hash function, including the hardware on which it is run and the properties of the input data. Cryptographic hashes are deliberately slow, as their focus is on producing a hash that is extremely unlikely to collide with the hashes of other input strings, and for which the task of determining

Name	Throughput	Score	Cryptographic?
xxHash	High	10	No
MurmurHash 3a	High	10	No
SBox	Medium	9	No‡
Lookup3	Medium	9	No
CityHash64	Medium	10	No
CRC32	Low	9	No
MD5-32	Low	10	Yes
SHA1-32	Low	10	Yes

Table 6.1. Comparison of well-known hashing algorithms in terms of their relative throughput and their scores on the SMHasher test. ‡Note that SBox is not itself a cryptographic hash, but it is one component of the symmetric key algorithms used in cryptography.

a string that would produce a given hash value is extremely computationally difficult.

For more information on hash functions, see the excellent article by Paul Hsieh available at <http://www.azillionmonkeys.com/qed/hash.html>.

6.3.6.3 Implementing a Closed Hash Table

In a closed hash table, the key-value pairs are stored directly in the table, rather than in a linked list at each table entry. This approach allows the programmer to define a priori the exact amount of memory that will be used by the hash table. A problem arises when we encounter a *collision*—two keys that end up wanting to be stored in the same slot in the table. To address this, we use a process known as *probing*.

The simplest approach is *linear probing*. Imagine that our hashing function has yielded a table index of i , but that slot is already occupied; we simply try slots $(i + 1)$, $(i + 2)$ and so on until an empty slot is found (wrapping around to the start of the table when $i = N$). Another variation on linear probing is to alternate searching forwards and backwards, $(i + 1)$, $(i - 1)$, $(i + 2)$, $(i - 2)$ and so on, making sure to modulo the resulting indices into the valid range of the table.

Linear probing tends to cause key-value pairs to “clump up.” To avoid these clusters, we can use an algorithm known as *quadratic probing*. We start at the occupied table index i and use the sequence of probes $i_j = (i \pm j^2)$ for $j = 1, 2, 3, \dots$. In other words, we try $(i + 1^2)$, $(i - 1^2)$, $(i + 2^2)$, $(i - 2^2)$ and so

on, remembering to always modulo the resulting index into the valid range of the table.

When using closed hashing, it is a good idea to make your table size a *prime number*. Using a prime table size in conjunction with quadratic probing tends to yield the best coverage of the available table slots with minimal clustering. See <http://stackoverflow.com/questions/1145217/why-should-hash-functions-use-a-prime-number-modulus> for a good discussion of why prime hash table sizes are preferable.

6.3.6.4 Robin Hood Hashing

Robin Hood hashing is another probing method for closed hash tables that has gained popularity recently. This probing scheme improves the performance of a closed hash table, even when the table is nearly full. For a good discussion of how Robin Hood hashing works, see <https://www.sebastiansylvan.com/post/robin-hood-hashing-should-be-your-default-hash-table-implementation/>.

6.4 Strings

Strings are ubiquitous in almost every software project, and game engines are no exception. On the surface, the string may seem like a simple, fundamental data type. But, when you start using strings in your projects, you will quickly discover a wide range of design issues and constraints, all of which must be carefully accounted for.

6.4.1 The Problem with Strings

The most fundamental question about strings is how they should be stored and managed in your program. In C and C++, strings aren't even an atomic type—they are implemented as *arrays of characters*. The variable length of strings means we either have to hard-code limitations on the sizes of our strings, or we need to dynamically allocate our string buffers.

Another big string-related problem is that of *localization*—the process of adapting your software for release in other languages. This is also known as *internationalization*, or I18N for short. Any string that you display to the user in English must be translated into whatever languages you plan to support. (Strings that are used internally to the program but are never displayed to the user are exempt from localization, of course.) This not only involves making sure that you can represent all the character glyphs of all the languages you plan to support (via an appropriate set of fonts), but it also means ensuring that your game can handle different text orientations. For example, traditional

Chinese text is oriented vertically instead of horizontally (although modern Chinese and Japanese are commonly written horizontally and left-to-right), and some languages like Hebrew read right-to-left. Your game also needs to gracefully deal with the possibility that a translated string will be either much longer or much shorter than its English counterpart.

Finally, it's important to realize that strings are used internally within a game engine for things like resource file names and object ids. For example, when a game designer lays out a level, it's highly convenient to permit him or her to identify the objects in the level using meaningful names, like "Player-Camera," "enemy-tank-01" or "explosionTrigger."

How our engine deals with these internal strings often has pervasive ramifications on the performance of the game. This is because strings are inherently expensive to work with at runtime. Comparing or copying ints or floats can be accomplished via simple machine language instructions. On the other hand, comparing strings requires an $O(n)$ scan of the character arrays using a function like `strcmp()` (where n is the length of the string). Copying a string requires an $O(n)$ memory copy, not to mention the possibility of having to dynamically allocate the memory for the copy. During one project I worked on, we profiled our game's performance only to discover that `strcmp()` and `strcpy()` were the top two most expensive functions! By eliminating unnecessary string operations and using some of the techniques outlined in this section, we were able to all but eliminate these functions from our profile and increase the game's frame rate significantly. (I've heard similar stories from developers at a number of different studios.)

6.4.2 String Classes

Many C++ programmers prefer to use a *string class*, such as the C++ standard library's `std::string`, rather than deal directly with character arrays. Such classes can make working with strings much more convenient for the programmer. However, a string class can have hidden costs that are difficult to see until the game is profiled. For example, passing a string to a function using a C-style character array is fast because the address of the first character is typically passed in a hardware register. On the other hand, passing a string object might incur the overhead of one or more copy constructors, if the function is not declared or used properly. Copying strings might involve dynamic memory allocation, causing what looks like an innocuous function call to end up costing literally thousands of machine cycles.

Because of the myriad issues with string classes, I generally prefer to avoid them in runtime game code. However, if you feel a strong urge to use a string

class, make sure you pick or implement one that has acceptable runtime performance characteristics—and be sure all programmers that use it are aware of its costs. Know your string class: Does it treat all string buffers as read-only? Does it utilize the *copy on write* optimization? (See <http://en.wikipedia.org/wiki/Copy-on-write>.) In C++11, does it provide a move constructor? Does it own the memory associated with the string, or can it reference memory that it does not own? (See http://www.boost.org/doc/libs/1_57_0/libs/utility/doc/html/string_ref.html for more on the issue of memory ownership in string classes.) As a rule of thumb, always pass string objects by reference, never by value (as the latter often incurs string-copying costs). Profile your code early and often to ensure that your string class isn't becoming a major source of lost frame rate!

One situation in which a specialized string class does seem justifiable to me is when storing and managing file system paths. Here, a hypothetical `Path` class could add significant value over a raw C-style character array. For example, it might provide functions for extracting the filename, file extension or directory from the path. It might hide operating system differences by automatically converting Windows-style backslashes to UNIX-style forward slashes or some other operating system's path separator. Writing a `Path` class that provides this kind of functionality in a cross-platform way could be highly valuable within a game engine context. (See Section 7.1.1.4 for more details on this topic.)

6.4.3 Unique Identifiers

The *objects* in any virtual game world need to be uniquely identified in some way. For example, in Pac Man we might encounter game objects named "pac_man," "blinky," "pinky," "inky" and "clyde." Unique object identifiers allow game designers to keep track of the myriad objects that make up their game worlds and also permit those objects to be found and operated on at runtime by the engine. In addition, the *assets* from which our game objects are constructed—meshes, materials, textures, audio clips, animations and so on—all need unique identifiers as well.

Strings seem like a natural choice for such identifiers. Assets are often stored in individual files on disk, so they can usually be identified uniquely by their file paths, which of course are strings. And game objects are created by game designers, so it is natural for them to assign their objects understandable string names, rather than have to remember integer object indices, or 64- or 128-bit globally unique identifiers (GUIDs). However, the speed with which *comparisons* between unique identifiers can be made is of paramount impor-

tance in a game, so `strcmp()` simply doesn't cut it. We need a way to have our cake and eat it too—a way to get all the descriptiveness and flexibility of a string, but with the speed of an integer.

6.4.3.1 Hashed String Ids

One good solution is to *hash* our strings. As we've seen, a hash function maps a string onto a semi-unique integer. String hash codes can be compared just like any other integers, so comparisons are fast. If we store the actual strings in a hash table, then the original string can always be recovered from the hash code. This is useful for debugging purposes and to permit hashed strings to be displayed on-screen or in log files. Game programmers sometimes use the term *string id* to refer to such a hashed string. The Unreal engine uses the term *name* instead (implemented by class `FName`).

As with any hashing system, *collisions* are a possibility (i.e., two different strings might end up with the same hash code). However, with a suitable hash function, we can all but guarantee that collisions will not occur for all reasonable input strings we might use in our game. After all, a 32-bit hash code represents more than four billion possible values. So, if our hash function does a good job of distributing strings evenly throughout this very large range, we are unlikely to collide. At Naughty Dog, we started out using a variant of the CRC-32 algorithm to hash our strings, and we encountered only a handful of collisions during many years of development on *Uncharted* and *The Last of Us*. And when a collision did occur, fixing it was a simple matter of slightly altering one of the strings (e.g., append a "2" or a "b" to one of the strings, or use a totally different but synonymous string). That being said, Naughty Dog has moved to a 64-bit hashing function for *The Last of Us Part II* and all of our future game titles; this should essentially eliminate the possibility of hash collisions, given the quantity and typical lengths of the strings we use in any one game.

6.4.3.2 Some Implementation Ideas

Conceptually, it's easy enough to run a hash function on your strings in order to generate string ids. Practically speaking, however, it's important to consider *when* the hash will be calculated. Most game engines that use string ids do the hashing at runtime. At Naughty Dog, we permit runtime hashing of strings, but we also use C++11's *user-defined literals* feature to transform the syntax `"any_string"_sid` directly into a hashed integer value at compile time. This permits string ids to be used anywhere that an integer manifest constant can be used, including the constant `case` labels of a `switch` statement. (The result of a function call that generates a string id at runtime is not

a constant, so it cannot be used as a case label.)

The process of generating a string id from a string is sometimes called *interning* the string, because in addition to hashing it, the string is typically also added to a global string table. This allows the original string to be recovered from the hash code later. You may also want your tools to be capable of hashing strings into string ids. That way, when the tool generates data for consumption by your engine, the strings will already have been hashed.

The main problem with interning a string is that it is a slow operation. The hashing function must be run on the string, which can be an expensive proposition, especially when a large number of strings are being interned. In addition, memory must be allocated for the string, and it must be copied into the lookup table. As a result (if you are not generating string ids at compile-time), it is usually best to intern each string only once and save off the result for later use. For example, it would be preferable to write code like this because the latter implementation causes the strings to be unnecessarily re-interned every time the function `f()` is called.

```
static StringId    sid_foo = internString("foo");
static StringId    sid_bar = internString("bar");

// ...

void f(StringId id)
{
    if (id == sid_foo)
    {
        // handle case of id == "foo"
    }
    else if (id == sid_bar)
    {
        // handle case of id == "bar"
    }
}
```

The following approach is less efficient:

```
void f(StringId id)
{
    if (id == internString("foo"))
    {
        // handle case of id == "foo"
    }
}
```

```

        else if (id == internString("bar"))
        {
            // handle case of id == "bar"
        }
    }

```

Here's one possible implementation of `internString()`.

stringid.h

```

typedef U32 StringId;

extern StringId internString(const char* str);

```

stringid.cpp

```

static HashTable<StringId, const char*> gStringIdTable;

StringId internString(const char* str)
{
    StringId sid = hashCrc32(str);

    HashTable<StringId, const char*>::iterator it
        = gStringIdTable.find(sid);

    if (it == gStringTable.end())
    {
        // This string has not yet been added to the
        // table. Add it, being sure to copy it in case
        // the original was dynamically allocated and
        // might later be freed.
        gStringTable[sid] = strdup(str);
    }

    return sid;
}

```

Another idea employed by the Unreal Engine is to wrap the string id and a pointer to the corresponding C-style character array in a tiny class. In the Unreal Engine, this class is called `FName`. At Naughty Dog we do the same and wrap our string ids in a `StringId` class. We define a macro so that `SID("any_string")` produces an instance of this class, with its hashed value produced by our user-defined string literal syntax `"any_string"_sid`.

Using Debug Memory for Strings

When using string ids, the strings themselves are only kept around for human consumption. When you ship your game, you almost certainly won't need the strings—the game itself should only ever use the ids. As such, it's a good idea to store your string table in a region of memory that won't exist in the retail game. For example, a PS3 development kit has 256 MiB of retail memory, plus an additional 256 MiB of “debug” memory that is not available on a retail unit. If we store our strings in debug memory, we needn't worry about their impact on the memory footprint of the final shipping game. (We just need to be careful never to write production code that depends on the strings being available!)

6.4.4 Localization

Localization of a game (or any software project) is a big undertaking. It is a task best handled by planning for it from day one and accounting for it at every step of development. However, this is not done as often as we all would like. Here are some tips that should help you plan your game engine project for localization. For an in-depth treatment of software localization, see [34].

6.4.4.1 Unicode

The problem for most English-speaking software developers is that they are trained from birth (or thereabouts!) to think of strings as arrays of eight-bit ASCII character codes (i.e., characters following the ANSI standard). ANSI strings work great for a language with a simple alphabet, like English. But, they just don't cut it for languages with complex alphabets containing a great many more characters, sometimes totally different glyphs than English's 26 letters. To address the limitations of the ANSI standard, the Unicode character set system was devised.

The basic idea behind Unicode is to assign every character or glyph from every language in common use around the globe to a unique hexadecimal code known as a *code point*. When storing a string of characters in memory, we select a particular *encoding*—a specific means of representing the Unicode code points for each character—and following those rules, we lay down a sequence of bits in memory that represent the string. UTF-8 and UTF-16 are two common encodings. You should select the specific encoding standard that best suits your needs.

Please set down this book right now and read the article entitled, “The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!)” by Joel Spolsky. You can find it here: <http://www.joelonsoftware.com/articles/Unicode.html>. (Once you've done that, please pick up the book again!)

UTF-32

The simplest Unicode encoding is UTF-32. In this encoding, each Unicode code point is encoded into a 32-bit (4-byte) value. This encoding wastes a lot of space, for two reasons: First, most strings in Western European languages do not use any of the highest-valued code points, so an average of at least 16 bits (2 bytes) is usually wasted per character. Second, the highest Unicode code point is 0x10FFFF, so even if we wanted to create a string that uses every possible Unicode glyph, we'd still only need 21 bits per character, not 32.

That said, UTF-32 does have simplicity in its favor. It is a *fixed-length* encoding, meaning that every character occupies the same number of bits in memory (32 bits to be precise). As such, we can determine the length of any UTF-32 string by taking its length in bytes and dividing by four.

UTF-8

In the UTF-8 encoding scheme, the code points for each character in a string are stored using eight-bit (one-byte) granularity, but some code points occupy more than one byte. Hence the number of bytes occupied by a UTF-8 character string is not necessarily the length of the string in characters. This is known as a *variable-length* encoding, or a *multibyte character set* (MBCS), because each character in a string may take one or more bytes of storage.

One of the big benefits of the UTF-8 encoding is that it is backwards-compatible with the ANSI encoding. This works because the first 127 Unicode code points correspond numerically to the old ANSI character codes. This means that every ANSI character will be represented by exactly one byte in UTF-8, and a string of ANSI characters can be interpreted as a UTF-8 string without modification.

To represent higher-valued code points, the UTF-8 standard uses multibyte characters. Each multibyte character starts with a byte whose most-significant bit is 1 (i.e., its value lies in the range 128–255, inclusive). Such high-valued bytes will never appear in an ANSI string, so there is no ambiguity when distinguishing between single-byte characters and multibyte characters.

UTF-16

The UTF-16 encoding employs a somewhat simpler, albeit more expensive approach. Each character in a UTF-16 string is represented by either one or two 16-bit values. The UTF-16 encoding is known as a wide character set (WCS) because each character is at least 16 bits wide, instead of the eight bits used by

“regular” ANSI chars and their UTF-8 counterparts.

In UTF-16, the set of all possible Unicode code points is divided into 17 *planes* containing 2^{16} code points each. The first plane is known as the *basic multilingual plane* (BMP). It contains the most commonly used code points across a wide range of languages. As such, many UTF-16 strings can be represented entirely by code points within the first plane, meaning that each character in such a string is represented by only *one* 16-bit value. However, if a character from one of the other planes (known as *supplementary planes*) is required in a string, it is represented by two consecutive 16-bit values.

The UCS-2 (2-byte universal character set) encoding is a limited subset of the UTF-16 encoding, utilizing *only* the basic multilingual page. As such, it cannot represent characters whose Unicode code points are numerically higher than 0xFFFF. This simplifies the format, because every character is guaranteed to occupy exactly 16 bits (two bytes). In other words, UCS-2 is a *fixed-length* character encoding, while in general UTF-8 and UTF-16 are *variable-length* encodings.

If we know a priori that a UTF-16 string only utilizes code points from the BMP (or if we are dealing with a UCS-2 encoded string), we can determine the number of characters in the string by simply dividing the number of bytes by two. Of course, if supplemental planes are used in a UTF-16 string, this simple “trick” no longer works.

Note that a UTF-16 encoding can be little-endian or big-endian (see Section 3.3.2.1), depending on the native endianness of your target CPU. When storing UTF-16 text on-disc, it’s common to precede the text data with a *byte order mark* (BOM) indicating whether the individual 16-bit characters are stored in little- or big-endian format. (This is true of UTF-32 encoded string data as well, of course.)

6.4.4.2 char versus wchar_t

The standard C/C++ library defines two data types for dealing with character strings—`char` and `wchar_t`. The `char` type is intended for use with legacy ANSI strings and with multibyte character sets (MBCS), including (but not limited to) UTF-8. The `wchar_t` type is a “wide” character type, intended to be capable of representing any valid code point in a single integer. As such, its size is compiler- and system-specific. It could be eight bits on a system that does not support Unicode at all. It could be 16 bits if the UCS-2 encoding is assumed for all wide characters, or if a multi-word encoding like UTF-16 is being employed. Or it could be 32 bits if UTF-32 is the “wide” character encoding of choice.

Because of this inherent ambiguity in the definition of `wchar_t`, if you

need to write truly portable string-handling code, you'll need to define your own character data type(s) and provide a library of functions for dealing with whatever Unicode encoding(s) you need to support. However, if you are targeting a specific platform and compiler, you can write your code within the limits of that particular implementation, at the loss of some portability.

The following article does a good job of outlining the pros and cons of using the `wchar_t` data type: http://icu-project.org/docs/papers/unicode_wchar_t.html.

6.4.4.3 Unicode under Windows

Under Windows, the `wchar_t` data type is used exclusively for UTF-16 encoded Unicode strings, and the `char` type is used for ANSI strings and legacy *Windows code page* string encodings. When reading the Windows API docs, the term “Unicode” is therefore always synonymous with “wide character set” (WCS) and UTF-16 encoding. This is a bit confusing, because of course Unicode strings can in general be encoded in the “non-wide” multibyte UTF-8 format.

The Windows API defines three sets of character/string manipulation functions: one set for single-byte character set ANSI strings (SBCS), one set for multibyte character set (MBCS) strings, and one set for wide character set strings. The ANSI functions are essentially the old-school “C-style” string functions we all grew up with. The MBCS string functions handle a variety of multibyte encodings and are primarily designed for dealing with legacy Windows code pages encodings. The WCS functions handle Unicode UTF-16 strings.

Throughout the Windows API, a prefix or suffix of “w,” “wcs” or “W” indicates a wide character set (UTF-16) encoding; a prefix or suffix of “mb” indicates a multibyte encoding; and a prefix or suffix of “a” or “A,” or the lack of any prefix or suffix, indicates an ANSI or Windows code pages encoding. The C++ standard library uses a similar convention—for example, `std::string` is its ANSI string class, while `std::wstring` is its wide character equivalent. Unfortunately, the names of the functions aren't always 100% consistent. This all leads to some confusion among programmers who aren't in the know. (But *you* aren't one of those programmers!) Table 6.2 lists some examples.

Windows also provides functions for translating between ANSI character strings, multibyte strings and wide UTF-16 strings. For example, `wcstombs()` converts a wide UTF-16 string into a multibyte string according to the currently active *locale* setting.

The Windows API uses a little preprocessor trick to allow you to write code that is at least superficially portable between wide (Unicode) and non-wide

ANSI	WCS	MBCS
<code>strcmp()</code>	<code>wcsncmp()</code>	<code>_mbstrcmp()</code>
<code>strcpy()</code>	<code>wcsncpy()</code>	<code>_mbstrcpy()</code>
<code>strlen()</code>	<code>wcslen()</code>	<code>_mbstrlen()</code>

Table 6.2. Variants of some common C standard library string functions for use with ANSI, wide and multibyte character sets.

(ANSI/MBCS) string encodings. The generic character data type `TCHAR` is defined to be a typedef to `char` when building your application in “ANSI mode,” and it’s defined to be a typedef to `wchar_t` when building your application in “Unicode mode.” The macro `_T()` is used to convert an eight-bit string literal (e.g., `char* s = "this is a string";`) into a wide string literal (e.g., `wchar_t* s = L"this is a string";`) when compiling in “Unicode mode.” Likewise, a suite of “fake” API functions are provided that “automagically” morph into their appropriate 8-bit or 16-bit variant, depending on whether you are building in “Unicode mode” or not. These magic character-set-independent functions are either named with no prefix or suffix, or with a “t,” “tcs” or “T” prefix or suffix.

Complete documentation for all of these functions can be found on Microsoft’s MSDN website. Here’s a link to the documentation for `strcmp()` and its ilk, from which you can quite easily navigate to the other related string-manipulation functions using the tree view on the left-hand side of the page, or via the search bar: [http://msdn2.microsoft.com/en-us/library/kk6xf663\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/kk6xf663(VS.80).aspx).

6.4.4.4 Unicode on Consoles

The Xbox 360 software development kit (XDK) uses WCS strings pretty much exclusively, for all strings—even for internal strings like file paths. This is certainly one valid approach to the localization problem, and it makes for very consistent string handling throughout the XDK. However, the UTF-16 encoding is a bit wasteful on memory, so different game engines may employ different conventions. At Naughty Dog, we use eight-bit `char` strings throughout our engine, and we handle foreign languages via a UTF-8 encoding. The choice of encoding is not particularly important, as long as you select one as early in the project as possible and stick with it consistently.

6.4.4.5 Other Localization Concerns

Even once you have adapted your software to use Unicode characters, there is still a host of other localization problems to contend with. For one thing,

Id	English	French
p1score	"Player 1 Score"	"Joueur 1 Score"
p2score	"Player 2 Score"	"Joueur 2 Score"
p1wins	"Player one wins!"	"Joueur un gagne!"
p2wins	"Player two wins!"	"Joueur deux gagne!"

Table 6.3. Example of a string database used for localization.

strings aren't the only place where localization issues arise. Audio clips including recorded voices must be translated. Textures may have English words painted into them that require translation. Many symbols have different meanings in different cultures. Even something as innocuous as a no-smoking sign might be misinterpreted in another culture. In addition, some markets draw the boundaries between the various game-rating levels differently. For example, in Japan a Teen-rated game is not permitted to show blood of any kind, whereas in North America small red blood spatters are considered acceptable.

For strings, there are other details to worry about as well. You will need to manage a database of all human-readable strings in your game, so that they can all be reliably translated. The software must display the proper language given the user's installation settings. The formatting of the strings may be totally different in different languages—for example, Chinese is sometimes written vertically, and Hebrew reads right-to-left. The lengths of the strings will vary greatly from language to language. You'll also need to decide whether to ship a single DVD or Blu-ray disc that contains all languages or ship different discs for particular territories.

The most crucial components in your localization system will be the central database of human-readable strings and an in-game system for looking up those strings by id. For example, let's say you want a heads-up display that lists the score of each player with "Player 1 Score:" and "Player 2 Score:" labels and that also displays the text "Player 1 Wins" or "Player 2 Wins" at the end of a round. These four strings would be stored in the localization database under unique ids that are understandable to you, the developer of the game. So our database might use the ids "p1score," "p2score," "p1wins" and "p2wins," respectively. Once our game's strings have been translated into French, our database would look something like the simple example shown in Table 6.3. Additional columns can be added for each new language your game supports.

The exact format of this database is up to you. It might be as simple as a Microsoft Excel worksheet that can be saved as a comma-separated values (CSV) file and parsed by the game engine or as complex as a full-fledged Or-

acle database. The specifics of the string database are largely unimportant to the game engine, as long as it can read in the string ids and the corresponding Unicode strings for whatever language(s) your game supports. (However, the specifics of the database may be *very* important from a practical point of view, depending upon the organizational structure of your game studio. A small studio with in-house translators can probably get away with an Excel spreadsheet located on a network drive. But a large studio with branch offices in Britain, Europe, South America and Japan would probably find some kind of distributed database a great deal more amenable.)

At runtime, you'll need to provide a simple function that returns the Unicode string in the "current" language, given the unique id of that string. The function might be declared like this:

```
wchar_t getLocalizedString(const char* id);
```

and it might be used like this:

```
void drawScoreHud(const Vector3& score1Pos,
                  const Vector3& score2Pos)
{
    renderer.displayTextOrtho(getLocalizedString("p1score"),
                              score1Pos);

    renderer.displayTextOrtho(getLocalizedString("p2score"),
                              score2Pos);

    // ...
}
```

Of course, you'll need some way to set the "current" language globally. This might be done via a configuration setting, which is fixed during the installation of the game. Or you might allow users to change the current language on the fly via an in-game menu. Either way, the setting is not difficult to implement; it can be as simple as a global integer variable specifying the index of the column in the string table from which to read (e.g., column one might be English, column two French, column three Spanish and so on).

Once you have this infrastructure in place, your programmers must remember to *never display a raw string to the user*. They must always use the id of a string in the database and call the look-up function in order to retrieve the string in question.

6.4.4.6 Case Study: Naughty Dog's Localization Tool

At Naughty Dog, we use a localization database that we developed in-house. The localization tool's back end consists of a MySQL database located on a

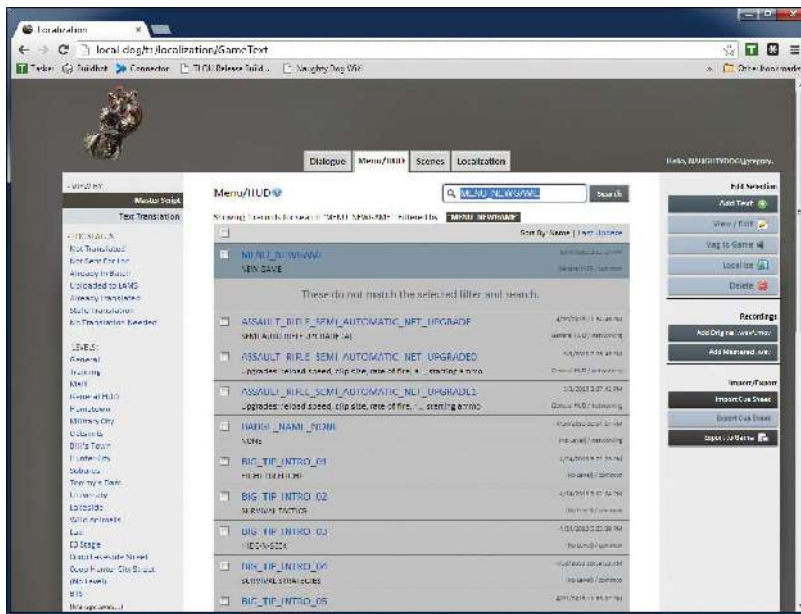


Figure 6.10. Naughty Dog's localization tool's main window, showing a list of pure text assets used in the menus and HUD. The user has just performed a search for an asset called MENU_NEWGAME.

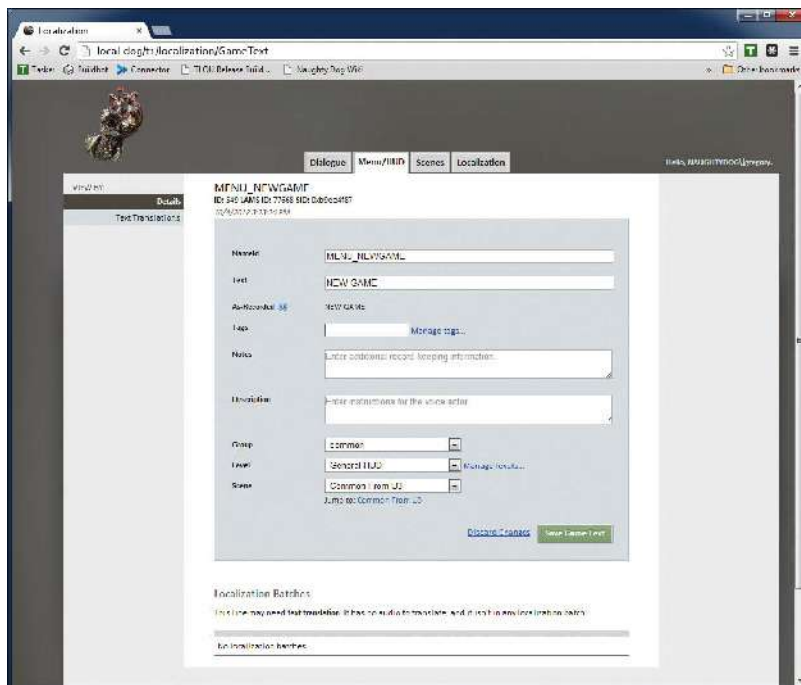


Figure 6.11. Detailed asset view, showing the MENU_NEWGAME string.

server that is accessible both to the developers within Naughty Dog and also to the various external companies with which we work to translate our text and speech audio clips into the various languages our games support. The front end is a web interface that “speaks” to the database, allowing users to view all of the text and audio assets, edit their contents, provide translations for each asset, search for assets by id or by content and so on.

In Naughty Dog’s localization tool, each asset is either a string (for use in the menus or HUD) or a speech audio clip with optional subtitle text (for use as in-game dialog or within cinematics). Each asset has a unique identifier, which is represented as a hashed string id (see Section 6.4.3.1). If a string is required for use in the menus or HUD, we look it up by its id and get back a Unicode (UTF-8) string suitable for display on-screen. If a line of dialog must be played, we likewise look up the audio clip by its id and use the data in-engine to look up its corresponding subtitle (if any). The subtitle is treated just like a menu or HUD string, in that it is returned by the localization tool’s API as a UTF-8 string suitable for display.

Figure 6.10 shows the main interface of the localization tool, in this case displayed in the Chrome web browser. In this image, you can see that the user has typed in the id `MENU_NEWGAME` in order to look up the string “NEW GAME” (used on the game’s main menu for launching a new game). Figure 6.11 shows the detailed view of the `MENU_NEWGAME` asset. If the user hits the “Text Translations” button in the upper-left corner of the asset details window, the screen shown in Figure 6.12 comes up, allowing the user to enter or edit the various translations of the string. Figure 6.13 shows another tab on the localization tool’s main page, this time listing audio speech assets. Finally, Figure 6.14 depicts the detailed asset view for the speech asset `BADA_GAM_MIL_ESCAPE_OVERPASS_001` (“We missed all the action”), showing translations of this line of dialog into some of the supported languages.

6.5 Engine Configuration

Game engines are complex beasts, and they invariably end up having a large number of configurable options. Some of these options are exposed to the player via one or more options menus in-game. For example, a game might expose options related to graphics quality, the volume of music and sound effects, or controller configuration. Other options are created for the benefit of the game development team only and are either hidden or stripped out of the game completely before it ships. For example, the player character’s

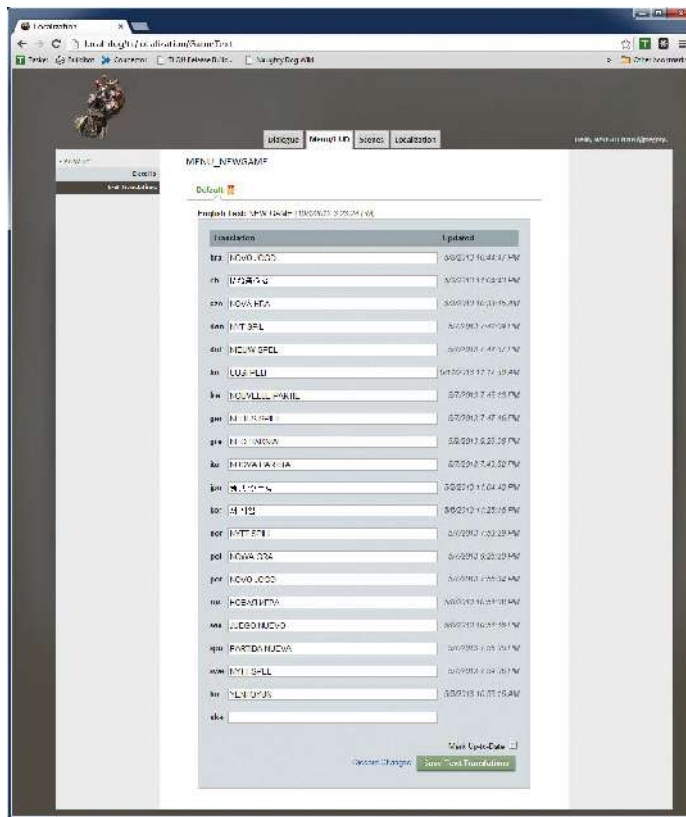


Figure 6.12. Text translations of the string “NEW GAME” into all languages supported by Naughty Dog’s *The Last of Us*.

maximum walk speed might be exposed as an option so that it can be fine-tuned during development, but it might be changed to a hard-coded value prior to ship.

6.5.1 Loading and Saving Options

A configurable option can be implemented trivially as a global variable or a member variable of a singleton class. However, configurable options are not particularly useful unless their values can be configured, stored on a hard disk, memory card or other storage medium, and later retrieved by the game. There are a number of simple ways to load and save configuration options:

- *Text configuration files.* By far the most common method of saving and loading configuration options is by placing them into one or more text

files. The format of these files varies widely from engine to engine, but it is usually very simple. For example, Windows INI files (which are used by the OGRE renderer) consist of flat lists of key-value pairs grouped into logical sections. The JSON format is another common choice for configurable game options files. XML is another viable option, although most developers these days find JSON to be less verbose and easier to read than XML.

- *Compressed binary files.* Most modern consoles have hard disk drives in them, but older consoles could not afford this luxury. As a result, all game consoles since the Super Nintendo Entertainment System (SNES) have come equipped with proprietary removable memory cards that permit both reading and writing of data. Game options are sometimes stored on these cards, along with saved games. Compressed binary files are the format of choice on a memory card, because the storage space available on these cards is often very limited.
- *The Windows registry.* The Microsoft Windows operating system provides a global options database known as the registry. It is stored as a tree, where the interior nodes (known as *registry keys*) act like file folders, and the leaf nodes store the individual options as key-value pairs. That being said, I don't recommend using the Windows registry for storing engine configuration information. The registry is a monolithic database that can easily be corrupted, lost (when Windows is reinstalled), or thrown out-of-sync with the files in the filesystem. For more on the weaknesses of the Windows registry, see <https://blog.codinghorror.com/was-the-windows-registry-a-good-idea/>.
- *Command line options.* The command line can be scanned for option settings. The engine might provide a mechanism for controlling any option in the game via the command line, or it might expose only a small subset of the game's options here.
- *Environment variables.* On personal computers running Windows, Linux or MacOS, environment variables are sometimes used to store configuration options as well.
- *Online user profiles.* With the advent of online gaming communities like Xbox Live, each user can create a profile and use it to save achievements, purchased and unlockable game features, game options and other information. The data are stored on a central server and can be accessed by the player wherever an Internet connection is available.

6.5.2 Per-User Options

Most game engines differentiate between global options and per-user options. This is necessary because most games allow each player to configure the game to his or her liking. It is also a useful concept during development of the game, because it allows each programmer, artist and designer to customize his or her work environment without affecting other team members.

Obviously care must be taken to store per-user options in such a way that each player “sees” only his or her options and not the options of other players on the same computer or console. In a console game, the user is typically allowed to save his or her progress, along with per-user options such as controller preferences, in “slots” on a memory card or hard disk. These slots are usually implemented as files on the media in question.

On a Windows machine, each user has a folder under `C:\Users` containing information such as the user’s desktop, his or her My Documents folder, his or her Internet browsing history and temporary files and so on. A hidden subfolder named *AppData* is used to store per-user information on a per-application basis; each application creates a folder under *AppData* and can use it to store whatever per-user information it requires.

Windows games sometimes store per-user configuration data in the registry. The registry is arranged as a tree, and one of the top-level children of the root node, called `HKEY_CURRENT_USER`, stores settings for whichever user happens to be logged on. Every user has his or her own subtree in the registry (stored under the top-level subtree `HKEY_USERS`), and `HKEY_CURRENT_USER` is really just an alias to the current user’s subtree. So games and other applications can manage per-user configuration options by simply reading and writing them to keys under the `HKEY_CURRENT_USER` subtree.

6.5.3 Configuration Management in Some Real Engines

In this section, we’ll take a brief look at how some real game engines manage their configuration options.

6.5.3.1 Example: Quake’s Cvars

The Quake family of engines uses a configuration management system known as *console variables*, or *cvars* for short. A cvar is just a floating-point or string global variable whose value can be inspected and modified from within Quake’s in-game console. The values of some cvars can be saved to disk and later reloaded by the engine.

At runtime, cvars are stored in a global linked list. Each cvar is a dynamically allocated instance of `struct cvar_t`, which contains the variable’s

name, its value as a string or float, a set of flag bits, and a pointer to the next cvar in the linked list of all cvars. Cvars are accessed by calling `Cvar_Get()`, which creates the variable if it doesn't already exist and modified by calling `Cvar_Set()`. One of the bit flags, `CVAR_ARCHIVE`, controls whether or not the cvar will be saved into a configuration file called *config.cfg*. If this flag is set, the value of the cvar will persist across multiple runs of the game.

6.5.3.2 Example: OGRE

The OGRE rendering engine uses a collection of text files in Windows INI format for its configuration options. By default, the options are stored in three files, each of which is located in the same folder as the executable program:

- *plugins.cfg* contains options specifying which optional engine plug-ins are enabled and where to find them on disk.
- *resources.cfg* contains a *search path* specifying where game assets (a.k.a. media, a.k.a. resources) can be found.
- *ogre.cfg* contains a rich set of options specifying which renderer (DirectX or OpenGL) to use and the preferred video mode, screen size, etc.

Out of the box, OGRE provides no mechanism for storing per-user configuration options. However, the OGRE source code is freely available, so it would be quite easy to change it to search for its configuration files in the user's home directory instead of in the folder containing the executable. The `Ogre::ConfigFile` class makes it easy to write code that reads and writes brand new configuration files as well.

6.5.3.3 Example: The Uncharted and The Last of Us Series

Naughty Dog's engine makes use of a number of configuration mechanisms.

In-Game Menu Settings

The Naughty Dog engine supports a powerful in-game menu system, allowing developers to control global configuration options and invoke commands. The data types of the configurable options must be relatively simple (primarily Boolean, integer and floating-point variables), but this limitation has not prevented the developers at Naughty Dog from creating literally hundreds of useful menu-driven options.

Each configuration option is implemented as a global variable, or a member of a singleton struct or class. When the menu option that controls an option is created, the address of the variable is provided, and the menu item directly controls its value. As an example, the following function creates a

submenu item containing some options for Naughty Dog's *rail vehicles* (simple vehicles that ride on splines which have been used in pretty much every Naughty Dog game, from the "Out of the Frying Pan" jeep chase level in *Uncharted: Drake's Fortune* to the truck convoy / jeep chase sequence in *Uncharted 4*). It defines menu items controlling three global variables: two Booleans and one floating-point value. The items are collected onto a menu, and a special item is returned that will bring up the menu when selected. Presumably the code calling this function adds this item to the parent menu that it is building.

```
DMENU::ItemSubmenu * CreateRailVehicleMenu()
{
    extern bool g_railVehicleDebugDraw2D;
    extern bool g_railVehicleDebugDrawCameraGoals;
    extern float g_railVehicleFlameProbability;

    DMENU::Menu * pMenu
        = new DMENU::Menu("RailVehicle");

    pMenu->PushBackItem(
        new DMENU::ItemBool("Draw 2D Spring Graphs",
            DMENU::ToggleBool,
            &g_railVehicleDebugDraw2D));

    pMenu->PushBackItem(
        new DMENU::ItemBool("Draw Goals (Untracked)",
            DMENU::ToggleBool,
            &g_railVehicleDebugDrawCameraGoals));

    DMENU::ItemFloat * pItemFloat;
    pItemFloat = new DMENU::ItemFloat(
        "FlameProbability",
        DMENU::EditFloat, 5, "%5.2f",
        &g_railVehicleFlameProbability);

    pItemFloat->SetRangeAndStep(0.0f, 1.0f, 0.1f, 0.01f);
    pMenu->PushBackItem(pItemFloat);

    DMENU::ItemSubmenu * pSubmenuItem;
    pSubmenuItem = new DMENU::ItemSubmenu(
        "RailVehicle...", pMenu);

    return pSubmenuItem;
}
```

The value of any option can be saved by simply marking it with the circle button on the Dualshock joypad when the corresponding menu item is se-

lected. The menu settings are saved in an INI-style text file, allowing the saved global variables to retain the values across multiple runs of the game. The ability to control which options are saved on a *per-menu-item* basis is highly useful, because any option that is not saved will take on its programmer-specified default value. If a programmer changes a default, all users will “see” the new value, unless of course a user has saved a custom value for that particular option.

Command Line Arguments

The Naughty Dog engine scans the command line for a predefined set of special options. The name of the level to load can be specified, along with a number of other commonly used arguments.

Scheme Data Definitions

The vast majority of engine and game configuration information in the Naughty Dog engine (used to produce the *Uncharted* and *The Last of Us* series) is specified using a Lisp-like language called Scheme. Using a proprietary data compiler, data structures defined in the Scheme language are transformed into binary files that can be loaded by the engine. The data compiler also spits out header files containing C `struct` declarations for every data type defined in Scheme. These header files allow the engine to properly interpret the data contained in the loaded binary files. The binary files can even be recompiled and reloaded on the fly, allowing developers to alter the data in Scheme and see the effects of their changes immediately (as long as data members are not added or removed, as that would require a recompile of the engine).

The following example illustrates the creation of a data structure specifying the properties of an animation. It then exports three unique animations to the game. You may have never read Scheme code before, but for this relatively simple example it should be pretty self-explanatory. One oddity you’ll notice is that hyphens are permitted within Scheme symbols, so `simple-animation` is a single symbol (unlike in C/C++ where `simple-animation` would be the subtraction of two variables, `simple` and `animation`).

simple-animation.scm

```
;; Define a new data type called simple-animation.
(deftype simple-animation ()
  (
    (name          string)
    (speed         float   :default 1.0)
```

```

        (fade-in-seconds float :default 0.25)
        (fade-out-seconds float :default 0.25)
    )
)

;; Now define three instances of this data structure...
(define-export anim-walk
  (new simple-animation
    :name "walk"
    :speed 1.0
  )
)

(define-export anim-walk-fast
  (new simple-animation
    :name "walk"
    :speed 2.0
  )
)

(define-export anim-jump
  (new simple-animation
    :name "jump"
    :fade-in-seconds 0.1
    :fade-out-seconds 0.1
  )
)

```

This Scheme code would generate the following C/C++ header file:

simple-animation.h

```

// WARNING: This file was automatically generated from
// Scheme. Do not hand-edit.

struct SimpleAnimation
{
    const char* m_name;
    float      m_speed;
    float      m_fadeInSeconds;
    float      m_fadeOutSeconds;
};

```

In-game, the data can be read by calling the `LookupSymbol()` function, which is templated on the data type returned, as follows:

```
#include "simple-animation.h"
void someFunction()
{
    SimpleAnimation* pWalkAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-walk"));

    SimpleAnimation* pFastWalkAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-walk-fast"));

    SimpleAnimation* pJumpAnim
        = LookupSymbol<SimpleAnimation*>(
            SID("anim-jump"));

    // use the data here...
}
```

This system gives the programmers a great deal of flexibility in defining all sorts of configuration data—from simple Boolean, floating-point and string options all the way to complex, nested, interconnected data structures. It is used to specify detailed animation trees, physics parameters, player mechanics and so on.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

7

Resources and the File System

Games are by nature multimedia experiences. A game engine therefore needs to be capable of loading and managing a wide variety of different kinds of media—texture bitmaps, 3D mesh data, animations, audio clips, collision and physics data, game world layouts, and the list goes on. Moreover, because memory is usually scarce, a game engine needs to ensure that only one copy of each media file is loaded into memory at any given time. For example, if five meshes share the same texture, then we would like to have only one copy of that texture in memory, not five. Most game engines employ some kind of *resource manager* (a.k.a. *asset manager*, a.k.a. *media manager*) to load and manage the myriad resources that make up a modern 3D game.

Every resource manager makes heavy use of the file system. On a personal computer, the file system is exposed to the programmer via a library of operating system calls. However, game engines often “wrap” the native file system API in an engine-specific API, for two primary reasons. First, the engine might be cross-platform, in which case the game engine’s file system API can shield the rest of the software from differences between different target hardware platforms. Second, the operating system’s file system API might not provide all the tools needed by a game engine. For example, many engines support file *streaming* (i.e., the ability to load data “on the fly” while the game is running), yet most operating systems don’t provide a streaming file system API

out of the box. Console game engines also need to provide access to a variety of removable and non-removable media, from memory sticks to optional hard drives to a DVD-ROM or Blu-ray fixed disk to network file systems (e.g., Xbox Live or the PlayStation Network, PSN). The differences between various kinds of media can likewise be “hidden” behind a game engine’s file system API.

In this chapter, we’ll first explore the kinds of file system APIs found in modern 3D game engines. Then we’ll see how a typical resource manager works.

7.1 File System

A game engine’s file system API typically addresses the following areas of functionality:

- manipulating file names and paths,
- opening, closing, reading and writing individual files,
- scanning the contents of a directory, and
- handling asynchronous file I/O requests (for streaming).

We’ll take a brief look at each of these in the following sections.

7.1.1 File Names and Paths

A *path* is a string describing the location of a file or directory within a file system hierarchy. Each operating system uses a slightly different path format, but paths have essentially the same structure on every operating system. A path generally takes the following form:

```
volume/directory1/directory2/.../directoryN/file-name
```

or

```
volume/directory1/directory2/.../directory(N - 1)/directoryN
```

In other words, a path generally consists of an optional *volume specifier* followed by a sequence of *path components* separated by a reserved path separator character such as the forward or backward slash (/ or \). Each component names a directory along the route from the root directory to the file or directory in question. If the path specifies the location of a file, the last component in the path is the file name; otherwise it names the target directory. The root directory is usually indicated by a path consisting of the optional volume specifier followed by a single path separator character (e.g., / on UNIX, or C : \ on Windows).

7.1.1.1 Differences across Operating Systems

Each operating system introduces slight variations on this general path structure. Here are some of the key differences between Microsoft DOS, Microsoft Windows, the UNIX family of operating systems and Apple Macintosh OS:

- UNIX uses a forward slash (/) as its path component separator, while DOS and older versions of Windows used a backslash (\) as the path separator. Recent versions of Windows allow either forward or backward slashes to be used to separate path components, although some applications still fail to accept forward slashes.
- Mac OS 8 and 9 use the colon (:) as the path separator character. Mac OS X is based on BSD UNIX, so it supports UNIX's forward slash notation.
- Some filesystems consider paths and filenames to be case-sensitive (like UNIX and its variants), while others are case-insensitive (like Windows). This can cause problems when dealing with files across multiple operating systems during development, or when writing a cross-platform game. (For example, should an asset file named `EnemyAnims.json` be considered equivalent to an asset file named `enemyanim.json` or not?)
- UNIX and its variants don't support volumes as separate directory hierarchies. The entire file system is contained within a single monolithic hierarchy, and local disk drives, network drives and other resources are *mounted* so that they appear to be subtrees within the main hierarchy. As a result, UNIX paths never have a volume specifier.
- On Microsoft Windows, volumes can be specified in two ways. A local disk drive is specified using a single letter followed by a colon (e.g., the ubiquitous `C:`). A remote network share can either be mounted so that it looks like a local disk, or it can be referenced via a volume specifier consisting of two backslashes followed by the remote computer name and the name of a shared directory or resource on that machine (e.g., `\\some-computer\some-share`). This double backslash notation is an example of the Universal Naming Convention (UNC).
- Under DOS and early versions of Windows, a file name could be up to eight characters in length, with a three-character *extension* which was separated from the main file name by a dot. The extension described the file's type, for example `.txt` for a text file or `.exe` for an executable file. In recent Windows implementations, file names can contain any number of dots (as they can under UNIX), but the characters after the

final dot are still interpreted as the file's extension by many applications including the Windows Explorer.

- Each operating system disallows certain characters in the names of files and directories. For example, a colon cannot appear anywhere in a Windows or DOS path except as part of a drive letter volume specifier. Some operating systems permit a subset of these reserved characters to appear in a path as long as the path is quoted in its entirety or the offending character is *escaped* by preceding it with a backslash or some other reserved *escape character*. For example, file and directory names may contain spaces under Windows, but such a path must be surrounded by double quotes in certain contexts.
- Both UNIX and Windows have the concept of a *current working directory* or CWD (also known as the *present working directory* or PWD). The CWD can be set from a command shell via the `cd` (*change directory*) command on both operating systems, and it can be queried by typing `cd` with no arguments under Windows or by executing the `pwd` command on UNIX. Under UNIX there is only one CWD. Under Windows, each volume has its own private CWD.
- Operating systems that support multiple volumes, like Windows, also have the concept of a *current working volume*. From a Windows command shell, the current volume can be set by entering its drive letter and a colon followed by the Enter key (e.g., `C : <Enter>`).
- Consoles often also employ a set of predefined path prefixes to represent multiple volumes. For example, PlayStation 3 uses the prefix `/dev_bdvd/` to refer to the Blu-ray disk drive, while `/dev_hddx/` refers to one or more hard disks (where *x* is the index of the device). On a PS3 development kit, `/app_home/` maps to a user-defined path on whatever host machine is being used for development. During development, the game usually reads its assets from `/app_home/` rather than from the Blu-ray or the hard disk.

7.1.1.2 Absolute and Relative Paths

All paths are specified relative to some location within the file system. When a path is specified relative to the root directory, we call it an *absolute path*. When it is relative to some *other* directory in the file system hierarchy, we call it a *relative path*.

Under both UNIX and Windows, absolute paths start with a path separator character (`/` or `\`), while relative paths have no leading path separator. On Windows, both absolute and relative paths may have an optional volume

specifier—if the volume is omitted, then the path is assumed to refer to the current working volume.

The following paths are all absolute:

Windows

- C:\Windows\System32
- D:\ (root directory on the D: volume)
- \ (root directory on the current working volume)
- \game\assets\animation\walk.anim (current working volume)
- \\joe-dell\Shared_Files\Images\foo.jpg (network path)

UNIX

- /usr/local/bin/grep
- /game/src/audio/effects.cpp
- / (root directory)

The following paths are all relative:

Windows

- System32 (relative to CWD \Windows on the current volume)
- X:animation\walk.anim (relative to CWD \game\assets on the X: volume)

UNIX

- bin/grep (relative to CWD /usr/local)
- src/audio/effects.cpp (relative to CWD /game)

7.1.1.3 Search Paths

The term *path* should not be confused with the term *search path*. A *path* is a string representing the location of a single file or directory within the file system hierarchy. A *search path* is a string containing a list of paths, each separated by a special character such as a colon or semicolon, which is searched when looking for a file. For example, when you run any program from a command prompt, the operating system finds the executable file by searching each directory on the search path contained in the shell's environment variable.

Some game engines also use search paths to locate resource files. For example, the OGRE rendering engine uses a resource search path contained in a text file named `resources.cfg`. The file provides a simple list of directories and ZIP archives that should be searched in order when trying to find an asset. That said, searching for assets at runtime is a time-consuming proposition. Usually there's no reason our assets' paths cannot be known a priori. Presuming this is the case, we can avoid having to search for assets at all—which is clearly a superior approach.

7.1.1.4 Path APIs

Clearly, paths are much more complex than simple strings. There are many things a programmer may need to do when dealing with paths, such as isolating the directory, filename and extension, canonicalizing a path, converting back and forth between absolute and relative paths and so on. It can be extremely helpful to have a feature-rich API to help with these tasks.

Microsoft Windows provides an API for this purpose. It is implemented by the dynamic link library `shlwapi.dll`, and it is exposed via the header file `shlwapi.h`. Complete documentation for this API is provided on the Microsoft Developer's Network (MSDN) at the following URL: [http://msdn2.microsoft.com/en-us/library/bb773559\(VS.85\).aspx](http://msdn2.microsoft.com/en-us/library/bb773559(VS.85).aspx).

Of course, the `shlwapi` API is only available on Win32 platforms. Sony provides similar APIs for use on the PlayStation 3 and PlayStation 4. But when writing a cross-platform game engine, we cannot use platform-specific APIs directly. A game engine may not need all of the functions provided by an API like `shlwapi` anyway. For these reasons, game engines often implement a stripped-down path-handling API that meets the engine's particular needs and works on every operating system targeted by the engine. Such an API can be implemented as a thin wrapper around the native API on each platform or it can be written from scratch.

7.1.2 Basic File I/O

The C standard library provides two APIs for opening, reading and writing the contents of files—one buffered and the other unbuffered. Every file I/O API requires data blocks known as *buffers* to serve as the source or destination of the bytes passing between the program and the file on disk. We say a file I/O API is *buffered* when the API manages the necessary input and output data buffers for you. With an unbuffered API, it is the responsibility of the programmer using the API to allocate and manage the data buffers. The C standard library's buffered file I/O routines are sometimes referred to as the *stream I/O* API, because they provide an abstraction which makes disk files look like streams of bytes.

The C standard library functions for buffered and unbuffered file I/O are listed in Table 7.1.

The C standard library I/O functions are well-documented, so we will not repeat detailed documentation for them here. For more information, please refer to <http://msdn.microsoft.com/en-us/library/c565h7xx.aspx> for Microsoft's implementation of the buffered (stream I/O) API, and to <http://msdn.microsoft.com/en-us/library/40bbyw78.aspx> for Microsoft's imple-

Operation	Buffered API	Unbuffered API
Open a file	<code>fopen()</code>	<code>open()</code>
Close a file	<code>fclose()</code>	<code>close()</code>
Read from a file	<code>fread()</code>	<code>read()</code>
Write to a file	<code>fwrite()</code>	<code>write()</code>
Seek to an offset	<code>fseek()</code>	<code>seek()</code>
Return current offset	<code>ftell()</code>	<code>tell()</code>
Read a single line	<code>fgets()</code>	n/a
Write a single line	<code>fputs()</code>	n/a
Read formatted string	<code>fscanf()</code>	n/a
Write formatted string	<code>fprintf()</code>	n/a
Query file status	<code>fstat()</code>	<code>stat()</code>

Table 7.1. Buffered and unbuffered file operations in the C standard library.

mentation of the unbuffered (low-level I/O) API.

On UNIX and its variants, the C standard library's unbuffered I/O routes are native operating system calls. However, on Microsoft Windows these routines are merely wrappers around an even lower-level API. The Win32 function `CreateFile()` creates or opens a file for writing or reading, `ReadFile()` and `WriteFile()` read and write data, respectively, and `CloseFile()` closes an open file handle. The advantage to using low-level system calls as opposed to C standard library functions is that they expose all of the details of the native file system. For example, you can query and control the security attributes of files when using the Windows native API—something you cannot do with the C standard library.

Some game teams find it useful to manage their own buffers. For example, the *Red Alert 3* team at Electronic Arts observed that writing data into log files was causing significant performance degradation. They changed the logging system so that it accumulated its output into a memory buffer, writing the buffer out to disk only when it was filled. Then they moved the buffer dump routine out into a separate thread to avoid stalling the main game loop.

7.1.2.1 To Wrap or Not to Wrap

A game engine can be written to use the C standard library's file I/O functions or the operating system's native API. However, many game engines *wrap* the file I/O API in a library of custom I/O functions. There are at least three advantages to wrapping the operating system's I/O API. First, the engine programmers can guarantee identical behavior across all target platforms, even when native libraries are inconsistent or buggy on a particular platform.

Second, the API can be simplified down to only those functions actually required by the engine, which keeps maintenance efforts to a minimum. Third, extended functionality can be provided. For example, the engine's custom wrapper API might be capable of dealing with files on a hard disk, a DVD-ROM or Blu-ray disk on a console, files on a network (e.g., remote files managed by Xbox Live or PSN), and also with files on memory sticks or other kinds of removable media.

7.1.2.2 Synchronous File I/O

Both of the standard C library's file I/O libraries are *synchronous*, meaning that the program making the I/O request must wait until the data has been completely transferred to or from the media device before continuing. The following code snippet demonstrates how the entire contents of a file might be read into an in-memory buffer using the synchronous I/O function `fread()`. Notice how the function `syncReadFile()` does not return until all the data has been read into the buffer provided.

```
bool syncReadFile(const char* filePath,
                  U8* buffer,
                  size_t bufferSize,
                  size_t& rBytesRead)
{
    FILE* handle = fopen(filePath, "rb");
    if (handle)
    {
        // BLOCK here until all data has been read.
        size_t bytesRead = fread(buffer, 1,
                                bufferSize, handle);

        int err = ferror(handle); // get error if any

        fclose(handle);

        if (0 == err)
        {
            rBytesRead = bytesRead;
            return true;
        }
    }
    rBytesRead = 0;
    return false;
}

void main(int argc, const char* argv[])
{

```

```

    U8 testBuffer[512];
    size_t bytesRead = 0;

    if (syncReadFile("C:\\testfile.bin",
                    testBuffer, sizeof(testBuffer),
                    bytesRead))
    {
        printf("success: read %u bytes\n", bytesRead);
        // contents of buffer can be used here...
    }
}

```

7.1.3 Asynchronous File I/O

Streaming refers to the act of loading data in the background while the main program continues to run. Many games provide the player with a seamless, load-screen-free playing experience by streaming data for upcoming levels from the DVD-ROM, Blu-ray disk or hard drive while the game is being played. Audio and texture data are probably the most commonly streamed types of data, but any type of data can be streamed, including geometry, level layouts and animation clips.

In order to support streaming, we must utilize an *asynchronous* file I/O library, i.e., one which permits the program to continue to run while its I/O requests are being satisfied. Some operating systems provide an asynchronous file I/O library out of the box. For example, the Windows Common Language Runtime (CLR, the virtual machine upon which languages like Visual BASIC, C#, managed C++ and J# are implemented) provides functions like `System.IO.BeginRead()` and `System.IO.BeginWrite()`. An asynchronous API known as `fios` is available for the PlayStation 3 and PlayStation 4. If an asynchronous file I/O library is not available for your target platform, it is possible to write one yourself. And even if you don't have to write it from scratch, it's probably a good idea to wrap the system API for portability.

The following code snippet demonstrates how the entire contents of a file might be read into an in-memory buffer using an asynchronous read operation. Notice that the `asyncReadFile()` function returns immediately—the data is not present in the buffer until our callback function `asyncReadComplete()` has been called by the I/O library.

```

AsyncRequestHandle g_hRequest; // async I/O request handle
U8 g_asyncBuffer[512];        // input buffer

static void asyncReadComplete(AsyncRequestHandle hRequest);

```

```

void main(int argc, const char* argv[])
{
    // NOTE: This call to asyncOpen() might itself be an
    // asynchronous call, but we'll ignore that detail
    // here and just assume it's a blocking function.

    AsyncFileHandle hFile = asyncOpen(
        "C:\\testfile.bin");

    if (hFile)
    {
        // This function requests an I/O read, then
        // returns immediately (non-blocking).
        g_hRequest = asyncReadFile(
            hFile,                // file handle
            g_asyncBuffer,        // input buffer
            sizeof(g_asyncBuffer), // size of buffer
            asyncReadComplete);    // callback function
    }

    // Now go on our merry way...
    // (This loop simulates doing real work while we wait
    // for the I/O read to complete.)

    for (;;)
    {
        OutputDebugString("zzz...\n");
        Sleep(50);
    }
}

// This function will be called when the data has been read.
static void asyncReadComplete(AsyncRequestHandle hRequest)
{
    if (hRequest == g_hRequest
        && asyncWasSuccessful(hRequest))
    {
        // The data is now present in g_asyncBuffer[] and
        // can be used. Query for the number of bytes
        // actually read:
        size_t bytes = asyncGetBytesReadOrWritten(
            hRequest);

        char msg[256];
        snprintf(msg, sizeof(msg),
            "async success, read %u bytes\n",

```



```

        bytes);
    OutputDebugString(msg);
}
}

```

Most asynchronous I/O libraries permit the main program to wait for an I/O operation to complete some time after the request was made. This can be useful in situations where only a limited amount of work can be done before the results of a pending I/O request are needed. This is illustrated in the following code snippet.

```

U8 g_asyncBuffer[512]; // input buffer

void main(int argc, const char* argv[])
{
    AsyncRequestHandle hRequest = ASYNC_INVALID_HANDLE;
    AsyncFileHandle hFile = asyncOpen(
        "C:\\testfile.bin");

    if (hFile)
    {
        // This function requests an I/O read, then
        // returns immediately (non-blocking).
        hRequest = asyncReadFile(
            hFile,           // file handle
            g_asyncBuffer,   // input buffer
            sizeof(g_asyncBuffer), // size of buffer
            nullptr);        // no callback
    }

    // Now do some limited amount of work...
    for (int i = 0; i < 10; i++)
    {
        OutputDebugString("zzz...\n");
        Sleep(50);
    }

    // We can't do anything further until we have that
    // data, so wait for it here.
    asyncWait(hRequest);

    if (asyncWasSuccessful(hRequest))
    {
        // The data is now present in g_asyncBuffer[] and
        // can be used. Query for the number of bytes
        // actually read:
        size_t bytes = asyncGetBytesReadOrWritten(
            hRequest);
    }
}

```

```

        char msg[256];
        snprintf(msg, sizeof(msg),
            "async success, read %u bytes\n",
            bytes);
        OutputDebugString(msg);
    }
}

```

Some asynchronous I/O libraries allow the programmer to ask for an estimate of how long a particular asynchronous operation will take to complete. Some APIs also allow you to set *deadlines* on a request (which effectively prioritizes the request relative to other pending requests), and to specify what happens when a request misses its deadline (e.g., cancel the request, notify the program and keep trying, etc.)

7.1.3.1 Priorities

It's important to remember that file I/O is a real-time system, subject to deadlines just like the rest of the game. Therefore, asynchronous I/O operations often have varying priorities. For example, if we are streaming audio from the hard disk or Blu-ray and playing it on the fly, loading the next buffer-full of audio data is clearly higher priority than, say, loading a texture or a chunk of a game level. Asynchronous I/O systems must be capable of suspending lower-priority requests, so that higher-priority I/O requests have a chance to complete within their deadlines.

7.1.3.2 How Asynchronous File I/O Works

Asynchronous file I/O works by handling I/O requests in a separate thread. The main thread calls functions that simply place requests on a queue and then return immediately. Meanwhile, the I/O thread picks up requests from the queue and handles them sequentially using blocking I/O routines like `read()` or `fread()`. When a request is completed, a callback provided by the main thread is called, thereby notifying it that the operation is done. If the main thread chooses to wait for an I/O request to complete, this is handled via a *semaphore*. (Each request has an associated semaphore, and the main thread can put itself to sleep waiting for that semaphore to be signaled by the I/O thread upon completion of the request. See Section 4.6.4 for more on semaphores.)

Virtually *any* synchronous operation you can imagine can be transformed into an asynchronous operation by moving the code into a separate thread—or by running it on a physically separate processor, such as on one of the CPU cores on the PlayStation 4. See Section 8.6 for more details.

7.2 The Resource Manager

Every game is constructed from a wide variety of *resources* (sometimes called *assets* or *media*). Examples include meshes, materials, textures, shader programs, animations, audio clips, level layouts, collision primitives, physics parameters, and the list goes on. A game's resources must be managed, both in terms of the offline tools used to create them, and in terms of loading, unloading and manipulating them at runtime. Therefore, every game engine has a *resource manager* of some kind.

Every resource manager is comprised of two distinct but integrated components. One component manages the chain of offline tools used to create the assets and transform them into their engine-ready form. The other component manages the resources at runtime, ensuring that they are loaded into memory in advance of being needed by the game and making sure they are unloaded from memory when no longer needed.

In some engines, the resource manager is a cleanly designed, unified, centralized subsystem that manages all types of resources used by the game. In other engines, the resource manager doesn't exist as a single subsystem per se, but rather is spread across a disparate collection of subsystems, perhaps written by different individuals at various times over the engine's long and sometimes colorful history. But no matter how it is implemented, a resource manager invariably takes on certain responsibilities and solves a well-understood set of problems. In this section, we'll explore the functionality and some of the implementation details of a typical game engine resource manager.

7.2.1 Offline Resource Management and the Tool Chain

7.2.1.1 Revision Control for Assets

On a small game project, the game's assets can be managed by keeping loose files sitting around on a shared network drive with an ad hoc directory structure. This approach is not feasible for a modern commercial 3D game, comprised of a massive number and variety of assets. For such a project, the team requires a more formalized way to track and manage its assets.

Some game teams use a source code revision control system to manage their resources. Art source files (Maya scenes, Photoshop PSD files, Illustrator files, etc.) are checked in to Perforce or a similar package by the artists. This approach works reasonably well, although some game teams build custom asset management tools to help flatten the learning curve for their artists. Such tools may be simple wrappers around a commercial revision control system, or they might be entirely custom.

Dealing with Data Size

One of the biggest problems in the revision control of art assets is the sheer amount of data. Whereas C++ and script source code files are small, relative to their impact on the project, art files tend to be much, much larger. Because many source control systems work by copying files from the central repository down to the user's local machine, the sheer size of the asset files can render these packages almost entirely useless.

I've seen a number of different solutions to this problem employed at various studios. Some studios turn to commercial revision control systems like Alienbrain that have been specifically designed to handle very large data sizes. Some teams simply "take their lumps" and allow their revision control tool to copy assets locally. This can work, as long as your disks are big enough and your network bandwidth sufficient, but it can also be inefficient and slow the team down. Some teams build elaborate systems on top of their revision control tool to ensure that a particular end user only gets local copies of the files he or she actually needs. In this model, the user either has no access to the rest of the repository or can access it on a shared network drive when needed.

At Naughty Dog we use a proprietary tool that makes use of UNIX symbolic links to virtually eliminate data copying, while permitting each user to have a complete local view of the asset repository. As long as a file is not checked out for editing, it is a symlink to a master file on a shared network drive. A symbolic link occupies very little space on the local disk, because it is nothing more than a directory entry. When the user checks out a file for editing, the symlink is removed, and a local copy of the file replaces it. When the user is done editing and checks the file in, the local copy becomes the new master copy, its revision history is updated in a master database, and the local file turns back into a symlink. This system works very well, but it requires the team to build their own revision control system from scratch; I am unaware of any commercial tool that works like this. Also, symbolic links are a UNIX feature—such a tool could probably be built with Windows junctions (the Windows equivalent of a symbolic link), but I haven't seen anyone try it as yet.

7.2.1.2 The Resource Database

As we'll explore in depth in the next section, most assets are not used in their original format by the game engine. They need to pass through some kind of asset conditioning pipeline, whose job it is to convert the assets into the binary format needed by the engine. For every resource that passes through

the asset conditioning pipeline, there is some amount of *metadata* that describes *how* that resource should be processed. When compressing a texture bitmap, we need to know what *type* of compression best suits that particular image. When exporting an animation, we need to know what range of frames in Maya should be exported. When exporting character meshes out of a Maya scene containing multiple characters, we need to know which mesh corresponds to which character in the game.

To manage all of this metadata, we need some kind of database. If we are making a very small game, this database might be housed in the brains of the developers themselves. I can hear them now: “Remember: the player’s animations need to have the ‘flip X’ flag set, but the other characters must *not* have it set...or...rats...is it the other way around?”

Clearly for any game of respectable size, we simply cannot rely on the memories of our developers in this manner. For one thing, the sheer volume of assets becomes overwhelming quite quickly. Processing individual resource files by hand is also far too time-consuming to be practical on a full-fledged commercial game production. Therefore, every professional game team has some kind of semiautomated resource pipeline, and the data that drive the pipeline is stored in some kind of *resource database*.

The resource database takes on vastly different forms in different game engines. In one engine, the metadata describing how a resource should be built might be embedded into the source assets themselves (e.g., it might be stored as so-called blind data within a Maya file). In another engine, each source resource file might be accompanied by a small text file that describes how it should be processed. Still other engines encode their resource building metadata in a set of XML files, perhaps wrapped in some kind of custom graphical user interface. Some engines employ a true relational database, such as Microsoft Access, MySQL or conceivably even a heavyweight database like Oracle.

Whatever its form, a resource database must provide the following basic functionality:

- The ability to deal with multiple *types* of resources, ideally (but certainly not necessarily) in a somewhat consistent manner.
- The ability to create new resources.
- The ability to delete resources.
- The ability to inspect and modify existing resources.
- The ability to move a resource’s source file(s) from one location to another on-disk. (This is very helpful because artists and game designers

often need to rearrange assets to reflect changing project goals, rethinking of game designs, feature additions and cuts, etc.)

- The ability of a resource to cross-reference other resources (e.g., the material used by a mesh, or the collection of animations needed by level 17). These cross-references typically drive both the resource building process and the loading process at runtime.
- The ability to maintain *referential integrity* of all cross-references within the database and to do so in the face of all common operations such as deleting or moving resources around.
- The ability to maintain a revision history, complete with a log of who made each change and why.
- It is also very helpful if the resource database supports searching or querying in various ways. For example, a developer might want to know in which levels a particular animation is used or which textures are referenced by a set of materials. Or they might simply be trying to find a resource whose name momentarily escapes them.

It should be pretty obvious from looking at the above list that creating a reliable and robust resource database is no small task. When designed well and implemented properly, the resource database can quite literally make the difference between a team that ships a hit game and a team that spins its wheels for 18 months before being forced by management to abandon the project (or worse). I know this to be true, because I've personally experienced both.

7.2.1.3 Some Successful Resource Database Designs

Every game team will have different requirements and make different decisions when designing their resource database. However, for what it's worth, here are some designs that have worked well in my own experience.

Unreal Engine 4

Unreal's resource database is managed by their über-tool, UnrealEd. UnrealEd is responsible for literally everything, from resource metadata management to asset creation to level layout and more. UnrealEd has its drawbacks, but its single biggest benefit is that UnrealEd is a part of the game engine itself. This permits assets to be created and then immediately viewed in their full glory, exactly as they will appear in-game. The game can even be run from within UnrealEd in order to visualize the assets in their natural surroundings and see if and how they work in-game.

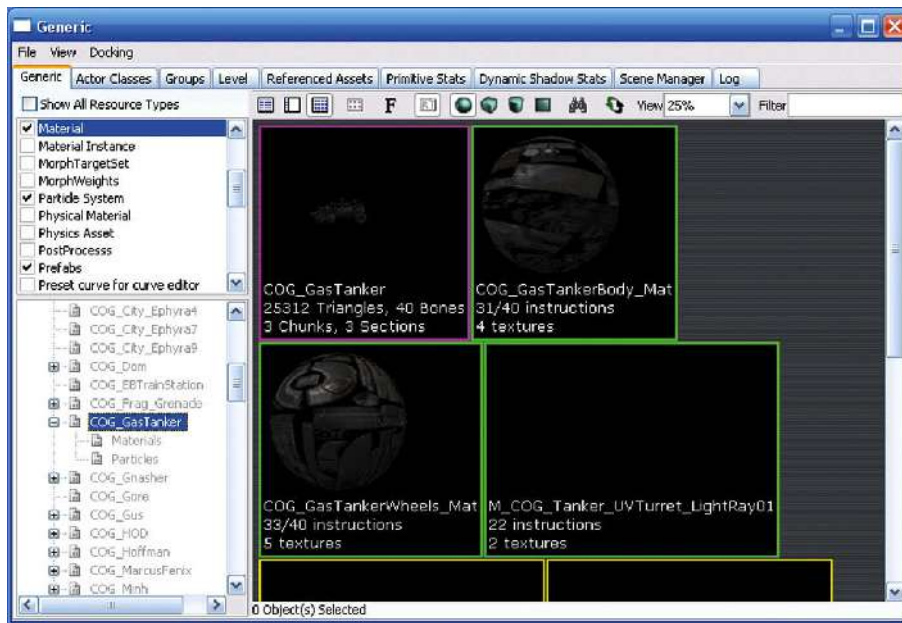


Figure 7.1. UnrealEd's Generic Browser.

Another big benefit of UnrealEd is what I would call *one-stop shopping*. UnrealEd's Generic Browser (depicted in Figure 7.1) allows a developer to access literally every resource that is consumed by the engine. Having a single, unified and reasonably consistent interface for creating and managing all types of resources is a big win. This is especially true considering that the resource data in most other game engines is fragmented across countless inconsistent and often cryptic tools. Just being able to *find* any resource easily in UnrealEd is a big plus.

Unreal can be less error-prone than many other engines, because assets must be explicitly imported into Unreal's resource database. This allows resources to be checked for validity very early in the production process. In most game engines, any old data can be thrown into the resource database, and you only know whether or not that data is valid when it is eventually built—or sometimes not until it is actually loaded into the game at runtime. But with Unreal, assets can be validated as soon as they are imported into UnrealEd. This means that the person who created the asset gets immediate feedback as to whether his or her asset is configured properly.

Of course, Unreal's approach has some serious drawbacks. For one thing,

all resource data is stored in a small number of large package files. These files are binary, so they are not easily merged by a revision control package like CVS, Subversion or Perforce. This presents some major problems when more than one user wants to modify resources that reside in a single package. Even if the users are trying to modify *different* resources, only one user can lock the package at a time, so the other has to wait. The severity of this problem can be reduced by dividing resources into relatively small, granular packages, but it cannot practically be eliminated.

Referential integrity is quite good in UnrealEd, but there are still some problems. When a resource is renamed or moved around, all references to it are maintained automatically using a dummy object that remaps the old resource to its new name/location. The problem with these dummy remapping objects is that they hang around and accumulate and sometimes cause problems, especially if a resource is deleted. Overall, Unreal's referential integrity is quite good, but it is not perfect.

Despite its problems, UnrealEd is by far the most user-friendly, well-integrated and streamlined asset creation toolkit, resource database and asset conditioning pipeline that I have ever worked with.

Naughty Dog's Engine

For *Uncharted: Drake's Fortune*, Naughty Dog stored its resource metadata in a MySQL database. A custom graphical user interface was written to manage the contents of the database. This tool allowed artists, game designers and programmers alike to create new resources, delete existing resources and inspect and modify resources as well. This GUI was a crucial component of the system, because it allowed users to avoid having to learn the intricacies of interacting with a relational database via SQL.

The original MySQL database used on *Uncharted* did not provide a useful history of the changes made to the database, nor did it provide a good way to roll back "bad" changes. It also did not support multiple users editing the same resource, and it was difficult to administer. Naughty Dog has since moved away from MySQL in favor of an XML file-based asset database, managed under Perforce.

Builder, Naughty Dog's resource database GUI, is depicted in Figure 7.2. The window is broken into two main sections: a tree view showing all resources in the game on the left and a properties window on the right, allowing the resource(s) that are selected in the tree view to be viewed and edited. The resource tree contains folders for organizational purposes, so that the artists and game designers can organize their resources in any way they see fit. Various types of resources can be created and managed within any folder, includ-

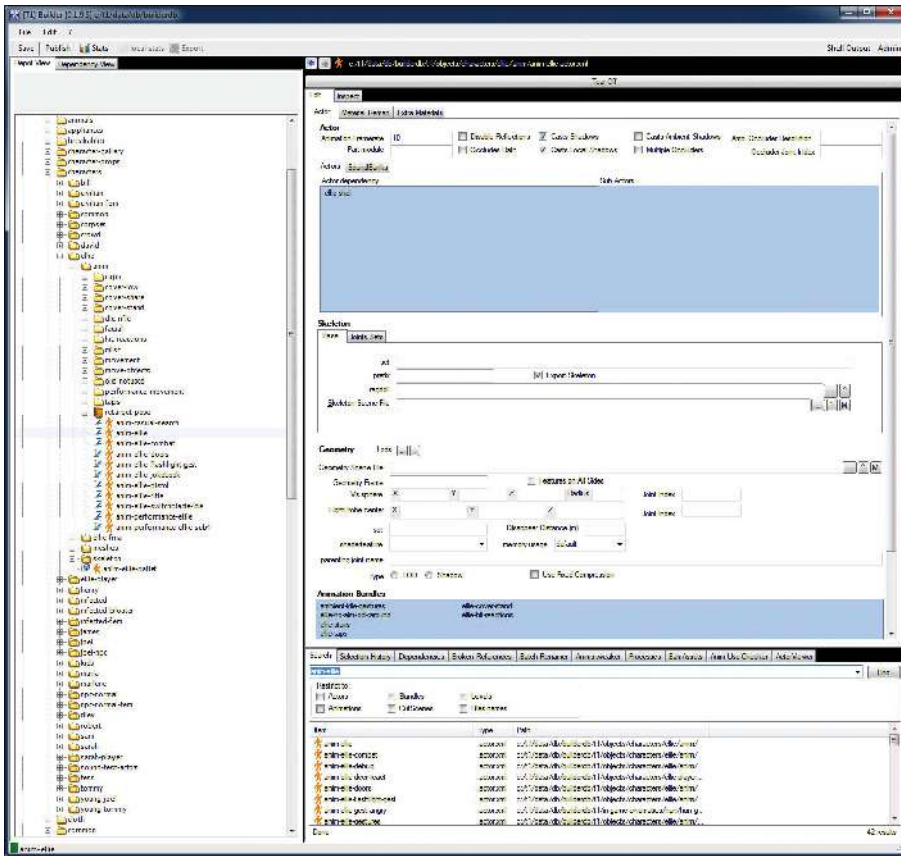


Figure 7.2. The front-end GUI for Naughty Dog's offline resource database, Builder.

ing actors and levels, and the various subresources that comprise them (primarily meshes, skeletons and animations). Animations can also be grouped into pseudo-folders known as bundles. This allows large groups of animations to be created and then managed as a unit, and prevents a lot of wasted time dragging individual animations around in the tree view.

The asset conditioning pipeline employed on the *Uncharted* and *The Last of Us* series consists of a set of resource exporters, compilers and linkers that are run from the command line. The engine is capable of dealing with a wide variety of different kinds of data objects, but these are packaged into one of two types of resource files: actors and levels. An actor can contain skeletons, meshes, materials, textures and/or animations. A level contains static background meshes, materials and textures, and also level-layout information. To

build an actor, one simply types `ba name-of-actor` on the command line; to build a level, one types `b1 name-of-level`. These command-line tools query the database to determine exactly *how* to build the actor or level in question. This includes information on how to export the assets from DCC tools like Maya and Photoshop, how to process the data, and how to package it into binary `.pak` files that can be loaded by the game engine. This is much simpler than in many engines, where resources have to be *exported manually* by the artists—a time-consuming, tedious and error-prone task.

The benefits of the resource pipeline design used by Naughty Dog include:

- *Granular resources.* Resources can be manipulated in terms of logical entities in the game—meshes, materials, skeletons and animations. These resource types are granular enough that the team almost never has conflicts in which two users want to edit the same resource simultaneously.
- *The necessary features (and no more).* The Builder tool provides a powerful set of features that meet the needs of the team, but Naughty Dog didn't waste any resources creating features they didn't need.
- *Obvious mapping to source files.* A user can very quickly determine which source assets (native DCC files, like Maya `.ma` files or photoshop `.psd` files) make up a particular resource.
- *Easy to change how DCC data is exported and processed.* Just click on the resource in question and twiddle its processing properties within the resource database GUI.
- *Easy to build assets.* Just type `ba` or `b1` followed by the resource name on the command line. The dependency system takes care of the rest.

Of course, Naughty Dog's tool chain does have some drawbacks as well, including:

- *Lack of visualization tools.* The only way to preview an asset is to load it into the game or the model/animation viewer (which is really just a special mode of the game itself).
- *The tools aren't fully integrated.* Naughty Dog uses one tool to lay out levels, another to manage the majority of resources in the resource database, and a third to set up materials and shaders (this is not part of the resource database front end). Building the assets is done on the command line. It might be a bit more convenient if all of these functions were to be integrated into a single tool. However, Naughty Dog has no plans to do this, because the benefit would probably not outweigh the costs involved.

OGRE's Resource Manager System

OGRE is a rendering engine, not a full-fledged game engine. That said, OGRE does boast a reasonably complete and very well-designed runtime resource manager. A simple, consistent interface is used to load virtually any kind of resource. And the system has been designed with extensibility in mind. Any programmer can quite easily implement a resource manager for a brand new kind of asset and integrate it easily into OGRE's resource framework.

One of the drawbacks of OGRE's resource manager is that it is a runtime-only solution. OGRE lacks any kind of offline resource database. OGRE does provide some exporters that are capable of converting a Maya file into a mesh that can be used by OGRE (complete with materials, shaders, a skeleton and optional animations). However, the exporter must be run manually from within Maya itself. Worse, all of the metadata describing how a particular Maya file should be exported and processed must be entered by the user doing the export.

In summary, OGRE's runtime resource manager is powerful and well-designed. But, OGRE would benefit a great deal from an equally powerful and modern resource database and asset conditioning pipeline on the tools side.

Microsoft's XNA

XNA is a game development toolkit by Microsoft, targeted at the PC and Xbox 360 platforms. Although it was retired by Microsoft in 2014, it's still a good resource for learning about game engines. XNA's resource management system is unique, in that it leverages the project management and build systems of the Visual Studio IDE to manage and build the assets in the game as well. XNA's game development tool, Game Studio Express, is just a plug-in to Visual Studio Express.

7.2.1.4 The Asset Conditioning Pipeline

In Section 1.7, we learned that resource data is typically created using advanced digital content creation (DCC) tools like Maya, ZBrush, Photoshop or Houdini. However, the data formats used by these tools are usually not suitable for direct consumption by a game engine. So the majority of resource data is passed through an *asset conditioning pipeline* (ACP) on its way to the game engine. The ACP is sometimes referred to as the *resource conditioning pipeline* (RCP), or simply the *tool chain*.

Every resource pipeline starts with a collection of *source assets* in native DCC formats (e.g., Maya .ma or .mb files, Photoshop .psd files, etc.). These

assets are typically passed through three processing stages on their way to the game engine:

1. *Exporters.* We need some way of getting the data out of the DCC's native format and into a format that we can manipulate. This is usually accomplished by writing a custom plug-in for the DCC in question. It is the plug-in's job to export the data into some kind of intermediate file format that can be passed to later stages in the pipeline. Most DCC applications provide a reasonably convenient mechanism for doing this. Maya actually provides three: a C++ SDK, a scripting language called MEL and most recently a Python interface as well.

In cases where a DCC application provides no customization hooks, we can always save the data in one of the DCC tool's native formats. With any luck, one of these will be an open format, a reasonably intuitive text format, or some other format that we can reverse engineer. Presuming this is the case, we can pass the file directly to the next stage of the pipeline.

2. *Resource compilers.* We often have to "massage" the raw data exported from a DCC application in various ways in order to make them game-ready. For example, we might need to rearrange a mesh's triangles into strips, or compress a texture bitmap, or calculate the arc lengths of the segments of a Catmull-Rom spline. Not all types of resources need to be compiled—some might be game-ready immediately upon being exported.
3. *Resource linkers.* Multiple resource files sometimes need to be combined into a single useful package prior to being loaded by the game engine. This mimics the process of linking together the object files of a compiled C++ program into an executable file, and so this process is sometimes called *resource linking*. For example, when building a complex composite resource like a 3D model, we might need to combine the data from multiple exported mesh files, multiple material files, a skeleton file and multiple animation files into a single resource. Not all types of resources need to be linked—some assets are game-ready after the export or compile steps.

Resource Dependencies and Build Rules

Much like compiling the source files in a C or C++ project and then linking them into an executable, the asset conditioning pipeline processes source assets (in the form of Maya geometry and animation files, Photoshop PSD files,

raw audio clips, text files, etc.), converts them into game-ready form and then links them together into a cohesive whole for use by the engine. And just like the source files in a computer program, game assets often have interdependencies. (For example, a mesh refers to one or more materials, which in turn refer to various textures.) These interdependencies typically have an impact on the order in which assets must be processed by the pipeline. (For example, we might need to build a character's skeleton before we can process any of that character's animations.) In addition, the dependencies between assets tell us which assets need to be rebuilt when a particular source asset changes.

Build dependencies revolve not only around changes to the assets themselves, but also around changes to data formats. If the format of the files used to store triangle meshes changes, for instance, all meshes in the entire game may need to be reexported and/or rebuilt. Some game engines employ data formats that are robust to version changes. For example, an asset may contain a version number, and the game engine may include code that "knows" how to load and make use of legacy assets. The downside of such a policy is that asset files and engine code tend to become bulky. When data format changes are relatively rare, it may be better to just bite the bullet and reprocess all the files when format changes do occur.

Every asset conditioning pipeline requires a set of rules that describe the interdependencies between the assets, and some kind of build tool that can use this information to ensure that the proper assets are built, in the proper order, when a source asset is modified. Some game teams roll their own build system. Others use an established tool, such as `make`. Whatever solution is selected, teams should treat their build dependency system with utmost care. If you don't, changes to sources, assets may not trigger the proper assets to be rebuilt. The result can be inconsistent game assets, which may lead to visual anomalies or even engine crashes. In my personal experience, I've witnessed countless hours wasted in tracking down problems that could have been avoided had the asset interdependencies been properly specified and the build system implemented to use them reliably.

7.2.2 Runtime Resource Management

Let us turn our attention now to how the assets in our resource database are loaded, managed and unloaded within the engine at runtime.

7.2.2.1 Responsibilities of the Runtime Resource Manager

A game engine's runtime resource manager takes on a wide range of responsibilities, all related to its primary mandate of loading resources into memory:

- Ensures that only *one copy* of each unique resource exists in memory at any given time.
- Manages the *lifetime* of each resource.
- *Loads* needed resources and *unloads* resources that are no longer needed.
- Handles loading of *composite resources*. A composite resource is a resource comprised of other resources. For example, a *3D model* is a composite resource that consists of a mesh, one or more materials, one or more textures and optionally a skeleton and multiple skeletal animations.
- Maintains *referential integrity*. This includes *internal* referential integrity (cross-references within a single resource) and *external* referential integrity (cross-references between resources). For example, a model refers to its mesh and skeleton; a mesh refers to its materials, which in turn refer to texture resources; animations refer to a skeleton, which ultimately ties them to one or more models. When loading a composite resource, the resource manager must ensure that all necessary subresources are loaded, and it must patch in all of the cross-references properly.
- Manages the *memory usage* of loaded resources and ensures that resources are stored in the appropriate place(s) in memory.
- Permits *custom processing* to be performed on a resource after it has been loaded, on a per-resource-type basis. This process is sometimes known as *logging in* or *load-initializing* the resource.
- Usually (but not always) provides a single *unified interface* through which a wide variety of resource types can be managed. Ideally a resource manager is also easily extensible, so that it can handle new types of resources as they are needed by the game development team.
- Handles *streaming* (i.e., asynchronous resource loading), if the engine supports this feature.

7.2.2.2 Resource File and Directory Organization

In some game engines (typically PC engines), each individual resource is managed in a separate “loose” file on-disk. These files are typically contained within a tree of directories whose internal organization is designed primarily for the convenience of the people creating the assets; the engine typically doesn’t care where resource files are located within the resource tree. Here’s a typical resource directory tree for a hypothetical game called *Space Evaders*:

SpaceEvaders	Root directory for entire game.
Resources	Root of all resources.
NPC	Non-player character models and animations.
Pirate	Models and animations for pirates.
Marine	Models and animations for marines.
...	
Player	Player character models and animations.
Weapons	Models and animations for weapons.
Pistol	Models and animations for the pistol.
Rifle	Models and animations for the rifle.
BFG	Models and animations for the big...uh...gun.
...	
Levels	Background geometry and level layouts.
Level1	First level's resources.
Level2	Second level's resources.
...	
Objects	Miscellaneous 3D objects.
Crate	The ubiquitous breakable crate.
Barrel	The ubiquitous exploding barrel.

Other engines package multiple resources together in a single file, such as a ZIP archive, or some other composite file (perhaps of a proprietary format). The primary benefit of this approach is improved load times. When loading data from files, the three biggest costs are *seek times* (i.e., moving the read head to the correct place on the physical media), the time required to open each individual file, and the time to read the data from the file into memory. Of these, the seek times and file-open times can be nontrivial on many operating systems. When a single large file is used, all of these costs are minimized. A single file can be organized sequentially on the disk, reducing seek times to a minimum. And with only one file to open, the cost of opening individual resource files is eliminated.

Solid-state drives (SSD) do not suffer from the seek time problems that plague spinning media like DVDs, Blu-ray discs and hard disc drives (HDD). However, no game console to date includes a solid-state drive as the primary fixed storage device (not even the PS4 and Xbox One). So designing your game's I/O patterns in order to minimize seek times is likely to be a necessity for some time to come.

The OGRE rendering engine's resource manager permits resources to exist

as loose files on disk, or as virtual files within a large ZIP archive. The primary benefits of the ZIP format are the following:

1. *It is an open format.* The `zlib` and `zzip` libraries used to read and write ZIP archives are freely available. The `zlib` SDK is totally free (see <http://www.zlib.net>), while the `zzip` SDK falls under the Lesser Gnu Public License (LGPL) (see <http://zzip.sourceforge.net>).
2. *The virtual files within a ZIP archive “remember” their relative paths.* This means that a ZIP archive “looks like” a raw file system for most intents and purposes. The OGRE resource manager identifies all resources uniquely via strings that appear to be file system paths. However, these paths sometimes identify virtual files within a ZIP archive instead of loose files on disk, and a game programmer needn’t be aware of the difference in most situations.
3. *ZIP archives may be compressed.* This reduces the amount of disk space occupied by resources. But, more importantly, it again speeds up load times, as less data need be loaded into memory from the fixed disk. This is especially helpful when reading data from a DVD-ROM or Blu-ray disk, as the data transfer rates of these devices are much slower than a hard disk drive. Hence the cost of decompressing the data after it has been loaded into memory is often more than offset by the time saved in loading less data from the device.
4. *ZIP archives are modular.* Resources can be grouped together into a ZIP file and managed as a unit. One particularly elegant application of this idea is in product localization. All of the assets that need to be localized (such as audio clips containing dialogue and textures that contain words or region-specific symbols) can be placed in a single ZIP file, and then different versions of this ZIP file can be generated, one for each language or region. To run the game for a particular region, the engine simply loads the corresponding version of the ZIP archive.

The Unreal Engine takes a similar approach, with a few important differences. In Unreal, all resources must be contained within large composite files known as *packages* (a.k.a. “pak files”). No loose disk files are permitted. The format of a package file is proprietary. The Unreal Engine’s game editor, UnrealEd, allows developers to create and manage packages and the resources they contain.

7.2.2.3 Resource File Formats

Each type of resource file potentially has a different format. For example, a mesh file is always stored in a different format than that of a texture bitmap. Some kinds of assets are stored in standardized, open formats. For example, textures are typically stored as Targa files (TGA), Portable Network Graphics files (PNG), Tagged Image File Format files (TIFF), Joint Photographic Experts Group files (JPEG) or Windows Bitmap files (BMP)—or in a standardized compressed format such as DirectX's S3 Texture Compression family of formats (S3TC, also known as DXT n or DXTC). Likewise, 3D mesh data is often exported out of a modeling tool like Maya or Lightwave into a standardized format such as OBJ or COLLADA for consumption by the game engine.

Sometimes a single file format can be used to house many different types of assets. For example, the *Granny* SDK by Rad Game Tools (<http://www.radgametools.com>) implements a flexible open file format that can be used to store 3D mesh data, skeletal hierarchies and skeletal animation data. (In fact the Granny file format can be easily repurposed to store virtually any kind of data imaginable.)

Many game engine programmers roll their own file formats for various reasons. This might be necessary if no standardized format provides all of the information needed by the engine. Also, many game engines endeavor to do as much offline processing as possible in order to minimize the amount of time needed to load and process resource data at runtime. If the data needs to conform to a particular layout in memory, for example, a raw binary format might be chosen so that the data can be laid out by an offline tool (rather than attempting to format it at runtime after the resource has been loaded).

7.2.2.4 Resource GUIDs

Every resource in a game must have some kind of *globally unique identifier* (GUID). The most common choice of GUID is the resource's file system path (stored either as a string or a 32-bit hash). This kind of GUID is intuitive, because it clearly maps each resource to a physical file on-disk. And it's guaranteed to be unique across the entire game, because the operating system already guarantees that no two files will have the same path.

However, a file system path is by no means the only choice for a resource GUID. Some engines use a less-intuitive type of GUID, such as a 128-bit hash code, perhaps assigned by a tool that guarantees uniqueness. In other engines, using a file system path as a resource identifier is infeasible. For example, Unreal Engine stores many resources in a single large file known as a *package*, so the path to the package file does not uniquely identify any one resource. To overcome this problem, an Unreal package file is organized into a folder

hierarchy containing individual resources. Unreal gives each individual resource within a package a unique name, which looks much like a file system path. So in Unreal, a resource GUID is formed by concatenating the (unique) name of the package file with the in-package path of the resource in question. For example, the *Gears of War* resource GUID `Locust_Boomer.PhysicalMaterials.LocustBoomerLeather` identifies a material called `LocustBoomerLeather` within the `PhysicalMaterials` folder of the `Locust_Boomer` package file.

7.2.2.5 The Resource Registry

In order to ensure that only one copy of each unique resource is loaded into memory at any given time, most resource managers maintain some kind of *registry* of loaded resources. The simplest implementation is a *dictionary*—i.e., a collection of *key-value pairs*. The keys contain the unique ids of the resources, while the values are typically pointers to the resources in memory.

Whenever a resource is loaded into memory, an entry for it is added to the resource registry dictionary, using its GUID as the key. Whenever a resource is unloaded, its registry entry is removed. When a resource is requested by the game, the resource manager looks up the resource by its GUID within the resource registry. If the resource can be found, a pointer to it is simply returned. If the resource cannot be found, it can either be loaded automatically or a failure code can be returned.

At first blush, it might seem most intuitive to automatically load a requested resource if it cannot be found in the resource registry. And in fact, some game engines do this. However, there are some serious problems with this approach. Loading a resource is a slow operation, because it involves locating and opening a file on disk, reading a potentially large amount of data into memory (from a potentially slow device like a DVD-ROM drive), and also possibly performing post-load initialization of the resource data once it has been loaded. If the request comes during active gameplay, the time it takes to load the resource might cause a very noticeable hitch in the game's frame rate, or even a multi-second freeze. For this reason, engines tend to take one of two alternative approaches:

1. Resource loading might be disallowed completely during active gameplay. In this model, all of the resources for a game level are loaded *en masse* just prior to gameplay, usually while the player watches a loading screen or progress bar of some kind.
2. Resource loading might be done *asynchronously* (i.e., the data might be *streamed*). In this model, while the player is engaged in level X, the re-

sources for level Y are being loaded in the background. This approach is preferable because it provides the player with a load-screen-free play experience. However, it is considerably more difficult to implement.

7.2.2.6 Resource Lifetime

The *lifetime* of a resource is defined as the time period between when it is first loaded into memory and when its memory is reclaimed for other purposes. One of the resource manager's jobs is to manage resource lifetimes—either automatically or by providing the necessary API functions to the game, so it can manage resource lifetimes manually.

Each resource has its own lifetime requirements:

- Some resources must be loaded when the game first starts up and must stay resident in memory for the entire duration of the game. That is, their lifetimes are effectively infinite. These are sometimes called *global resources* or *global assets*. Typical examples include the player character's mesh, materials, textures and core animations, textures and fonts used on the heads-up display, and the resources for all of the standard-issue weapons used throughout the game. Any resource that is visible or audible to the player throughout the entire game (and cannot be loaded on the fly when needed) should be treated as a global resource.
- Other resources have a lifetime that matches that of a particular game level. These resources must be in memory by the time the level is first seen by the player and can be dumped once the player has permanently left the level.
- Some resources might have a lifetime that is shorter than the duration of the level in which they are found. For example, the animations and audio clips that make up an *in-game cinematic* (a mini-movie that advances the story or provides the player with important information) might be loaded in advance of the player seeing the cinematic and then dumped once the cinematic has played.
- Some resources like background music, ambient sound effects or full-screen movies are streamed "live" as they play. The lifetime of this kind of resource is difficult to define, because each byte only persists in memory for a tiny fraction of a second, but the entire piece of music sounds like it lasts for a long period of time. Such assets are typically loaded in chunks of a size that matches the underlying hardware's requirements. For example, a music track might be read in 4 KiB chunks, because that might be the buffer size used by the low-level sound system. Only two

Event	A	B	C	D	E
Initial state	0	0	0	0	0
Level X counts incremented	1	1	1	0	0
Level X loads	(1)	(1)	(1)	0	0
Level X plays	1	1	1	0	0
Level Y counts incremented	1	2	2	1	1
Level X counts decremented	0	1	1	1	1
Level X unloads, level Y loads	(0)	1	1	(1)	(1)
Level Y plays	0	1	1	1	1

Table 7.2. Resource usage as two levels load and unload.

chunks are ever present in memory at any given moment—the chunk that is currently playing and the chunk immediately following it that is being loaded into memory.

The question of when to load a resource is usually answered quite easily, based on knowledge of when the asset is first seen by the player. However, the question of when to unload a resource and reclaim its memory is not so easily answered. The problem is that many resources are shared across multiple levels. We don't want to unload a resource when level X is done, only to immediately reload it because level Y needs the same resource.

One solution to this problem is to reference-count the resources. Whenever a new game level needs to be loaded, the list of all resources used by that level is traversed, and the reference count for each resource is incremented by one (but they are not loaded yet). Next, we traverse the resources of any unneeded levels and decrement their reference counts by one; any resource whose reference count drops to zero is unloaded. Finally, we run through the list of all resources whose reference count just went from zero to one and load those assets into memory.

For example, imagine that level X uses resources A, B and C, and that level Y uses resources B, C, D and E. (B and C are shared between both levels.) Table 7.2 shows the reference counts of these five resources as the player plays through levels X and Y. In this table, reference counts are shown in boldface type to indicate that the corresponding resource actually exists in memory, while a grey background indicates that the resource is not in memory. A reference count in parentheses indicates that the corresponding resource data is being loaded or unloaded.

7.2.2.7 Memory Management for Resources

Resource management is closely related to memory management, because we must inevitably decide *where* the resources should end up in memory once they have been loaded. The destination of every resource is not always the same. For one thing, certain types of resources must reside in video RAM (or, on the PlayStation 4, in a memory block that has been mapped for access via the high-speed “garlic” bus). Typical examples include textures, vertex buffers, index buffers and shader code. Most other resources can reside in main RAM, but different kinds of resources might need to reside within different address ranges. For example, a resource that is loaded and stays resident for the entire game (global resources) might be loaded into one region of memory, while resources that are loaded and unloaded frequently might go somewhere else.

The design of a game engine’s memory allocation subsystem is usually closely tied to that of its resource manager. Sometimes we will design the resource manager to take best advantage of the types of memory allocators we have available, or vice versa—we may design our memory allocators to suit the needs of the resource manager.

As we saw in Section 6.2.1.4, one of the primary problems facing any resource management system is the need to avoid fragmenting memory as resources are loaded and unloaded. We’ll discuss a few of the more-common solutions to this problem below.

Heap-Based Resource Allocation

One approach is to simply ignore memory fragmentation issues and use a general-purpose heap allocator to allocate your resources (like the one implemented by `malloc()` in C, or the global `new` operator in C++). This works best if your game is only intended to run on personal computers, on operating systems that support virtual memory allocation. On such a system, physical memory will become fragmented, but the operating system’s ability to map noncontiguous pages of physical RAM into a contiguous virtual memory space helps to mitigate some of the effects of fragmentation.

If your game is running on a console with limited physical RAM and only a rudimentary virtual memory manager (or none whatsoever), then fragmentation will become a problem. In this case, one alternative is to defragment your memory periodically. We saw how to do this in Section 6.2.2.2.

Stack-Based Resource Allocation

A stack allocator does not suffer from fragmentation problems, because memory is allocated contiguously and freed in an order opposite to that in which it was allocated. A stack allocator can be used to load resources if the following two conditions are met:

- The game is linear and level-centric (i.e., the player watches a loading screen, then plays a level, then watches another loading screen, then plays another level).
- Each level fits into memory in its entirety.

Presuming that these requirements are satisfied, we can use a stack allocator to load resources as follows: When the game first starts up, the global resources are allocated first. The top of the stack is then marked, so that we can free back to this position later. To load a level, we simply allocate its resources on the top of the stack. When the level is complete, we can simply set the stack top back to the marker we took earlier, thereby freeing all of the level's resources in one fell swoop without disturbing the global resources. This process can be repeated for any number of levels, without ever fragmenting memory. Figure 7.3 illustrates how this is accomplished.

A double-ended stack allocator can be used to augment this approach. Two stacks are defined within a single large memory block. One grows up from the bottom of the memory area, while the other grows down from the top. As long as the two stacks never overlap, the stacks can trade memory resources back and forth naturally—something that wouldn't be possible if each stack resided in its own fixed size block.

On *Hydro Thunder*, Midway used a double-ended stack allocator. The lower stack was used for persistent data loads, while the upper was used for temporary allocations that were freed every frame. Another way a double-ended stack allocator can be used is to ping-pong level loads. Such an approach was used at Bionic Games, Inc. for one of their projects. The basic idea is to load a compressed version of level B into the upper stack, while the currently active level A resides (in uncompressed form) in the lower stack. To switch from level A to level B, we simply free level A's resources (by clearing the lower stack) and then decompress level B from the upper stack into the lower stack. Decompression is generally much faster than loading data from disk, so this approach effectively eliminates the load time that would otherwise be experienced by the player between levels.

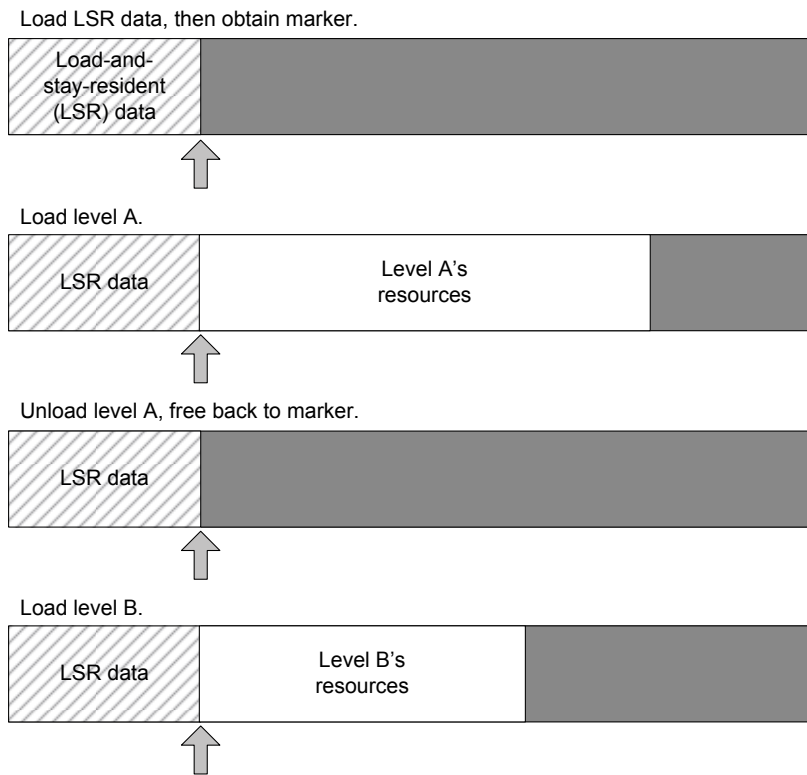


Figure 7.3. Loading resources using a stack allocator.

Pool-Based Resource Allocation

Another resource allocation technique that is common in game engines that support streaming is to load resource data in equally sized chunks. Because the chunks are all the same size, they can be allocated using a *pool allocator* (see Section 6.2.1.2). When resources are later unloaded, the chunks can be freed without causing fragmentation.

Of course, a chunk-based allocation approach requires that all resource data be laid out in a manner that permits division into equally sized chunks. We cannot simply load an arbitrary resource file in chunks, because the file might contain a contiguous data structure like an array or a very large `struct` that is larger than a single chunk. For example, if the chunks that contain an array are not arranged sequentially in RAM, the continuity of the array will be lost, and array indexing will cease to function properly. This means that all resource data must be designed with “chunkiness” in mind. Large contiguous data structures must be avoided in favor of data structures that are either small enough to fit within a single chunk or do not require contiguous RAM

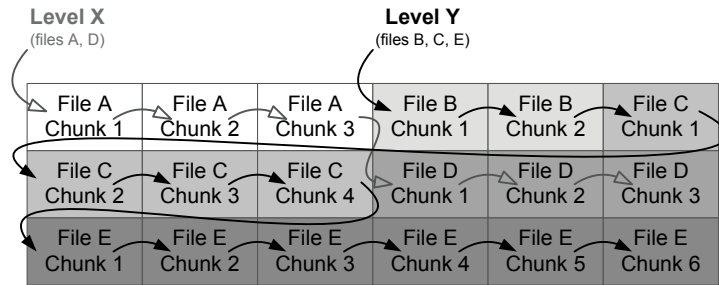


Figure 7.4. Chunky allocation of resources for levels X and Y.

to function properly (e.g., linked lists).

Each chunk in the pool is typically associated with a particular game level. (One simple way to do this is to give each level a linked list of its chunks.) This allows the engine to manage the lifetimes of each chunk appropriately, even when multiple levels with different life spans are in memory concurrently. For example, when level X is loaded, it might allocate and make use of N chunks. Later, level Y might allocate an additional M chunks. When level X is eventually unloaded, its N chunks are returned to the free pool. If level Y is still active, its M chunks need to remain in memory. By associating each chunk with a specific level, the lifetimes of the chunks can be managed easily and efficiently. This is illustrated in Figure 7.4.

One big trade-off inherent in a “chunky” resource allocation scheme is wasted space. Unless a resource file’s size is an exact multiple of the chunk size, the last chunk in a file will not be fully utilized (see Figure 7.5). Choosing a smaller chunk size can help to mitigate this problem, but the smaller the chunks, the more onerous the restrictions on the layout of the resource data. (As an extreme example, if a chunk size of one byte were selected, then no data structure could be larger than a single byte—clearly an untenable situation.) A typical chunk size is on the order of a few kibibytes. For example, at Naughty Dog, we use a chunky resource allocator as part of our resource streaming system, and our chunks are 512 KiB in size on the PS3 and 1 MiB on the PS4. You may also want to consider selecting a chunk size that is a multiple of the operating system’s I/O buffer size to maximize efficiency when loading individual chunks.

Resource Chunk Allocators

One way to limit the effects of wasted chunk memory is to set up a special memory allocator that can utilize the unused portions of chunks. As far as I’m

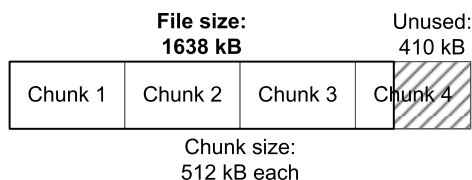


Figure 7.5. The last chunk of a resource file is often not fully utilized.

aware, there is no standardized name for this kind of allocator, but we will call it a *resource chunk allocator* for lack of a better name.

A resource chunk allocator is not particularly difficult to implement. We need only maintain a linked list of all chunks that contain unused memory, along with the locations and sizes of each free block. We can then allocate from these free blocks in any way we see fit. For example, we might manage the linked list of free blocks using a general-purpose heap allocator. Or we might map a small stack allocator onto each free block; whenever a request for memory comes in, we could then scan the free blocks for one whose stack has enough free RAM and then use that stack to satisfy the request.

Unfortunately, there's a rather grotesque-looking fly in our ointment here. If we allocate memory in the unused regions of our resource chunks, what happens when those chunks are freed? We cannot free part of a chunk—it's an all or nothing proposition. So any memory we allocate within an unused portion of a resource chunk will magically disappear when that resource is unloaded.

A simple solution to this problem is to only use our free-chunk allocator for memory requests whose lifetimes match the lifetime of the level with which a particular chunk is associated. In other words, we should only allocate memory out of level A's chunks for data that is associated exclusively with level A and only allocate from B's chunks memory that is used exclusively by level B. This requires our resource chunk allocator to manage each level's chunks separately. And it requires the users of the chunk allocator to specify which level they are allocating for, so that the correct linked list of free blocks can be used to satisfy the request.

Thankfully, most game engines need to allocate memory dynamically when loading resources, over and above the memory required for the resource files themselves. So a resource chunk allocator can be a fruitful way to reclaim chunk memory that would otherwise have been wasted.

Sectioned Resource Files

Another useful idea that is related to “chunky” resource files is the concept of *file sections*. A typical resource file might contain between one and four sections, each of which is divided into one or more chunks for the purposes of pool allocation as described above. One section might contain data that is destined for main RAM, while another section might contain video RAM data. Another section could contain temporary data that is needed during the loading process but is discarded once the resource has been completely loaded. Yet another section might contain debugging information. This debug data could be loaded when running the game in debug mode, but not loaded at all in the final production build of the game. The Granny SDK’s file system (<http://www.radgametools.com>) is an excellent example of how to implement file sectioning in a simple and flexible manner.

7.2.2.8 Composite Resources and Referential Integrity

Usually a game’s resource database consists of multiple *resource files*, each file containing one or more *data objects*. These data objects can refer to and depend upon one another in arbitrary ways. For example, a mesh data structure might contain a reference to its material, which in turn contains a list of references to textures. Usually cross-references imply dependency (i.e., if resource A refers to resource B, then both A and B must be in memory in order for the resources to be functional in the game.) In general, a game’s resource database can be represented by a *directed graph* of interdependent data objects.

Cross-references between data objects can be *internal* (a reference between two objects within a single file) or *external* (a reference to an object in a different file). This distinction is important because internal and external cross-references are often implemented differently. When visualizing a game’s resource database, we can draw dotted lines surrounding individual resource files to make the internal/external distinction clear—any edge of the graph that crosses a dotted line file boundary is an external reference, while edges that do not cross file boundaries are internal. This is illustrated in Figure 7.6.

We sometimes use the term *composite resource* to describe a self-sufficient cluster of interdependent resources. For example, a *model* is a composite resource consisting of one or more *triangle meshes*, an optional *skeleton* and an optional collection of *animations*. Each mesh is mapped with a *material*, and each material refers to one or more *textures*. To fully load a composite resource like a 3D model into memory, all of its dependent resources must be loaded as well.

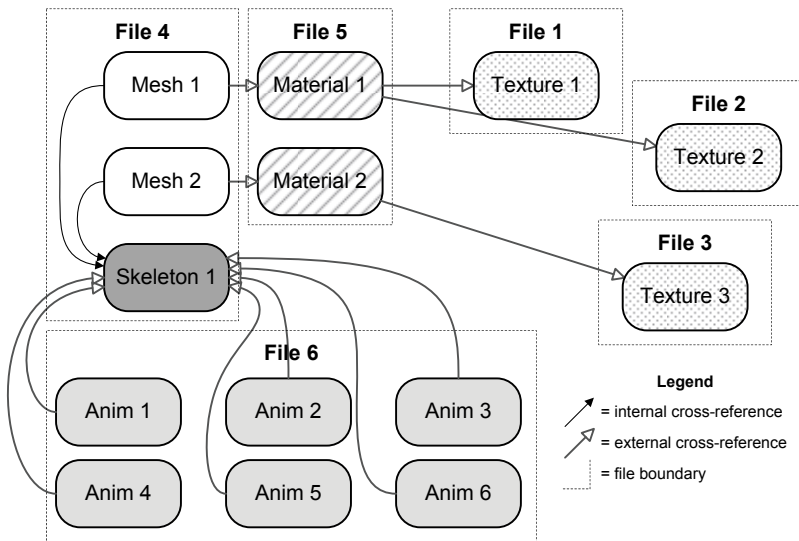


Figure 7.6. Example of a resource database dependency graph.

7.2.2.9 Handling Cross-References between Resources

One of the more-challenging aspects of implementing a resource manager is managing the cross-references between resource objects and guaranteeing that referential integrity is maintained. To understand how a resource manager accomplishes this, let's look at how cross-references are represented in memory, and how they are represented on-disk.

In C++, a cross-reference between two data objects is usually implemented via a *pointer* or a *reference*. For example, a mesh might contain the data member `Material* m_pMaterial` (a pointer) or `Material& m_material` (a reference) in order to refer to its material. However, pointers are just memory addresses—they lose their meaning when taken out of the context of the running application. In fact, memory addresses can and do change even between subsequent runs of the same application. Clearly when storing data to a disk file, we cannot use pointers to describe inter-object dependencies.

GUIDs as Cross-References

One good approach is to store each cross-reference as a string or hash code containing the unique id of the referenced object. This implies that every resource object that might be cross-referenced must have a *globally unique identifier* or *GUID*.

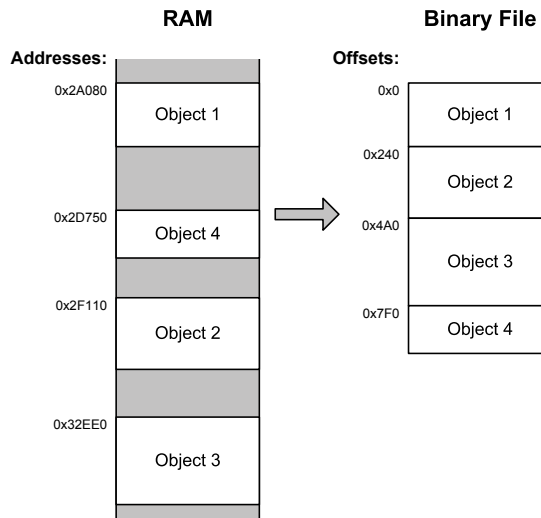


Figure 7.7. In-memory object images become contiguous when saved into a binary file.

To make this kind of cross-reference work, the runtime resource manager maintains a global resource look-up table. Whenever a resource object is loaded into memory, a pointer to that object is stored in the table with its GUID as the look-up key. After all resource objects have been loaded into memory and their entries added to the table, we can make a pass over all of the objects and convert all of their cross-references into pointers, by looking up the address of each cross-referenced object in the global resource look-up table via that object's GUID.

Pointer Fix-Up Tables

Another approach that is often used when storing data objects into a binary file is to convert the *pointers* into *file offsets*. Consider a group of C structs or C++ objects that cross-reference each other via pointers. To store this group of objects into a binary file, we need to visit each object once (and only once) in an arbitrary order and write each object's memory image into the file sequentially. This has the effect of serializing the objects into a *contiguous* image within the file, even when their memory images are not contiguous in RAM. This is shown in Figure 7.7.

Because the objects' memory images are now contiguous within the file, we can determine the *offset* of each object's image relative to the beginning of the file. During the process of writing the binary file image, we locate every

pointer within every data object, convert each pointer into an offset and store those offsets into the file in place of the pointers. We can simply overwrite the pointers with their offsets, because the offsets never require more bits to store than the original pointers. In effect, an *offset* is the binary file equivalent of a *pointer* in memory. (Do be aware of the differences between your development platform and your target platform. If you write out a memory image on a 64-bit Windows machine, its pointers will all be 64 bits wide and the resulting file won't be compatible with a 32-bit console.)

Of course, we'll need to convert the offsets back into pointers when the file is loaded into memory some time later. Such conversions are known as *pointer fix-ups*. When the file's binary image is loaded, the objects contained in the image retain their contiguous layout, so it is trivial to convert an offset into a pointer. We merely add the offset to the address of the file image as a whole. This is demonstrated by the code snippet below and illustrated in Figure 7.8.

```
U8* ConvertOffsetToPointer(U32 objectOffset,
                           U8* pAddressOfFileImage)
{
    U8* pObject = pAddressOfFileImage + objectOffset;
    return pObject;
}
```

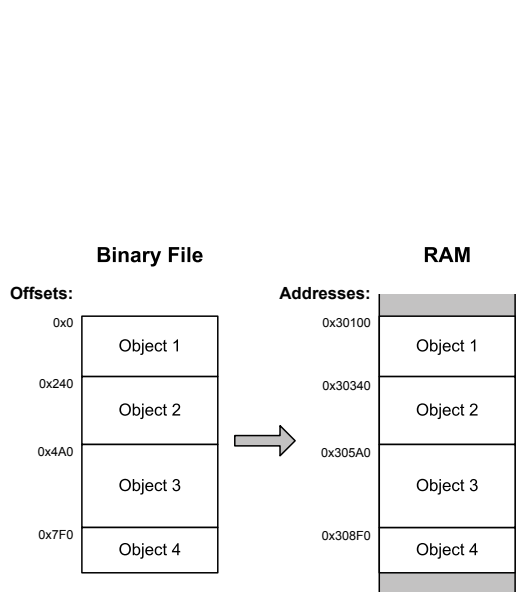


Figure 7.8. Contiguous resource file image, after it has been loaded into RAM.

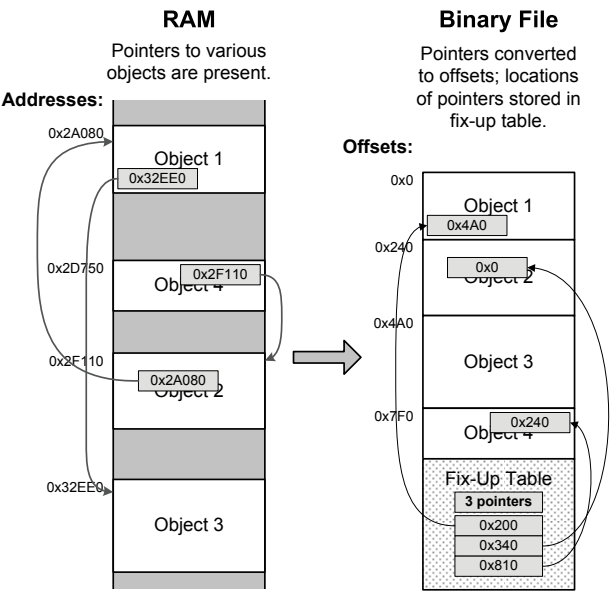


Figure 7.9. A pointer fix-up table.

The problem we encounter when trying to convert pointers into offsets, and vice versa, is how to *find* all of the pointers that require conversion. This problem is usually solved at the time the binary file is written. The code that writes out the images of the data objects has knowledge of the data types and classes being written, so it has knowledge of the locations of all the pointers within each object. The locations of the pointers are stored into a simple table known as a *pointer fix-up table*. This table is written into the binary file along with the binary images of all the objects. Later, when the file is loaded into RAM again, the table can be consulted in order to find and fix up every pointer. The table itself is just a list of offsets within the file—each offset represents a single pointer that requires fixing up. This is illustrated in Figure 7.9.

Storing C++ Objects as Binary Images: Constructors

One important step that is easy to overlook when loading C++ objects from a binary file is to ensure that the objects' constructors are called. For example, if we load a binary image containing three objects—an instance of class A, an instance of class B, and an instance of class C—then we must make sure that the correct constructor is called on each of these three objects.

There are two common solutions to this problem. First, you can simply decide not to support C++ objects in your binary files at all. In other words, restrict yourself to plain old data structures (abbreviated PODS or POD)—i.e., C structs and C++ structs and classes that contain *no virtual functions* and *trivial do-nothing constructors*. (See http://en.wikipedia.org/wiki/Plain_Old_Data_Structures for a more complete discussion of PODS.)

Second, you can save off a table containing the offsets of all non-PODS objects in your binary image along with some indication of which class each object is an instance of. Then, once the binary image has been loaded, you can iterate through this table, visit each object and call the appropriate constructor using *placement new* syntax (i.e., calling the constructor on a preallocated block of memory). For example, given the offset to an object within the binary image, we might write:

```
void* pObject = ConvertOffsetToPointer(objectOffset,
                                      pAddressOfFileImage);
::new(pObject) ClassName; // placement new syntax
```

where *ClassName* is the class of which the object is an instance.

Handling External References

The two approaches described above work very well when applied to resources in which all of the cross-references are *internal*—i.e., they only refer-

ence objects within a single resource file. In this simple case, you can load the binary image into memory and then apply the pointer fix-ups to resolve all the cross-references. But when cross-references reach out into other resource files, a slightly augmented approach is required.

To successfully represent an *external* cross-reference, we must specify not only the offset or GUID of the data object in question, but also the path to the resource file in which the referenced object resides.

The key to loading a multi-file composite resource is to load *all* of the interdependent files first. This can be done by loading one resource file and then scanning through its table of cross-references and loading any externally referenced files that have not already been loaded. As we load each data object into RAM, we can add the object's address to the master look-up table. Once all of the interdependent files have been loaded and all of the objects are present in RAM, we can make a final pass to fix up all of the pointers using the master look-up table to convert GUIDs or file offsets into real addresses.

7.2.2.10 Post-Load Initialization

Ideally, each and every resource would be completely prepared by our offline tools, so that it is ready for use the moment it has been loaded into memory. Practically speaking, this is not always possible. Many types of resources require at least some "massaging" after having been loaded in order to prepare them for use by the engine. In this book, I will use the term *post-load initialization* to refer to any processing of resource data after it has been loaded. Other engines may use different terminology. (For example, at Naughty Dog we call this *logging in* a resource.) Most resource managers also support some kind of tear-down step prior to a resource's memory being freed. (At Naughty Dog, we call this *logging out* a resource.)

Post-load initialization generally comes in one of two varieties:

- In some cases, post-load initialization is an unavoidable step. For example, on a PC, the vertices and indices that describe a 3D mesh are loaded into main RAM, but they must be transferred into video RAM before they can be rendered. This can only be accomplished at runtime, by creating a Direct X vertex buffer or index buffer, locking it, copying or reading the data into the buffer and then unlocking it.
- In other cases, the processing done during post-load initialization is avoidable (i.e., could be moved into the tools), but is done for convenience or expedience. For example, a programmer might want to add the calculation of accurate arc lengths to our engine's spline library. Rather than spend the time to modify the tools to generate the arc length data,

the programmer might simply calculate it at runtime during post-load initialization. Later, when the calculations are perfected, this code can be moved into the tools, thereby avoiding the cost of doing the calculations at runtime.

Clearly, each type of resource has its own unique requirements for post-load initialization and tear-down. So, resource managers typically permit these two steps to be configurable on a per-resource-type basis. In a non-object-oriented language like C, we can envision a look-up table that maps each type of resource to a pair of function pointers, one for post-load initialization and one for tear-down. In an object-oriented language like C++, life is even easier—we can make use of polymorphism to permit each class to handle post-load initialization and tear-down in a unique way.

In C++, post-load initialization could be implemented as a special constructor, and tear-down could be done in the class' destructor. However, there are some problems with using constructors and destructors for this purpose. For example, one typically needs to construct all loaded objects first, then apply pointer fix-ups, and finally perform post-load initialization as a separate step. As such, most developers defer post-load initialization and tear-down to plain old virtual functions. For example, we might choose to use a pair of virtual functions named something sensible like `Init()` and `Destroy()`.

Post-load initialization is closely related to a resource's memory allocation strategy, because new data is often generated by the initialization routine. In some cases, the data generated by the post-load initialization step *augments* the data loaded from the file. (For example, if we are calculating the arc lengths of the segments of a Catmull-Rom spline curve after it has been loaded, we would probably want to allocate some additional memory in which to store the results.) In other cases, the data generated during post-load initialization *replaces* the loaded data. (For example, we might allow mesh data in an older out-of-date format to be loaded and then automatically converted into the latest format for backwards compatibility reasons.) In this case, the loaded data may need to be discarded, either partially or in its entirety, after the post-load step has generated the new data.

The *Hydro Thunder* engine had a simple but powerful way of handling this. It would permit resources to be loaded in one of two ways: (a) directly into its final resting place in memory or (b) into a temporary area of memory. In the latter case, the post-load initialization routine was responsible for copying the finalized data into its ultimate destination; the temporary copy of the resource would be discarded after post-load initialization was complete. This was very useful for loading resource files that contained both relevant and irrelevant

data. The relevant data would be copied into its final destination in memory, while the irrelevant data would be discarded. For example, mesh data in an out-of-date format could be loaded into temporary memory and then converted into the latest format by the post-load initialization routine, without having to waste any memory keeping the old-format data kicking around.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

8

The Game Loop and Real-Time Simulation

Games are real-time, dynamic, interactive computer simulations. As such, *time* plays an incredibly important role in any electronic game. There are many different kinds of time to deal with in a game engine—real time, game time, the local timeline of an animation, the actual CPU cycles spent within a particular function, and the list goes on. Every engine system might define and manipulate time differently. We must have a solid understanding of all the ways time can be used in a game. In this chapter, we'll take a look at how real-time, dynamic simulation software works and explore the common ways in which time plays a role in such a simulation.

8.1 The Rendering Loop

In a graphical user interface (GUI), of the sort found on a Windows PC or a Macintosh, the majority of the screen's contents are static. Only a small part of any one window is actively changing appearance at any given moment. Because of this, graphical user interfaces have traditionally been drawn on-screen via a technique known as *rectangle invalidation*, in which only the small portions of the screen whose contents have actually changed are redrawn. Older 2D video games used similar techniques to minimize the number of pixels that needed to be drawn.

Real-time 3D computer graphics are implemented in an entirely different way. As the camera moves about in a 3D scene, the *entire contents* of the screen or window change continually, so the concept of invalid rectangles no longer applies. Instead, an illusion of motion and interactivity is produced in much the same way that a movie produces it—by presenting the viewer with a series of still images in rapid succession.

Obviously, producing a rapid succession of still images on-screen requires a loop. In a real-time rendering application, this is sometimes known as the *render loop*. At its simplest, a rendering loop is structured as follows:

```
while (!quit)
{
    // Update the camera transform based on interactive
    // inputs or by following a predefined path.
    updateCamera();

    // Update positions, orientations and any other
    // relevant visual state of any dynamic elements
    // in the scene.
    updateSceneElements();

    // Render a still frame into an off-screen frame
    // buffer known as the "back buffer".
    renderScene();

    // Swap the back buffer with the front buffer, making
    // the most recently rendered image visible
    // on-screen. (Or, in windowed mode, copy (blit) the
    // back buffer's contents to the front buffer.
    swapBuffers();
}
```

8.2 The Game Loop

A game is composed of many interacting subsystems, including device I/O, rendering, animation, collision detection and resolution, optional rigid body dynamics simulation, multiplayer networking, audio, and the list goes on. Most game engine subsystems require periodic *servicing* while the game is running. However, the *rate* at which these subsystems need to be serviced varies from subsystem to subsystem. Animation typically needs to be updated at a rate of 30 or 60 Hz, in synchronization with the rendering subsystem. However, a dynamics (physics) simulation may actually require more frequent updates (e.g., 120 Hz). Higher-level systems, like AI, might only need to be

served once or twice per second, and they needn't necessarily be synchronized with the rendering loop at all.

There are a number of ways to implement the periodic updating of our game engine subsystems. We'll explore some of the possible architectures in a moment. But for the time being, let's stick with the simplest way to update our engine's subsystems—using a single loop to update everything. Such a loop is often called the *game loop*, because it is the master loop that services every subsystem in the engine.

8.2.1 A Simple Example: Pong

Pong is a well-known genre of table tennis video games that got its start in 1958, in the form of an analog computer game called *Tennis for Two*, created by William A. Higinbotham at the Brookhaven National Laboratory and displayed on an oscilloscope. The genre is best known by its later incarnations on digital computers—the Magnavox Oddyssey game *Table Tennis* and the Atari arcade game *Pong*.

In pong, a ball bounces back and forth between two movable vertical paddles and two fixed horizontal walls. The human players control the positions of the paddles via control wheels. (Modern re-implementations allow control via a joystick, the keyboard or some other human interface device.) If the ball passes by a paddle without striking it, the other team wins the point and the ball is reset for a new round of play.

The following pseudocode demonstrates what the game loop of a pong game might look like at its core:

```
void main() // Pong
{
    initGame();

    while (true) // game loop
    {
        readHumanInterfaceDevices();

        if (quitButtonPressed())
        {
            break; // exit the game loop
        }

        movePaddles();

        moveBall();

        collideAndBounceBall();
    }
}
```

```

        if (ballImpactedSide(LEFT_PLAYER))
        {
            incremenentScore(RIGHT_PLAYER);
            resetBall();
        }
        else if (ballImpactedSide(RIGHT_PLAYER))
        {
            incrementScore(LEFT_PLAYER);
            resetBall();
        }

        renderPlayfield();
    }
}

```

Clearly this example is somewhat contrived. The original pong games were certainly not implemented by redrawing the entire screen at a rate of 30 frames per second. Back then, CPUs were so slow that they could barely muster the power to draw two lines for the paddles and a box for the ball in real time. Specialized 2D sprite hardware was often used to draw moving objects on-screen. However, we're only interested in the concepts here, not the implementation details of the original Pong.

As you can see, when the game first runs, it calls `initGame()` to do whatever set-up might be required by the graphics system, human I/O devices, audio system, etc. Then the main game loop is entered. The statement `while (true)` tells us that the loop will continue forever, unless interrupted internally. The first thing we do inside the loop is to read the human interface device(s). We check to see whether either human player pressed the “quit” button—if so, we exit the game via a `break` statement. Next, the positions of the paddles are adjusted slightly upward or downward in `movePaddles()`, based on the current deflection of the control wheels, joysticks or other I/O devices. The function `moveBall()` adds the ball's current velocity vector to its position in order to find its new position next frame. In `collideAndBounceBall()`, this position is then checked for collisions against both the fixed horizontal walls and the paddles. If collisions are detected, the ball's position is recalculated to account for any bounce. We also note whether the ball impacted either the left or right edge of the screen. This means that it missed one of the paddles, in which case we increment the other player's score and reset the ball for the next round. Finally, `renderPlayfield()` draws the entire contents of the screen.

8.3 Game Loop Architectural Styles

Game loops can be implemented in a number of different ways—but at their core, they usually boil down to one or more simple loops, with various embellishments. We'll explore a few of the more common architectures below.

8.3.1 Windows Message Pumps

On a Windows platform, games need to service messages from the Windows operating system in addition to servicing the various subsystems in the game engine itself. Windows games therefore contain a chunk of code known as a *message pump*. The basic idea is to service Windows messages whenever they arrive and to service the game engine only when no Windows messages are pending. A message pump typically looks something like this:

```
while (true)
{
    // Service any and all pending Windows messages.
    MSG msg;

    while (PeekMessage(&msg, nullptr, 0, 0) > 0)
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    // No more Windows messages to process -- run one
    // iteration of our "real" game loop.
    RunOneIterationOfGameLoop();
}
```

One of the side-effects of implementing the game loop like this is that Windows messages take precedence over rendering and simulating the game. As a result, the game will temporarily freeze whenever you resize or drag the game's window around on the desktop.

8.3.2 Callback-Driven Frameworks

Most game engine subsystems and third-party game middleware packages are structured as *libraries*. A library is a suite of functions and/or classes that can be called in any way the application programmer sees fit. Libraries provide maximum flexibility to the programmer. But, libraries are sometimes difficult to use, because the programmer must understand how to properly use the functions and classes they provide.

In contrast, some game engines and game middleware packages are structured as *frameworks*. A framework is a partially constructed application—the programmer completes the application by providing custom implementations of missing functionality within the framework (or overriding its default behavior). But he or she has little or no control over the overall flow of control within the application, because it is controlled by the framework.

In a framework-based rendering engine or game engine, the main game loop has been written for us, but it is largely empty. The game programmer can write callback functions in order to “fill in” the missing details. The OGRE rendering engine is an example of a library that has been wrapped in a framework. At the lowest level, OGRE provides functions that can be called directly by a game engine programmer. However, OGRE also provides a framework that encapsulates knowledge of how to use the low-level OGRE library effectively. If the programmer chooses to use the OGRE framework, he or she derives a class from `Ogre::FrameListener` and overrides two virtual functions: `frameStarted()` and `frameEnded()`. As you might guess, these functions are called before and after the main 3D scene has been rendered by OGRE, respectively. The OGRE framework’s implementation of its internal game loop looks something like the following pseudocode. (See `Ogre::Root::renderOneFrame()` in *OgreRoot.cpp* for the actual source code.)

```
while (true)
{
    for (each frameListener)
    {
        frameListener.frameStarted();
    }

    renderCurrentScene();

    for (each frameListener)
    {
        frameListener.frameEnded();
    }

    finalizeSceneAndSwapBuffers();
}
```

A particular game’s frame listener implementation might look something like this.


```

class GameFrameListener : public Ogre::FrameListener
{
public:
    virtual void frameStarted(const FrameEvent& event)
    {
        // Do things that must happen before the 3D scene
        // is rendered (i.e., service all game engine
        // subsystems).
        pollJoypad(event);
        updatePlayerControls(event);
        updateDynamicsSimulation(event);
        resolveCollisions(event);
        updateCamera(event);

        // etc.
    }

    virtual void frameEnded(const FrameEvent& event)
    {
        // Do things that must happen after the 3D scene
        // has been rendered.
        drawHud(event);

        // etc.
    }
};

```

8.3.3 Event-Based Updating

In games, an *event* is any interesting change in the state of the game or its environment. Some examples include: the human player pressing a button on the joypad, an explosion going off, an enemy character spotting the player, and the list goes on. Most game engines have an *event system*, which permits various engine subsystems to register interest in particular kinds of events and to respond to those events when they occur (see Section 16.8 for details). A game's event system is usually very similar to the event/messaging system underlying virtually all graphical user interfaces (for example, Microsoft Windows' window messages, the event handling system in Java's AWT or the services provided by C#'s delegate and event keywords).

Some game engines leverage their event system in order to implement the periodic servicing of some or all of their subsystems. For this to work, the event system must permit events to be posted into the future—that is, to be queued for later delivery. A game engine can then implement periodic updating by simply posting an event. In the event handler, the code can perform

whatever periodic servicing is required. It can then post a new event $1/30$ or $1/60$ of a second into the future, thus continuing the periodic servicing for as long as it is required.

8.4 Abstract Timelines

In game programming, it can be extremely useful to think in terms of *abstract timelines*. A timeline is a continuous, one-dimensional axis whose origin ($t = 0$) can lie at any arbitrary location relative to other timelines in the system. A timeline can be implemented via a simple clock variable that stores absolute time values in either integer or floating-point format.

8.4.1 Real Time

We can think of times measured directly via the CPU's high-resolution timer register (see Section 8.5.3) as lying on what we'll call the *real timeline*. The origin of this timeline is defined to coincide with the moment the CPU was last powered on or reset. It measures times in units of CPU cycles (or some multiple thereof), although these time values can be easily converted into units of seconds by multiplying them by the frequency of the high-resolution timer on the current CPU.

8.4.2 Game Time

We needn't limit ourselves to working with the real timeline exclusively. We can define as many other timeline(s) as we need in order to solve the problems at hand. For example, we can define a *game timeline* that is technically independent of real time. Under normal circumstances, game time coincides with real time. If we wish to pause the game, we can simply stop updating the game timeline temporarily. If we want our game to go into slow motion, we can update the game clock more slowly than the real-time clock. All sorts of effects can be achieved by scaling and warping one timeline relative to another.

Pausing or slowing down the game clock is also a highly useful debugging tool. To track down a visual anomaly, a developer can pause game time in order to freeze the action. Meanwhile, the rendering engine and debug fly-through camera can continue to run, as long as they are governed by a different clock (either the *real-time clock* or a separate *camera clock*). This allows the developer to fly the camera around the game world to inspect it from any angle desired. We can even support single-stepping the game clock, by advancing the game clock by one target frame interval (e.g., $1/30$ of a second) each time

a “single-step” button is pressed on the joypad or keyboard while the game is in a paused state.

When using the approach described above, it’s important to realize that the game loop is still running when the game is paused—only the game clock has stopped. Single-stepping the game by adding $1/30$ of a second to a paused game clock is not the same thing as setting a breakpoint in your main loop, and then hitting the F5 key repeatedly to run one iteration of the loop at a time. Both kinds of single-stepping can be useful for tracking down different kinds of problems. We just need to keep the differences between these approaches in mind.

8.4.3 Local and Global Timelines

We can envision all sorts of other timelines. For example, an animation clip or audio clip might have a *local timeline*, with its origin ($t = 0$) defined to coincide with the start of the clip. The local timeline measures how time progressed when the clip was originally authored or recorded. When the clip is played back in-game, we needn’t play it at the original rate. We might want to speed up an animation, or slow down an audio sample. We can even play an animation backwards by running its local clock in reverse.

Any one of these effects can be visualized as a *mapping* between the local timeline and a global timeline, such as real time or game time. To play an animation clip back at its originally authored speed, we simply map the start of the animation’s local timeline ($t = 0$) onto the desired start time ($\tau = \tau_{\text{start}}$) along the global timeline. This is shown in Figure 8.1.

To play an animation clip back at half speed, we can imagine scaling the local timeline to twice its original size prior to mapping it onto the global timeline. To accomplish this, we simply keep track of a time scale factor or playback rate R , in addition to the clip’s global start time τ_{start} . This is illustrated in Figure 8.2. A clip can even be played in reverse, by using a negative time scale ($R < 0$) as shown in Figure 8.3.

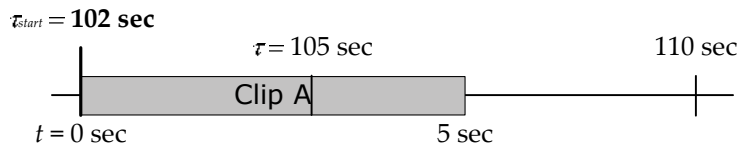


Figure 8.1. Playing an animation clip can be visualized as mapping its local timeline onto the global game timeline.

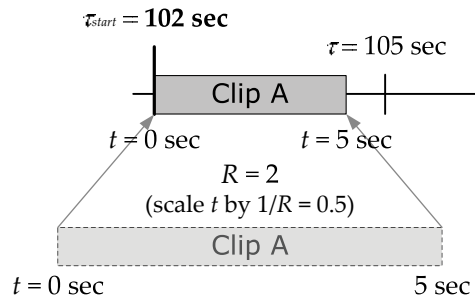


Figure 8.2. Animation playback speed can be controlled by simply scaling the local timeline prior to mapping it onto the global timeline.

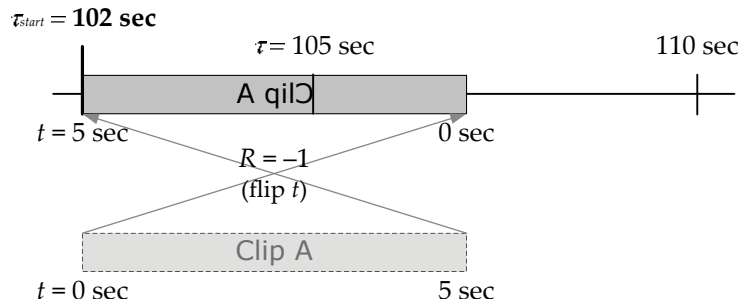


Figure 8.3. Playing an animation in reverse is like mapping the clip to the global timeline with a time scale of $R = -1$.

8.5 Measuring and Dealing with Time

In this section, we'll investigate some of the subtle and not-so-subtle distinctions between different kinds of timelines and clocks and see how they are implemented in real game engines.

8.5.1 Frame Rate and Time Deltas

The *frame rate* of a real-time game describes how rapidly the sequence of still 3D frames is presented to the viewer. The unit of *Hertz* (Hz), defined as the number of cycles per second, can be used to describe the rate of any periodic process. In games and film, frame rate is typically measured in *frames per second* (FPS), which is the same thing as Hertz for all intents and purposes. Films traditionally run at 24 FPS. Games in North America and Japan are typically rendered at 30 or 60 FPS, because this is the natural refresh rate of the NTSC color television standard used in these regions. In Europe and most of the rest

of the world, games update at 50 FPS, because this is the natural refresh rate of a PAL or SECAM color television signal.

The amount of time that elapses between frames is known as the *frame time*, *time delta* or *delta time*. This last term is commonplace because the duration between frames is often represented mathematically by the symbol Δt . (Technically speaking, Δt should really be called the *frame period*, since it is the inverse of the *frame frequency*: $T = 1/f$. But, game programmers hardly ever use the term “period” in this context.) If a game is being rendered at exactly 30 FPS, then its delta time is $1/30$ of a second, or 33.3 ms (milliseconds). At 60 FPS, the delta time is half as big, $1/60$ of a second or 16.6 ms. To really know how much time has elapsed during one iteration of the game loop, we need to measure it. We’ll see how this is done below.

We should note here that milliseconds are a common unit of time measurement in games. For example, we might say that the animation system is taking 4 ms to run, which implies that it occupies about 12% of the entire frame ($4/33.3 \approx 0.12$). Other common units include seconds and machine cycles. We’ll discuss time units and clock variables in more depth below.

8.5.2 From Frame Rate to Speed

Let’s imagine that we want to make a spaceship fly through our game world at a constant speed of 40 meters per second (or in a 2D game, we might specify this as 40 *pixels* per second). One simple way to accomplish this is to multiply the ship’s speed v (measured in meters per second) by the duration of one frame Δt (measured in seconds), yielding a change in position $\Delta x = v\Delta t$ (which is measured in *meters per frame*). This position delta can then be added to the ship’s current position x_1 , in order to find its position next frame: $x_2 = x_1 + \Delta x = x_1 + v\Delta t$.

This is actually a simple form of *numerical integration* known as the *explicit Euler* method (see Section 13.4.4). It works well as long as the speeds of our objects are roughly constant. To handle variable speeds, we need to resort to somewhat more complex integration methods. But all numerical integration techniques make use of the elapsed frame time Δt in one way or another. So it is safe to say that the *perceived speeds* of the objects in a game are dependent upon the frame duration, Δt . Hence a central problem in game programming is to determine a suitable value for Δt . In the sections that follow, we’ll discuss various ways of doing this.

8.5.2.1 Old-School CPU-Dependent Games

In many early video games, no attempt was made to measure how much real time had elapsed during the game loop. The programmers would essentially

ignore Δt altogether and instead specify the speeds of objects directly in terms of meters (or pixels, or some other distance unit) per *frame*. In other words, they were, perhaps unwittingly, specifying object speeds in terms of $\Delta x = v\Delta t$, instead of in terms of v .

The net effect of this simplistic approach was that the perceived speeds of the objects in these games were entirely dependent upon the frame rate that the game was actually achieving on a particular piece of hardware. If this kind of game were to be run on a computer with a faster CPU than the machine for which it was originally written, the game would appear to be running in fast forward. For this reason, I'll call these games *CPU-dependent games*.

Some older PCs provided a "Turbo" button to support these kinds of games. When the Turbo button was pressed, the PC would run at its fastest speed, but CPU-dependent games would run in fast forward. When the Turbo button was not pressed, the PC would mimic the processor speed of an older generation of PCs, allowing CPU-dependent games written for those PCs to run properly.

8.5.2.2 Updating Based on Elapsed Time

To make our games CPU-independent, we must measure Δt in some way, rather than simply ignoring it. Doing this is quite straightforward. We simply read the value of the CPU's high-resolution timer twice—once at the beginning of the frame and once at the end. Then we subtract, producing an accurate measure of Δt for the frame that has just passed. This delta is then made available to all engine subsystems that need it, either by passing it to every function that we call from within the game loop or by storing it in a global variable or encapsulating it within a singleton class of some kind. (We'll describe the CPU's high-resolution timer in more detail in Section 8.5.3.)

The approach outlined above is used by many game engines. In fact, I am tempted to go out on a limb and say that *most* game engines use it. However, there is one big problem with this technique: We are using the measured value of Δt taken during frame k as an estimate of the duration of the *upcoming* frame ($k + 1$). This isn't necessarily very accurate. (As they say in investing, "past performance is not a guarantee of future results.") Something might happen next frame that causes it to take much more time (or much less) than the current frame. We call such an event a *frame-rate spike*.

Using last frame's delta as an estimate of the upcoming frame can have some very real detrimental effects. For example, if we're not careful it can put the game into a "viscious cycle" of poor frame times. Let's assume that our physics simulation is most stable when updated once every 33.3 ms (i.e., at 30 Hz). If we get one bad frame, taking say 57 ms, then we might make the

mistake of stepping the physics system *twice* on the next frame, presumably to “cover” the 57 ms that has passed. Those two steps take roughly twice as long to complete as a regular step, causing the *next* frame to be at least as bad as this one was, and possibly worse. This only serves to exacerbate and prolong the problem.

8.5.2.3 Using a Running Average

It is true that game loops tend to have at least some frame-to-frame coherency. If the camera is pointed down a hallway containing lots of expensive-to-draw objects on one frame, there’s a good chance it will still be pointed down that hallway on the next. Therefore, one reasonable approach is to average the frame-time measurements over a small number of frames and use that as the next frame’s estimate of Δt . This allows the game to adapt to a varying frame rate, while softening the effects of momentary performance spikes. The longer the averaging interval, the less responsive the game will be to a varying frame rate, but spikes will have less of an impact as well.

8.5.2.4 Governing the Frame Rate

We can avoid the inaccuracy of using last frame’s Δt as an estimate of this frame’s duration altogether, by flipping the problem on its head. Rather than trying to *guess* at what next frame’s duration will be, we can instead attempt to *guarantee* that every frame’s duration will be exactly 33.3 ms (or 16.6 ms if we’re running at 60 FPS). To do this, we measure the duration of the current frame as before. If the measured duration is less than the ideal frame time, we simply put the main thread to sleep until the target frame time has elapsed. If the measured duration is more than the ideal frame time, we must “take our lumps” and wait for one more whole frame time to elapse. This is called *frame-rate governing*.

Clearly this approach only works when your game’s frame rate is reasonably close to your target frame rate on average. If your game is ping-ponging between 30 FPS and 15 FPS due to frequent “slow” frames, then the game’s quality can degrade significantly. As such, it’s still a good idea to design all engine systems so that they are capable of dealing with arbitrary frame durations. During development, you can leave the engine in “variable frame rate” mode, and everything will work as expected. Later on, when the game is getting closer to achieving its target frame rate consistently, we can switch on frame-rate governing and start to reap its benefits.

Keeping the frame rate consistent can be important for a number of reasons. Some engine systems, such as the numerical integrators used in a physics simulation, operate best when updated at a constant rate. A consistent frame rate

also looks better, and as we'll see in the next section, it can be used to avoid the *tearing* that can occur when the video buffer is updated at a rate that doesn't match the refresh rate of the monitor (see Section 8.5.2.5).

In addition, when elapsed frame times are consistent, features like *record and playback* become a lot more reliable. As its name implies, the record-and-playback feature allows a player's gameplay experience to be recorded and later played back in exactly the same way. This can be a fun game feature, and it's also a valuable testing and debugging tool. For example, difficult-to-find bugs can be reproduced by simply playing back a recorded game that demonstrates the bug.

To implement record and playback, we make note of every relevant event that occurs during gameplay, saving each one in a list along with an accurate time stamp. The list of events can then be replayed with exactly the same timing, using the same initial conditions and an identical initial random seed. In theory, doing this should produce a gameplay experience that is indistinguishable from the original playthrough. However, if the frame rate isn't consistent, things may not happen in exactly the same order. This can cause "drift," and pretty soon your AI characters are flanking when they should have fallen back.

8.5.2.5 Screen Tearing and V-Sync

A visual anomaly known as *screen tearing* occurs when the back buffer is swapped with the front buffer while the screen has only been partially "drawn" by the video hardware. When tearing occurs, a portion of the screen shows the new image, while the remainder shows the old one. To avoid tearing, many rendering engines wait for the *vertical blanking interval* of the monitor before swapping buffers.

Older CRT monitors and TVs "draw" the contents of the in-memory frame buffer by exciting phosphors on the screen via a beam of electrons that scans from left-to-right and top-to-bottom. On such displays, the v-blank interval is the time during which the electron gun is "blanked" (turned off) while it is being reset to the top-left corner of the screen. Modern LCD, plasma and LED displays no longer use an electron beam, and they don't require any time between finishing the draw of the last scan line of one frame and the first scan line of the next. But the v-blank interval still exists, in part because video standards were established when CRTs were the norm, and in part because of the need to support older displays.

Waiting for the v-blank interval is called *v-sync*. It is really just another form of frame-rate governing, because it effectively clamps the frame rate of the main game loop to a multiple of the screen's refresh rate. For example, on an NTSC monitor that refreshes at a rate of 60 Hz, the game's real update rate

is effectively quantized to a multiple of $1/60$ of a second. If more than $1/60$ of a second elapses between frames, we must wait until the next v-blank interval, which means waiting $2/60$ of a second (30 FPS). If we miss two v-blanks, then we must wait a total of $3/60$ of a second (20 FPS) and so on. Also, be careful not to make assumptions about the frame rate of your game, even when it is synchronized to the v-blank interval; if your game supports them, you must keep in mind that the PAL and SECAM standards are based around an update rate of 50 Hz, not 60 Hz.

8.5.3 Measuring Real Time with a High-Resolution Timer

We've talked a lot about measuring the amount of real "wall clock" time that elapses during each frame. In this section, we'll investigate how such timing measurements are made in detail.

Most operating systems provide a function for querying the system time, such as the C standard library function `time()`. However, such functions are not suitable for measuring elapsed times in a real-time game because they do not provide sufficient resolution. For example, `time()` returns an integer representing the number of *seconds* since midnight, January 1, 1970, so its resolution is one second—far too coarse, considering that a frame takes only tens of milliseconds to execute.

All modern CPUs contain a *high-resolution timer*, which is usually implemented as a hardware register that counts the number of CPU cycles (or some multiple thereof) that have elapsed since the last time the processor was powered on or reset. This is the timer that we should use when measuring elapsed time in a game, because its resolution is usually on the order of the duration of a few CPU cycles. For example, on a 3 GHz Pentium processor, the high-resolution timer increments once per CPU cycle, or 3 billion times per second. Hence the resolution of the high-res timer is $1/3 \text{ billion} = 3.33 \times 10^{-10} \text{ seconds} = 0.333 \text{ ns}$ (one-third of a nanosecond). This is more than enough resolution for all of our time-measurement needs in a game.

Different microprocessors and different operating systems provide different ways to query the high-resolution timer. On a Pentium, a special instruction called `rdtsc` (read time-stamp counter) can be used, although the Win32 API wraps this facility in a pair of functions: `QueryPerformanceCounter()` reads the 64-bit counter register and `QueryPerformanceFrequency()` returns the number of counter increments per second for the current CPU. On a PowerPC architecture, such as the chips found in the Xbox 360 and PlayStation 3, the instruction `mftb` (move from time base register) can be used to read the two 32-bit time base registers, while on other PowerPC architectures, the instruction `mfspr` (move from special-purpose register) is used instead.

A CPU's high-resolution timer register is 64 bits wide on most processors, to ensure that it won't wrap too often. The largest possible value of a 64-bit unsigned integer is $0xFFFFFFFFFFFFFFFF \approx 1.8 \times 10^{19}$ clock ticks. So, on a 3 GHz Pentium processor that updates its high-res timer once per CPU cycle, the register's value will wrap back to zero once every 195 years or so—definitely not a situation we need to lose too much sleep over. In contrast, a 32-bit integer clock will wrap after only about 1.4 seconds at 3 GHz.

8.5.3.1 High-Resolution Clock Drift

Be aware that even timing measurements taken via a high-resolution timer can be inaccurate in certain circumstances. For example, on some multi-core processors, the high-resolution timers are independent on each core, and they can (and do) drift apart. If you try to compare absolute timer readings taken on different cores to one another, you might end up with some strange results—even negative time deltas. Be sure to keep an eye out for these kinds of problems.

8.5.4 Time Units and Clock Variables

Whenever we measure or specify time durations in a game, we have two choices to make:

1. What *time units* should be used? Do we want to store our times in seconds, or milliseconds, or machine cycles...or in some other unit?
2. What *data type* should be used to store time measurements? Should we employ a 64-bit integer, or a 32-bit integer, or a 32-bit floating point variable?

The answers to these questions depend on the intended purpose of a given measurement. This gives rise to two more questions: How much precision do we need? And what range of magnitudes do we expect to be able to represent?

8.5.4.1 64-Bit Integer Clocks

We've already seen that a 64-bit unsigned integer clock, measured in machine cycles, supports both an extremely high precision (a single cycle is 0.333 ns in duration on a 3 GHz CPU) and a broad range of magnitudes (a 64-bit clock wraps once roughly every 195 years at 3 GHz). So this is the most flexible time representation, presuming you can afford 64 bits worth of storage.

8.5.4.2 32-Bit Integer Clocks

When measuring relatively short durations with high precision, we can turn to a 32-bit integer clock, measured in machine cycles. For example, to profile the performance of a block of code, we might do something like this:

```
// Grab a time snapshot.
U64 begin_ticks = readHiResTimer();

// This is the block of code whose performance we wish
// to measure.
doSomething();
doSomethingElse();
nowReallyDoSomething();

// Measure the duration.
U64 end_ticks = readHiResTimer();
U32 dt_ticks = static_cast<U32>(end_ticks - begin_ticks);

// Now use or cache the value of dt_ticks...
```

Notice that we still store the raw time measurements in 64-bit integer variables. Only the time delta `dt` is stored in a 32-bit variable. This circumvents potential problems with wrapping at the 32-bit boundary. For example, if `begin_ticks = 0x12345678FFFFFFB7` and `end_ticks = 0x1234567900000039`, then we would measure a negative time delta if we were to truncate the individual time measurements to 32 bits each prior to subtracting them.

8.5.4.3 32-Bit Floating-Point Clocks

Another common approach is to store relatively small time deltas in floating-point format, measured in units of seconds. To do this, we simply multiply a duration measured in CPU cycles by the CPU's clock frequency, which is in cycles per second. For example:

```
// Start off assuming an ideal frame time (30 FPS).
F32 dt_seconds = 1.0f / 30.0f;

// Prime the pump by reading the current time.
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    runOneIterationOfGameLoop(dt_seconds);

    // Read the current time again, and calculate the delta.
```

```

U64 end_ticks = readHiResTimer();

// Check our units: seconds = ticks / (ticks/second)
dt_seconds = (F32)(end_ticks - begin_ticks)
             / (F32)getHiResTimerFrequency();

// Use end_ticks as the new begin_ticks for next frame.
begin_ticks = end_ticks;
}

```

Notice once again that we must be careful to subtract the two 64-bit time measurements *before* converting them into floating-point format. This ensures that we don't store too large a magnitude into a 32-bit floating-point variable.

8.5.4.4 Limitations of Floating-Point Clocks

Recall that in a 32-bit IEEE float, the 23 bits of the mantissa are dynamically distributed between the whole and fractional parts of the value, by way of the exponent (see Section 3.3.1.4). Small magnitudes require only a few bits, leaving plenty of bits of precision for the fraction. But once the magnitude of our clock grows too large, its whole part eats up more bits, leaving fewer bits for the fraction. Eventually, even the least-significant bits of the whole part become implicit zeros. This means that we must be cautious when storing *long durations* in a floating-point clock variable. If we keep track of the amount of time that has elapsed since the game was started, a floating-point clock will eventually become inaccurate to the point of being unusable.

Floating-point clocks are usually only used to store relatively short time deltas, measuring at most a few minutes and more often just a single frame or less. If an absolute-valued floating-point clock is used in a game, you will need to reset the clock to zero periodically to avoid accumulation of large magnitudes.

8.5.4.5 Other Time Units

Some game engines allow timing values to be specified in a game-defined unit that is fine-grained enough to permit an integer format to be used (as opposed to requiring a floating-point format), precise enough to be useful for a wide range of applications within the engine, and yet large enough that a 32-bit clock won't wrap too often. One common choice is a 1/300 second time unit. This works well because (a) it is fine-grained enough for many purposes, (b) it only wraps once every 165.7 days and (c) it is an even multiple of both NTSC and PAL refresh rates. A 60 FPS frame would be 5 such units in duration, while a 50 FPS frame would be 6 units in duration.

Obviously a 1/300 second time unit is not precise enough to handle subtle effects, like time-scaling an animation. (If we tried to slow a 30 FPS animation down to less than 1/10 of its regular speed, we'd be out of precision!) So for many purposes, it's still best to use floating-point time units or machine cycles. But a 1/300 second time unit can be used effectively for things like specifying how much time should elapse between the shots of an automatic weapon, or how long an AI-controlled character should wait before starting his patrol, or the amount of time the player can survive when standing in a pool of acid.

8.5.5 Dealing with Breakpoints

When your game hits a breakpoint, its loop stops running and the debugger takes over. However, if your game is running on the same computer that your debugger is running on, then the CPU continues to run, and the real-time clock continues to accrue cycles. A large amount of wall clock time can pass while you are inspecting your code at a breakpoint. When you allow the program to continue, this can lead to a measured frame time many seconds, or even minutes or hours in duration!

Clearly if we allow such a huge delta time to be passed to the subsystems in our engine, bad things will happen. If we are lucky, the game might continue to function properly after lurching forward many seconds in a single frame. Worse, the game might just crash.

A simple approach can be used to get around this problem. In the main game loop, if we ever measure a frame time in excess of some predefined upper limit (e.g., 1 second), we can assume that we have just resumed execution after a breakpoint, and we set the delta-time artificially to 1/30 or 1/60 of a second (or whatever the target frame rate is). In effect, the game becomes frame-locked for one frame, in order to avoid a massive spike in the measured frame duration.

```
// Start off assuming the ideal dt (30 FPS).
F32 dt = 1.0f / 30.0f;

// Prime the pump by reading the current time.
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    updateSubsystemA(dt);
    updateSubsystemB(dt);
    // ...
    renderScene();
    swapBuffers();
}
```

```

    // Read the current time again, and calculate an
    // estimate of next frame's delta time.
    U64 end_ticks = readHiResTimer();

    dt = (F32)(end_ticks - begin_ticks)
        / (F32)getHiResTimerFrequency();

    // If dt is too large, we must have resumed from a
    // breakpoint -- frame-lock to the target rate this
    // frame.
    if (dt > 1.0f)
    {
        dt = 1.0f/30.0f;
    }

    // Use end_ticks as the new begin_ticks for next frame.
    begin_ticks = end_ticks;
}

```

8.6 Multiprocessor Game Loops

In Chapter 4, we explored the parallel computing hardware that is now ubiquitous in consumer-grade computers, mobile devices and game consoles, and we learned the mechanics of how to write *concurrent* software that takes advantage of these parallel computing resources. In this section, we'll discuss various ways in which this knowledge can be applied to a game engine's *game loop*.

8.6.1 Task Decomposition

To take advantage of parallel computing hardware, we need to *decompose* the various tasks that are performed during each iteration of our game loop into multiple subtasks, each of which can be executed in parallel. This act of decomposition transforms our software from a *sequential* program into a *concurrent* one.

There are many ways to decompose a software system for concurrency, but as we discussed in Section 4.1.3, we can place all of them roughly into one of two categories: *task parallelism* and *data parallelism*.

Task parallelism naturally fits situations in which a number of *different* things need to be done, and we opt to do them in parallel across multiple cores. For example, we might attempt to perform animation blending in parallel with collision detection during each iteration of our game loop, or we might submit

primitives to the GPU for rendering of frame N in parallel with starting to update the state of the game world for frame $N + 1$.

Data parallelism is best suited to situations in which a *single* computation needs to be performed repeatedly on a large number of data elements. The GPU is probably the best example of data parallelism in action—a GPU performs millions of per-pixel and per-vertex calculations every frame by distributing the work across a large number of processing cores that operate in parallel. However, as we'll see in the following sections, data parallelism isn't only found on the GPU—there are plenty of tasks performed by the CPU during the game loop that can benefit from data parallelism as well.

In the following sections, we'll take a look at a number of different ways to subdivide the work that's done by the game loop, some of which employ a task-parallel approach, others of which rely on data parallelism. We'll explore the pros and cons of each approach, and then see how a general-purpose *job system* can be a useful tool for converting literally any workload into a concurrent operation that can take advantage of hardware parallelism.

8.6.2 One Thread per Subsystem

One simple way to decompose our game loop for concurrency would be to assign particular engine subsystems to run in separate threads. For example, the rendering engine, the collision and physics simulation, the animation pipeline, and the audio engine could each be assigned to its own thread. A master thread would control and synchronize the operations of these secondary subsystem threads, and would also continue to handle the lion's share of the game's high-level logic (the main game loop). On a hardware platform with multiple physical CPUs, this design would allow the threaded engine subsystems to execute in parallel with one another and with the main game loop. This is a simple example of *task parallelism*; it is depicted in Figure 8.4.

There are a number of problems with the simple approach of assigning each engine subsystem to its own thread. For one thing, the number of engine subsystems probably won't match the number of cores on our gaming platform. As a result, we'll probably end up with more threads than we have cores, and some subsystem threads will need to share a core via time-slicing.

Another problem is that each engine subsystem requires a different amount of processing each frame. This means that while some threads (and their corresponding CPU cores) are highly utilized each frame, others may sit idle for large portions of the frame.

Yet another issue is that some engine subsystems *depend* on the data produced by others. For example, the rendering and audio subsystems cannot

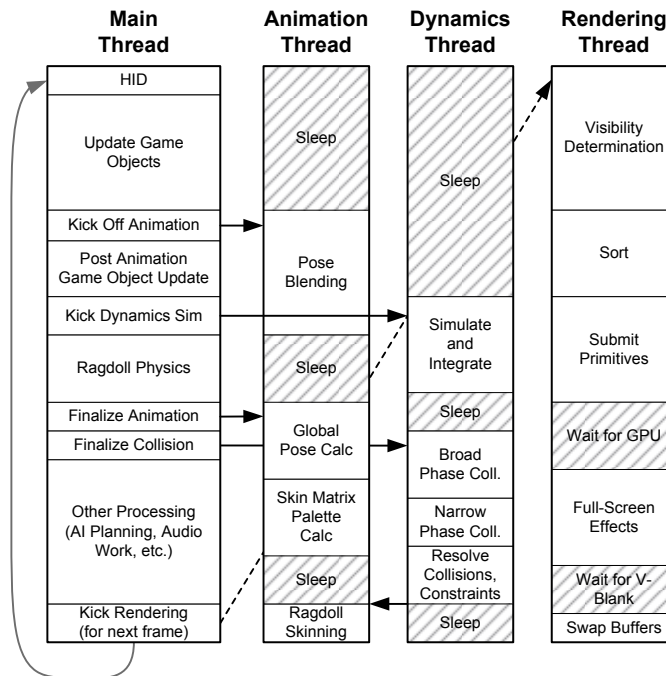


Figure 8.4. One thread per major engine subsystem.

start doing their work for frame N until the animation, collision and physics systems have completed their work for frame N . We cannot run two subsystems in parallel if they depend on one another like this.

As a result of all of these problems, attempting to assign each engine subsystem to its own thread is really not a practical design. We can do better.

8.6.3 Scatter/Gather

Many of the tasks performed during a single iteration of the game loop are data-intensive. For example, we may need to process a large number of ray cast requests, blend together a large batch of animation poses, or calculate world-space matrices for every object in a massive interactive scene. One way to take advantage of parallel computing hardware to perform these kinds of tasks is to use a divide-and-conquer approach. Instead of attempting to process 9000 ray casts one at a time on a single CPU core, we can divide the work into, say, six batches of 1500 ray casts each, and then execute one batch on each

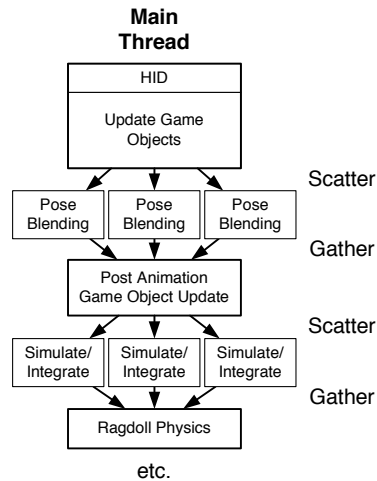


Figure 8.5. Scatter/gather used to parallelize selected CPU-intensive parts of the game loop.

of the six¹ CPU cores on our PS4 or Xbox One. This approach is a form of *data parallelism*.

In distributed systems terminology, this is known as a *scatter/gather* approach, because a unit of work is divided into smaller subunits, distributed onto multiple processing cores for execution (scatter), and then the results are combined or finalized in an appropriate manner once all workloads have been completed (gather).

8.6.3.1 Scatter/Gather in the Game Loop

In the context of our game loop, one or more scatter/gather operations might be performed, at various times during one iteration of the game loop, by the “master” game loop thread. This architecture is illustrated in Figure 8.5.

Given a dataset containing N data items that require processing, the master thread would divide the work into m batches, each containing roughly N/m elements. (The value of m would probably be determined based on the number of available cores in the system, although this may not be the case if, for example, we wish to leave some cores free for other work.) The master thread would then spawn m worker threads and provide each with a start index and count,

¹Both the PS4 and the Xbox One allow developers to access some of the processing power of a seventh core. Not all of its bandwidth is available to developers because this core is also utilized by the operating system. The eighth core in these 8-core machines is entirely off-limits. This is done to allow for the fact that some faulty cores are inevitably produced during any CPU’s fabrication process.

allowing it to process its assigned subset of the data. Each worker thread might update the data items in-place, or (usually better) it might produce output data into a separate preallocated buffer (one per worker thread).

Having successfully scattered the workload, the master thread would then be free to perform some other useful work while it waits for the worker threads to complete their tasks.

At some point later in the frame, the master thread would gather the results by waiting until all of the worker threads have terminated, perhaps using a function such as `pthread_join()`. If the worker threads have all exited, this function will return immediately; however, if any worker threads are still running, this call would have the effect of putting the master thread to sleep.

Once the gather step has been completed, the master thread could potentially combine the results in whatever manner is appropriate. For example, after blending animations together, the next step might be to calculate skinning matrices—this step would be kicked off only after all animation blending threads have completed their work. We presented an example very much like this in Section 4.4.6 when we covered thread creation and joining.

8.6.3.2 SIMD for Scatter/Gather

In Section 4.10, we explored loop *vectorization* as a means of leveraging SIMD parallelism to improve the performance of data-intensive work. This is really just another form of the scatter/gather approach, performed at a very fine level of granularity. SIMD could be used in lieu of thread-based scatter/gather, but it would likely be used in conjunction with it (with each worker thread utilizing vectorization internally to perform its work).

8.6.3.3 Making Scatter/Gather More Efficient

The scatter/gather approach is an intuitive way to distribute data-intensive work across multiple cores. However, as we've described it above, this parallelization method does suffer from one big problem—spawning threads is expensive. Spawning a thread involves a kernel call, as does joining the master thread with its workers. The kernel itself does quite a lot of set-up and tear-down work whenever threads come and go. So spawning a bunch of threads every time we want to perform a scatter/gather is impractical.

We could mitigate the costs of spawning threads by using of a pool of pre-spawned threads. Some operating systems like Windows provide an API for creating and managing a pool of threads. (See for example <https://bit.ly/2H8Chlp>.) In the absence of such an API, you can always implement a simple thread pool yourself, using *condition variables*, *semaphores*, atomic Boolean

variables, or some other mechanism in order to synchronize the activities of the threads.

We'd like our thread pool to be capable of performing a wide variety of scatter/gather operations over the course of each frame. This means that we can no longer simply spawn a bunch of threads for each scatter/gather, whose entry point function does one particular computation. Instead, each thread in the pool would have to be capable of performing any of the scatter/gather operations we might want to perform during any iteration of our game loop. We could imagine using a giant switch statement to implement this, but that idea sounds clunky, ugly and hard to maintain. Really, what we want is a general-purpose system for executing units of work concurrently, across the available cores on our target hardware.

8.6.4 Job Systems

A *job system* is a general-purpose system for executing arbitrary units of work, typically called *jobs*, across multiple cores. With a job system, a game programmer can subdivide each iteration of the game loop into a relatively large number of independent jobs, and submit them to the job system for execution. The job system maintains a queue of submitted jobs, and it schedules those jobs across the available cores, either by submitting them to be executed by worker threads in a thread pool, or by some other means. In a sense, a job system is like a custom, light-weight operating system kernel, except that instead of scheduling threads to run on the available cores, it schedules jobs.

Jobs can be arbitrarily fine-grained, and in a real game engine many are independent of one another. As shown in Figure 8.6, these facts help to maximize processor utilization. This architecture also scales up or down naturally to hardware with any number of CPU cores.

8.6.4.1 Typical Job System Interface

A typical job system provides a simple, easy-to-use API that looks very similar to the API for a threading library. There's a function for spawning a job (the equivalent of `pthread_create()`, often called *kicking* a job), a function that allows one job to wait for one or more other jobs to terminate (the equivalent of `pthread_join()`), and perhaps a way for a job to self-terminate "early" (before returning from its entry point function). A job system must also provide spin locks or mutexes of some kind for performing critical concurrent operations in an *atomic* manner. It may also provide facilities for putting jobs to sleep and waking them back up, via condition variables or some similar mechanism.

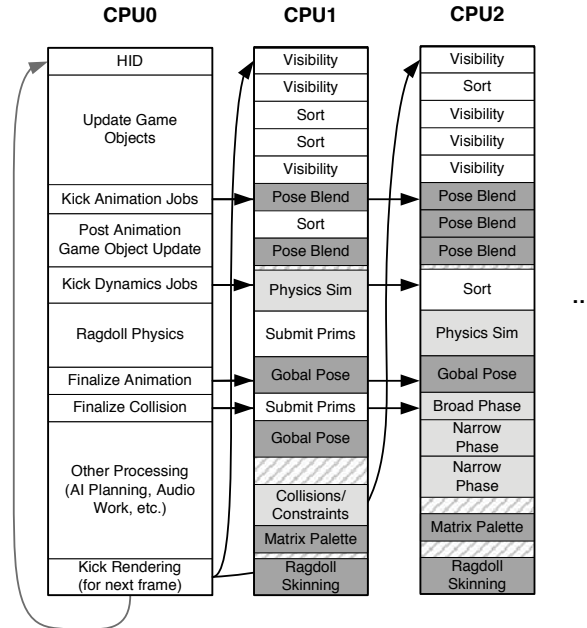


Figure 8.6. In a job model, work is broken down into fine-grained chunks that can be picked up by any available processor. This can help maximize processor utilization while providing the main game loop with improved flexibility.

To kick a job, we need to tell the job system *what* job to perform, and *how* to perform it. Normally this information is passed to the `KickJob()` function via a small data structure which we'll call a *job declaration* here.

At minimum, a job declaration must contain a pointer to the job's entry point function. It's also important to be able to pass arbitrary input parameters to the job's entry point function. This could be done in various ways, but the simplest is to provide a single parameter of type `uintptr_t`, which will be passed to the entry point function when the job actually runs. This allows us to pass simple information like a single Boolean or integer to the job with little fuss; but because a pointer can be safely cast to a `uintptr_t`, we can also use this job parameter to pass a pointer to an arbitrary data structure, that itself contains whatever input parameters the job might require.

A job system may also provide a priority mechanism, much as most thread libraries do. In this case, a priority might also be included within the job declaration.

Here's an example of a simple job declaration, along with a simple job system API:

```

namespace job
{
    // signature of all job entry points
    typedef void EntryPoint(uintptr_t param);

    // allowable priorities
    enum class Priority
    {
        LOW, NORMAL, HIGH, CRITICAL
    };

    // counter (implementation not shown)
    struct Counter ... ;
    Counter* AllocCounter();
    void FreeCounter(Counter* pCounter);

    // simple job declaration
    struct Declaration
    {
        EntryPoint*      m_pEntryPoint;
        uintptr_t        m_param;
        Priority          m_priority;
        Counter*         m_pCounter;
    };

    // kick a job
    void KickJob(const Declaration& decl);
    void KickJobs(int count, const Declaration aDecl[]);

    // wait for job to terminate (for its Counter to become zero)
    void WaitForCounter(Counter* pCounter);

    // kick jobs and wait for completion
    void KickJobAndWait(const Declaration& decl);
    void KickJobsAndWait(int count, const Declaration aDecl[]);
}

```

You'll notice there's an opaque type here called `job::Counter`. Counters allow one job to go to sleep and wait until one or more other jobs have completed executing. We'll discuss job counters in Section 8.6.4.6.

8.6.4.2 A Simple Job System Based on a Thread Pool

As we hinted at earlier, it's possible to build a job system around a pool of worker threads. It's a good idea to spawn one thread for each CPU that's present in the target machine, and use affinity to lock each thread to one core. Each worker thread sits in an infinite loop processing job requests that are

provided to it by other threads (and/or other jobs). At the top of this infinite loop, the worker thread goes to sleep (possibly via a condition variable), waiting for a job request to become available. When a request is received, the worker thread wakes up, calls the job's entry point function and passes the input parameter from the job declaration to it. When the entry point function returns, that indicates that the work is complete, so the worker thread goes back to the top of its infinite loop to execute more jobs. If none are available, it goes back to sleep, waiting for another job request to come in.

Here's what a job worker thread might look like under the hood. Please bear in mind that this is not a complete implementation; it's just for illustrative purposes.

```
namespace job
{
    void* JobWorkerThread(void*)
    {
        // keep on running jobs forever...
        while (true)
        {
            Declaration declCopy;

            // wait for a job to become available
            pthread_mutex_lock(&g_mutex);
            while (!g_ready)
            {
                pthread_cond_wait(&g_jobCv, &g_mutex);
            }

            // copy the JobDeclaration locally and
            // release our mutex lock
            declCopy = GetNextJobFromQueue();
            pthread_mutex_unlock(&g_mutex);

            // run the job
            declCopy.m_pEntryPoint(declCopy.m_param);

            // job is done! rinse and repeat...
        }
    }
}
```

8.6.4.3 A Limitation of Thread-Based Jobs

Let's imagine writing a job that updates the state of an AI-driven non-player character (NPC). The job's entry point function might look something like this:

```

void NpcThinkJob(uintparam_t param)
{
    npc* pNpc = reinterpret_cast<npc*>(param);

    pNpc->StartThinking();
    pNpc->DoSomeMoreUpdating();
    // ...

    // now let's cast a ray to see if we're aiming
    // at anything interesting -- this involves
    // kicking off another job that will run on
    // a different code (worker thread)
    RayCastHandle hRayCast = CastGunAimRay(pNpc);

    // the results of the ray cast aren't going to
    // be ready until later this frame, so let's
    // go to sleep until it's ready
    WaitForRayCast(hRayCast);

    // zzz...

    // wake up!

    // now fire my weapon, but only if the ray
    // cast indicates that we are aiming at an
    // enemy
    pNpc->TryFireWeaponAtTarget(hRayCast);

    // ...
}

```

This job seems simple enough: It performs some updates, kicks off a ray cast job (on another worker thread/core) to determine at what object the NPC is aiming his gun. The NPC then fires his weapon, but only if the ray cast reports that an enemy is in his sights.

Unfortunately this kind of job wouldn't work if we tried to run it within the simple job system that we described in the previous section. The problem is the call to `WaitForRayCast()`. In our simple thread-pool-based job system, every job must *run to completion* once it starts running. It cannot “go to sleep” waiting for the results of the ray cast, allowing other jobs to run on the worker thread, and then “wake up” later, when the ray cast results are ready.

This limitation arises because in our simple system, each running job shares the same call stack as the worker thread that invoked it. To put a job to sleep, we would need to effectively *context switch* to another job. That would involve saving off the outgoing job's call stack and registers, and then overwriting the

worker thread's call stack with the incoming job's call stack. There's no simple way to do that when using such a simple job execution method.

8.6.4.4 Jobs as Coroutines

One way to overcome this problem would be to change from a thread pool based job system to one based on *coroutines*. Recall from Section 4.4.8 that a coroutine possesses one important quality that ordinary threads do not: The ability to *yield* to another coroutine part-way through its execution, and to continue from where it left off at some later time when another coroutine yields control back to it. Coroutines can yield to one another like this because the implementation actually does swap the callstacks of the outgoing and incoming coroutines within the thread in which the coroutines are running. So unlike a purely thread-based job, a coroutine-based job could effectively “go to sleep” and allow other jobs to run while it waits for an operation such as a ray cast to complete.

8.6.4.5 Jobs as Fibers

Another way to allow jobs to sleep and yield to one another is to implement them with *fibers* instead of threads. Recall from Section 4.4.7 that the key difference between fibers and threads is that context switches between fibers are always *cooperative*, never *preemptive*. A fiber-based system begins by converting one of its threads into a fiber. The thread will continue to run that fiber until it explicitly calls `SwitchToFiber()` to explicitly *yield* control to another fiber. Just as with coroutines, the entire call stack of a fiber is saved off whenever a context switch to another fiber occurs. Fibers can even migrate from one thread to another. This makes them well suited for use in implementing a job system. Naughty Dog's job system is based on fibers.

8.6.4.6 Job Counters

If we implement our job system with coroutines or fibers, we have the ability to put a job to sleep (saving off its execution context), and to wake it back up at a future time (thus restoring its execution context). This in turn permits us to implement a *join* function for our job system—a function that causes the calling job to go to sleep, waiting until one or more other jobs have completed executing. Such a function would be roughly equivalent to `pthread_join()` in the POSIX thread library or `WaitForSingleObject()` under Windows.

One way to implement this would be to associate a *handle* with each job, just as threads have handles in most thread libraries. Waiting for a job would then amount to calling a `job::Join()` function of some kind, passing the handle of the job for which you wish to wait.

One downside of the handle-based approach is that it doesn't scale well to waiting for large numbers of jobs. Also, to wait for an individual job to complete, we'd need to poll periodically to check the status of all jobs in the system. Such polling would waste valuable CPU cycles. For these reasons, the job system API presented above introduces the concept of a *counter*, which acts a bit like a semaphore, only in reverse. Whenever a job is kicked, it can optionally be associated with a counter (provided to it via the `job::Declaration`). The act of kicking the job increments the counter, and when the job terminates the counter is decremented. Waiting for a batch of jobs, then, involves simply kicking them all off with the same counter, and then waiting until that counter reaches zero (which indicates that all jobs have completed their work.) Waiting until a counter reaches zero is much more efficient than polling the individual jobs, because the check can be made at the moment the counter is decremented. As such, a counter based system can be a performance win. Counters like this are used in the Naughty Dog job system.

8.6.4.7 Job Synchronization Primitives

Any concurrent program requires a mechanism for performing *atomic operations*, and a job system is no different. Just as a threading library provides a set of *thread synchronization primitives* such as mutexes, condition variables and semaphores, so too must a job system provide a collection of *job synchronization primitives*.

The implementation of job synchronization primitives varies somewhat depending on how the job system is actually implemented. But they're usually *not* simply wrappers around the kernel's thread synchronization primitives. To see why, consider what an OS mutex does: It puts a thread to sleep whenever the lock it's trying to acquire is already being held by another thread. If we were to implement our job system as a thread pool, then waiting on a mutex within a job would put the *entire worker thread* to sleep, not just the one job that wants to wait for the lock. Clearly this would pose a serious problem, because no jobs would be able to run on that thread's core until the thread wakes back up. Such a system would very likely be fraught with deadlock issues.

To overcome this problem, jobs could use *spin locks* instead of OS mutexes. This approach works well as long as there's not very much lock contention between threads, because in that case no job will ever busy-wait for every lock trying to obtain a lock. Naughty Dog's job system uses spin locks for most of its locking needs.

Sometimes, however, a job may experience a high-contention situation. A well-designed job system can handle this via a custom "mutex" mechanism

that can put a job to sleep while it waits for a resource to become available. Such a mutex might start by busy-waiting when a lock can't be obtained. If after a brief timeout the lock is still unavailable, the mutex could yield the coroutine or fiber to another waiting job, thereby putting the waiting job to sleep. Just as the kernel keeps track of all sleeping threads that are waiting on a mutex, our job system would need to keep track of all sleeping jobs so that it can wake them up when their mutexes free up.

8.6.4.8 Job Visualization and Profiling Tools

Once you start using a job system, the graph of running jobs and their dependencies becomes large and complex very quickly. It's a good idea to provide visualization and profiling tools with any job system.

For example, the Naughty Dog job system offers a visualization view like the one shown in Figure 8.7. In this display, you can see each of the seven cores (plus the GPU) listed vertically along the left-hand edge. Time progresses from left to right, and each logical frame is demarked with vertical markers. Along each core's timeline, thin blocks represent the various jobs that ran during the given frame. Beneath each job are additional rows of thin rectangles—these represent the call stack of the job (which functions it called, and how long each one took to run).



Figure 8.7. The job system used in *Uncharted: The Lost Legacy* (© 2017™ SIE). Created and developed by Naughty Dog, PlayStation 4) and Naughty Dog's other PS4 games offers a visualization tool that shows which jobs ran on each core over the course of a given frame. Time increases from left to right. Jobs are represented by thin boxes along the timeline of each core. The functions called by each job are shown beneath it as additional thin boxes.

Jobs are color coded by their function, so users can quickly zero in on a particular job. For example, let's say we're looking for a ray cast that's taking a particularly long time. If ray cast jobs are colored red, we can visually scan the display for all the red jobs that are wider than we'd like. Clicking on a job causes all other jobs that are not of the same type to be drawn in grey, allowing all jobs of the selected job's type to be easily seen. Also, when a job is clicked, thin lines connect it to the jobs it kicked, and to the job that kicked it. Hovering your mouse over a job, or over one of the functions in its call stack beneath it, pops up some text with the name of the job or function, and its execution time in milliseconds.

One other very useful job system feature is what I'll call a *profile trap*. Let's say an area of the game is running well at 30 FPS most of the time, but once in a while it dips to 24 FPS. We can set a trap for any frame taking longer than, say, 35 ms. Then we play the game normally. As soon as a frame taking longer than 35 ms is detected, the game is automatically paused by the trap system, and the profile display is put up on-screen. It's then possible to analyze the jobs that ran that frame to find the culprit(s) causing the slowdown.

8.6.4.9 The Naughty Dog Job System

The job system used by Naughty Dog on *The Last of Us: Remastered*, *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy* largely follows the design of the hypothetical job systems we've discussed thus far. It is based on fibers (rather than thread pools or coroutines). It makes use of spin locks and also provides a special job mutex that can put a job to sleep while it waits for a lock. It uses counters rather than job handles to implement a *join* operation.

Let's have a look at how the fiber-based job system used in the Naughty Dog engine works. When the system first boots up, the main thread converts itself to a fiber in order to enable fibers within the process as a whole. Next, job worker threads are spawned, one for each of the seven cores available to developers on the PS4. These threads are each locked to one core via their CPU affinity settings, so we can think of these worker threads and their cores as being roughly synonymous (although in reality, other higher-priority threads do sometimes interrupt the worker threads for very brief periods during the frame). Fiber creation is slow on the PS4, so a pool of fibers is pre-spawned, along with memory blocks to serve as each fiber's call stack.

When jobs are kicked, their declarations are placed onto a queue. As cores/worker threads become free (as jobs terminate), new jobs are pulled from this queue and executed. A running job can also add more jobs to the job queue.

To execute a job, an unused fiber is pulled from the fiber pool, and the

worker thread performs a `SwitchToFiber()` to start the job running. When a job returns from its entry point function or otherwise self-terminates, the job's final act is to perform a `SwitchToFiber()` back to the job system itself. It then selects another job from the queue, and the process repeats ad infinitum.

When a job waits on a counter, the job is put to sleep and its fiber (execution context) is placed on a wait list, along with the counter it is waiting for. When this counter hits zero, the job is woken back up so it can continue where it left off. Sleeping and waking jobs is again implemented by calling `SwitchToFiber()` between the job's fiber and the job system's management fibers on each core/worker thread.

For an excellent in-depth discussion of how the Naughty Dog job system was built, and why it was built that way, check out Christian Gyrling's excellent GDC 2015 talk entitled, "Parallelizing the Naughty Dog Engine," which is available at <https://bit.ly/2H6v0J4>. Christian's slides are available at <https://bit.ly/2ETr5x9>.

Human Interface Devices

Games are interactive computer simulations, so the human player(s) need some way of providing inputs to the game. All sorts of *human interface devices* (HID) exist for gaming, including joysticks, joypads, keyboards and mice, track balls, the Wii remote and specialized input devices like steering wheels, fishing rods, dance pads and even electric guitars. In this chapter, we'll investigate how game engines typically read, process and utilize the inputs from human interface devices. We'll also have a look at how outputs from these devices provide feedback to the human player.

9.1 Types of Human Interface Devices

A wide range of human interface devices are available for gaming purposes. Consoles like the Xbox 360 and PS3 come equipped with joypad controllers, as shown in Figures 9.1 and 9.2. Nintendo's Wii console is well known for its unique and innovative Wii Remote controller (commonly referred to as the "Wiimote"), shown in Figure 9.3. And with the Wii U, Nintendo has created an innovative mix between a controller and a semi-mobile gaming device (Figure 9.4). PC games are generally either controlled via a keyboard and mouse or via a joypad. (Microsoft designed the Xbox 360 joypad so that it can be used both on the Xbox 360 and on Windows/DirectX PC platforms.) As shown in Figure 9.5, arcade machines have one or more built-in controllers, such as a joystick and various buttons, or a track ball, a steering wheel, etc. An arcade machine's input device is usually somewhat customized to the game in



Figure 9.1. Standard joypads for the Xbox 360 and PlayStation 3 consoles.



Figure 9.2. The DualShock 4 joypad for the PlayStation 4.



Figure 9.3. The innovative Wii Remote for the Nintendo Wii.



Figure 9.4. The Wii U controller by Nintendo.

question, although input hardware is often reused among arcade machines produced by the same manufacturer.

On console platforms, specialized input devices and adapters are usually available, in addition to the “standard” input device such as the joypad. For example, guitar and drum devices are available for the *Guitar Hero* series of games, steering wheels can be purchased for driving games, and games like *Dance Dance Revolution* use a special dance pad device. Some of these devices



Figure 9.5. Buttons and joysticks for the arcade game *Mortal Kombat II* by Midway.



Figure 9.6. Many specialized input devices are available for use with consoles.

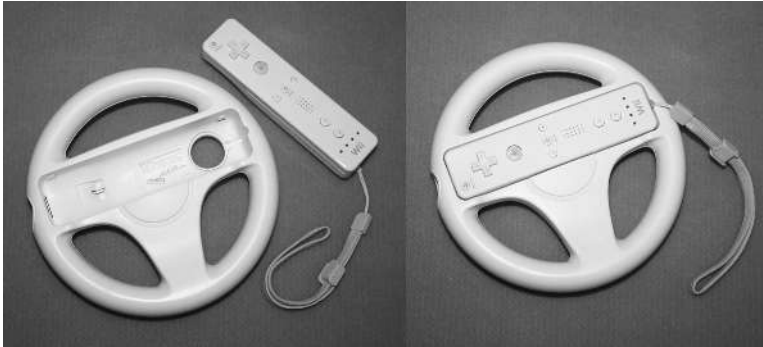


Figure 9.7. Steering wheel adapter for the Nintendo Wii.

are shown in Figure 9.6.

The Nintendo Wiimote is one of the most flexible input devices on the market today. As such, it is often adapted to new purposes, rather than replaced with an entirely new device. For example, *Mario Kart Wii* comes with a plastic steering wheel adapter into which the Wiimote can be inserted (see Figure 9.7).

9.2 Interfacing with a HID

All human interface devices provide input to the game software, and some also allow the software to provide feedback to the human player via various kinds of outputs as well. Game software reads and writes HID inputs and outputs in various ways, depending on the specific design of the device in question.

9.2.1 Polling

Some simple devices, like game pads and old-school joysticks, are read by *polling* the hardware periodically (usually once per iteration of the main game loop). This means explicitly querying the state of the device, either by read-

ing hardware registers directly, reading a memory-mapped I/O port, or via a higher-level software interface (which, in turn, reads the appropriate registers or memory-mapped I/O ports). Likewise, outputs might be sent to the HID by writing to special registers or memory-mapped I/O addresses, or via a higher-level API that does our dirty work for us.

Microsoft's XInput API, for use with Xbox 360 game pads on both the Xbox 360 and Windows PC platforms, is a good example of a simple polling mechanism. Every frame, the game calls the function `XInputGetState()`. This function communicates with the hardware and/or drivers, reads the data in the appropriate way and packages it all up for convenient use by the software. It returns a pointer to an `XINPUT_STATE` struct, which in turn contains an embedded instance of a struct called `XINPUT_GAMEPAD`. This struct contains the current states of all of the controls (buttons, thumb sticks and triggers) on the device.

9.2.2 Interrupts

Some HIDs only send data to the game engine when the state of the controller changes in some way. For example, a mouse spends a lot of its time just sitting still on the mouse pad. There's no reason to send a continuous stream of data between the mouse and the computer when the mouse isn't moving—we need only transmit information when it moves, or a button is pressed or released.

This kind of device usually communicates with the host computer via *hardware interrupts*. An interrupt is an electronic signal generated by the hardware, which causes the CPU to temporarily suspend execution of the main program and run a small chunk of code called an *interrupt service routine* (ISR). Interrupts are used for all sorts of things, but in the case of a HID, the ISR code will probably read the state of the device, store it off for later processing, and then relinquish the CPU back to the main program. The game engine can pick up the data the next time it is convenient to do so.

9.2.3 Wireless Devices

The inputs and outputs of a Bluetooth device, like the Wiimote, the Dual-Shock 3 and the Xbox 360 wireless controller, cannot be read and written by simply accessing registers or memory-mapped I/O ports. Instead, the software must “talk” to the device via the Bluetooth protocol. The software can request the HID to send input data (such as the states of its buttons) back to the host, or it can send output data (such as rumble settings or a stream of audio data) to the device. This communication is often handled by a thread separate

from the game engine's main loop, or at least encapsulated behind a relatively simple interface that can be called from the main loop. So from the point of view of the game programmer, the state of a Bluetooth device can be made to look pretty much indistinguishable from a traditional polled device.

9.3 Types of Inputs

Although human interface devices for games vary widely in terms of form factor and layout, most of the inputs they provide fall into one of a small number of categories. We'll investigate each category in depth below.

9.3.1 Digital Buttons

Almost every HID has at least a few *digital buttons*. These are buttons that can only be in one of two states: *pressed* and *not pressed*. Game programmers often refer to a pressed button as being *down* and a non-pressed button as being *up*.

Electrical engineers speak of a circuit containing a switch as being *closed* (meaning electricity is flowing through the circuit) or *open* (no electricity is flowing—the circuit has infinite *resistance*). Whether *closed* corresponds to *pressed* or *not pressed* depends on the hardware. If the switch is *normally open*, then when it is not pressed (up), the circuit is *open*, and when it is pressed (down), the circuit is *closed*. If the switch is *normally closed*, the reverse is true—the act of pressing the button opens the circuit.

In software, the state of a digital button (pressed or not pressed) is usually represented by a single bit. It's common for 0 to represent not pressed (up) and 1 to represent pressed (down). But again, depending on the nature of the circuitry and the decisions made by the programmers who wrote the device driver, the sense of these values might be reversed.

It is quite common for the states of all of the buttons on a device to be packed into a single unsigned integer value. For example, in Microsoft's XInput API, the state of the Xbox 360 joystick is returned in a struct called `XINPUT_GAMEPAD`, shown below.

```
typedef struct _XINPUT_GAMEPAD
{
    WORD    wButtons;
    BYTE    bLeftTrigger;
    BYTE    bRightTrigger;
    SHORT   sThumbLX;
    SHORT   sThumbLY;
    SHORT   sThumbRX;
    SHORT   sThumbRY;
```

```
} XINPUT_GAMEPAD;
```

This struct contains a 16-bit unsigned integer (WORD) variable named `wButtons` that holds the state of all buttons. The following masks define which physical button corresponds to each bit in the word. (Note that bits 10 and 11 are unused.)

```
#define XINPUT_GAMEPAD_DPAD_UP          0x0001 // bit 0
#define XINPUT_GAMEPAD_DPAD_DOWN        0x0002 // bit 1
#define XINPUT_GAMEPAD_DPAD_LEFT        0x0004 // bit 2
#define XINPUT_GAMEPAD_DPAD_RIGHT       0x0008 // bit 3
#define XINPUT_GAMEPAD_START            0x0010 // bit 4
#define XINPUT_GAMEPAD_BACK             0x0020 // bit 5
#define XINPUT_GAMEPAD_LEFT_THUMB       0x0040 // bit 6
#define XINPUT_GAMEPAD_RIGHT_THUMB      0x0080 // bit 7
#define XINPUT_GAMEPAD_LEFT_SHOULDER    0x0100 // bit 8
#define XINPUT_GAMEPAD_RIGHT_SHOULDER   0x0200 // bit 9
#define XINPUT_GAMEPAD_A                0x1000 // bit 12
#define XINPUT_GAMEPAD_B                0x2000 // bit 13
#define XINPUT_GAMEPAD_X                0x4000 // bit 14
#define XINPUT_GAMEPAD_Y                0x8000 // bit 15
```

An individual button's state can be read by masking the `wButtons` word with the appropriate bitmask via C/C++'s bitwise AND operator (`&`) and then checking if the result is nonzero. For example, to determine if the A button is pressed (down), we would write:

```
bool IsButtonDown(const XINPUT_GAMEPAD& pad)
{
    // Mask off all bits but bit 12 (the A button).
    return ((pad.wButtons & XINPUT_GAMEPAD_A) != 0);
}
```

9.3.2 Analog Axes and Buttons

An *analog input* is one that can take on a range of values (rather than just 0 or 1). These kinds of inputs are often used to represent the degree to which a trigger is pressed, or the two-dimensional position of a joystick (which is represented using two analog inputs, one for the *x*-axis and one for the *y*-axis, as shown in Figure 9.8). Because of this common usage, analog inputs are sometimes called *analog axes*, or just *axes*.

On some devices, certain buttons are analog as well, meaning that the game can actually detect how hard the player is pressing on them. However, the signals produced by analog buttons are usually too noisy to be particularly usable. Games that use analog button inputs effectively are rare. One good

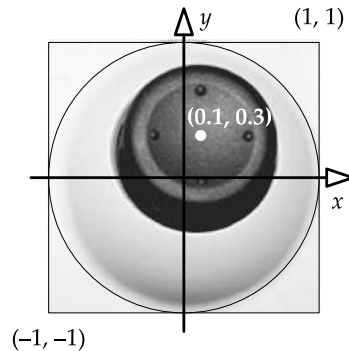


Figure 9.8. Two analog inputs can be used to represent the x and y deflection of a joystick.

example is *Metal Gear Solid 2* on the PS2. It uses pressure-sensitive (analog) button data in aim mode to tell the difference between releasing the X button quickly (which fires the weapon) and releasing it slowly (which aborts the shot)—a useful feature in a stealth game, where you don’t want to alert the enemies unless you have to!

Strictly speaking, analog inputs are not really analog by the time they make it to the game engine. An analog input signal is usually *digitized*, meaning it is quantized and represented using an integer in software. For example, an analog input might range from $-32,768$ to $32,767$ if represented by a 16-bit signed integer. Sometimes analog inputs are converted to floating point—the values might range from -1 to 1 , for instance. But as we know from Section 3.3.1.3, floating-point numbers are really just quantized digital values as well.

Reviewing the definition of `XINPUT_GAMEPAD` (repeated below), we can see that Microsoft chose to represent the deflections of the left and right thumb sticks on the Xbox 360 gamepad using 16-bit signed integers (`sThumbLX` and `sThumbLY` for the left stick and `sThumbRX` and `sThumbRY` for the right). Hence, these values range from $-32,768$ (left or down) to $32,767$ (right or up). However, to represent the positions of the left and right shoulder triggers, Microsoft chose to use eight-bit unsigned integers (`bLeftTrigger` and `bRightTrigger`, respectively). These input values range from 0 (not pressed) to 255 (fully pressed). Different game machines use different digital representations for their analog axes.

```
typedef struct _XINPUT_GAMEPAD
{
    WORD    wButtons;

    // 8-bit unsigned
    BYTE    bLeftTrigger;
```

```

    BYTE  bRightTrigger;

    // 16-bit signed
    SHORT sThumbLX;
    SHORT sThumbLY;
    SHORT sThumbRX;
    SHORT sThumbRY;
} XINPUT_GAMEPAD;

```

9.3.3 Relative Axes

The position of an analog button, trigger, joystick or thumb stick is *absolute*, meaning that there is a clear understanding of where zero lies. However, the inputs of some devices are *relative*. For these devices, there is no clear location at which the input value should be zero. Instead, a zero input indicates that the position of the device has not changed, while nonzero values represent a delta from the last time the input value was read. Examples include mice, mouse wheels and track balls.

9.3.4 Accelerometers

The PlayStation's DualShock joypads and the Nintendo Wiimote all contain acceleration sensors (accelerometers). These devices can detect acceleration along the three principal axes (x , y and z), as shown in Figure 9.9. These are *relative* analog inputs, much like a mouse's two-dimensional axes. When the controller is not accelerating these inputs are zero, but when the controller is accelerating, they measure the acceleration up to ± 3 g along each axis, quantized into three signed eight-bit integers, one for each of x , y and z .

9.3.5 3D Orientation with the Wiimote or DualShock

Some Wii and PS3 games make use of the three accelerometers in the Wiimote or DualShock joystick to estimate the orientation of the controller in the player's hand. For example, in *Super Mario Galaxy*, Mario hops onto a large

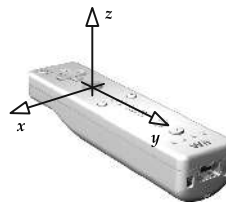


Figure 9.9. Accelerometer axes for the Wiimote.

ball and rolls it around with his feet. To control Mario in this mode, the Wiimote is held with the IR sensor facing the ceiling. Tilting the Wiimote left, right, forward or back causes the ball to accelerate in the corresponding direction.

A trio of accelerometers can be used to detect the orientation of the Wiimote or DualShock joystick, because of the fact that we are playing these games on the surface of the Earth where there is a constant downward acceleration due to gravity of 1 g ($\approx 9.8 \text{ m/s}^2$). If the controller is held perfectly level, with the IR sensor pointing toward your TV set, the vertical (z) acceleration should be approximately -1 g .

If the controller is held upright, with the IR sensor pointing toward the ceiling, we would expect to see a 0 g acceleration on the z sensor, and $+1 \text{ g}$ on the y sensor (because it is now experiencing the full gravitational effect). Holding the Wiimote at a 45-degree angle should produce roughly $\sin(45^\circ) = \cos(45^\circ) = 0.707 \text{ g}$ on both the y and z inputs. Once we've calibrated the accelerometer inputs to find the zero points along each axis, we can calculate pitch, yaw and roll easily, using inverse sine and cosine operations.

Two caveats here: First, if the person holding the Wiimote is not holding it still, the accelerometer inputs will include this acceleration in their values, invalidating our math. Second, the z-axis of the accelerometer has been calibrated to account for gravity, but the other two axes have not. This means that the z-axis has less precision available for detecting orientation. Many Wii games request that the user hold the Wiimote in a nonstandard orientation, such as with the buttons facing the player's chest, or with the IR sensor pointing toward the ceiling. This maximizes the precision of the orientation reading by placing the x- or y-accelerator axis in line with gravity, instead of the gravity-calibrated z-axis. For more information on this topic, see <http://druid.caughq.org/presentations/turbo/Wiimote-Hacking.pdf>.

9.3.6 Cameras

The Wiimote has a unique feature not found on any other standard console HID—an infrared (IR) sensor. This sensor is essentially a low-resolution camera that records a two-dimension infrared image of whatever the Wiimote is pointed at. The Wii comes with a “sensor bar” that sits on top of your television set and contains two infrared light emitting diodes (LEDs). In the image recorded by the IR camera, these LEDs appear as two bright dots on an otherwise dark background. Image processing software in the Wiimote analyzes the image and isolates the location and size of the two dots. (Actually, it can detect and transmit the locations and sizes of up to four dots.) This position

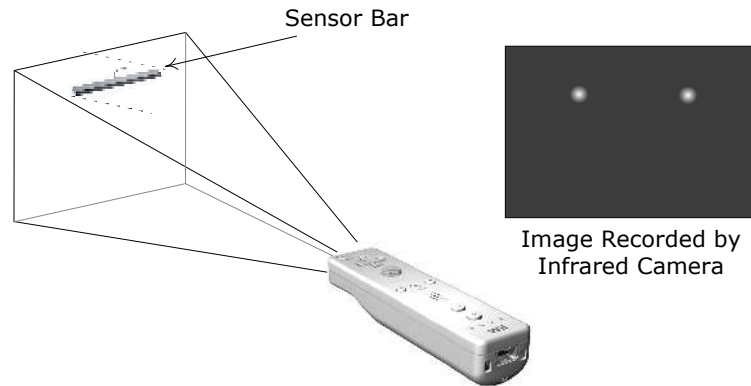


Figure 9.10. The Wii sensor bar houses two infrared LEDs, which produce two bright spots on the image recorded by the Wiimote's IR camera.

and size information can be read by the console via a Bluetooth wireless connection.

The position and orientation of the line segment formed by the two dots can be used to determine the pitch, yaw and roll of the Wiimote (as long as it is being pointed toward the sensor bar). By looking at the separation between the dots, software can also determine how close or far away the Wiimote is from the TV. Some software also makes use of the sizes of the dots. This is illustrated in Figure 9.10.



Figure 9.11. Sony's PlayStation Eye for the PS3.

Another popular camera device is Sony's PlayStation Eye for the PS3, shown in Figure 9.11. This device is basically a high-quality color camera, which can be used for a wide range of applications. It can be used for simple video conferencing, like any web cam. It could also conceivably be used much like the Wiimote's IR camera, for position, orientation and depth sensing. The gamut of possibilities for these kinds of advanced input devices has only begun to be tapped by the gaming community.

With the PlayStation 4, Sony has improved the Eye and re-dubbed it the PlayStation Camera. When combined with the PlayStation Move controller (see Figure 9.12) or the DualShock 4 controller, the PlayStation can detect gestures in basically the same way as Microsoft's innovative Kinect system can (Figure 9.13).



Figure 9.12. Sony's PlayStation Camera, PlayStation Move controller and DualShock 4 joypad for the PS4.



Figure 9.13. The Microsoft Kinect for Xbox 360 (top) and Xbox One (bottom).

9.4 Types of Outputs

Human interface devices are primarily used to transmit inputs from the player to the game software. However, some HIDs can also provide feedback to the human player via various kinds of outputs.

9.4.1 Rumble

Game pads like the PlayStation's DualShock line of controllers and the Xbox and Xbox 360 controllers have a *rumble* feature. This allows the controller to vibrate in the player's hands, simulating the turbulence or impacts that the character in the game world might be experiencing. Vibrations are usually

produced by one or more motors, each of which rotates a slightly unbalanced weight at various speeds. The game can turn these motors on and off, and control their speeds to produce different tactile effects in the player's hands.

9.4.2 Force-Feedback

Force-feedback is a technique in which an actuator on the HID is driven by a motor in order to slightly resist the motion the human operator is trying to impart to it. It is common in arcade driving games, where the steering wheel resists the player's attempt to turn it, simulating difficult driving conditions or tight turns. As with rumble, the game software can typically turn the motor(s) on and off, and can also control the strength and direction of the forces applied to the actuator.

9.4.3 Audio

Audio is usually a stand-alone engine system. However, some HIDs provide outputs that can be utilized by the audio system. For example, the Wiimote contains a small, low-quality speaker. The Xbox 360, Xbox One and DualShock 4 controllers have a headphone jack and can be used just like any USB audio device for both output (speakers) and input (microphone). One common use of USB headsets is for multiplayer games, in which human players can communicate with one another via a voice over IP (VoIP) connection.

9.4.4 Other Inputs and Outputs

Human interface devices may of course support many other kinds of inputs and outputs. On some older consoles like the Sega Dreamcast, the memory card slots were located on the game pad. The Xbox 360 game pad, the Sixaxis and DualShock 3, and the Wiimote all have four LEDs which can be illuminated by game software if desired. The color of the light bar on the front of the DualShock 4 controller can be controlled by game software. And of course specialized devices like musical instruments, dance pads, etc. have their own particular kinds of inputs and outputs.

Innovation is actively taking place in the field of human interfaces. Some of the most interesting areas today are gestural interfaces and thought-controlled devices. We can certainly expect more innovation from console and HID manufacturers in years to come.

9.5 Game Engine HID Systems

Most game engines don't use "raw" HID inputs directly. The data is usually massaged in various ways to ensure that the inputs coming from the HID

translate into smooth, pleasing, intuitive behaviors in-game. In addition, most engines introduce at least one additional level of indirection between the HID and the game in order to abstract HID inputs in various ways. For example, a button-mapping table might be used to translate raw button inputs into logical game actions, so that human players can reassign the buttons' functions as they see fit. In this section, we'll outline the typical requirements of a game engine HID system and then explore each one in some depth.

9.5.1 Typical Requirements

A game engine's HID system usually provides some or all of the following features:

- dead zones,
- analog signal filtering,
- event detection (e.g., button up, button down),
- detection of button *sequences* and multibutton combinations (known as *chords*),
- gesture detection,
- management of multiple HIDs for multiple players,
- multiplatform HID support,
- controller input remapping,
- context-sensitive inputs, and
- the ability to temporarily disable certain inputs.

9.5.2 Dead Zone

A joystick, thumb stick, shoulder trigger, or any other analog axis produces input values that range between a predefined minimum and maximum value, which we'll call $I_{\min}$ and $I_{\max}$. When the control is not being touched, we would expect it to produce a steady and clear "undisturbed" value, which we'll call I_0 . The undisturbed value is usually numerically equal to zero, and it either lies halfway between $I_{\min}$ and $I_{\max}$ for a centered, two-way control like a joystick axis, or it coincides with $I_{\min}$ for a one-way control like a trigger.

Unfortunately, because HIDs are analog devices by nature, the voltage produced by the device is noisy, and the actual inputs we observe may fluctuate slightly around I_0 . The most common solution to this problem is to introduce a small *dead zone* around I_0 . The dead zone might be defined as $[I_0 - \delta, I_0 + \delta]$ for a joy stick, or $[I_0, I_0 + \delta]$ for a trigger. Any input values that are within the dead zone are simply clamped to I_0 . The dead zone must be wide enough to

account for the noisiest inputs generated by an undisturbed control, but small enough not to interfere with the player's sense of the HID's responsiveness.

9.5.3 Analog Signal Filtering

Signal noise is a problem even when the controls are not within their dead zones. This noise can sometimes cause the in-game behaviors controlled by the HID to appear jerky or unnatural. For this reason, many games *filter* the raw inputs coming from the HID. A noise signal is usually of a high frequency relative to the signal produced by the human player. Therefore, one solution is to pass the raw input data through a simple *low-pass filter*, prior to it being used by the game.

A discrete first-order low-pass filter can be implemented by combining the current unfiltered input value with last frame's filtered input. If we denote the sequence of unfiltered inputs by the time-varying function $u(t)$ and the filtered inputs by $f(t)$, where t denotes time, then we can write

$$f(t) = (1 - a)f(t - \Delta t) + au(t), \quad (9.1)$$

where the parameter a is determined by the frame duration Δt and a filtering constant RC (which is just the product of the resistance and the capacitance in a traditional analog RC low-pass filter circuit):

$$a = \frac{\Delta t}{RC + \Delta t}. \quad (9.2)$$

This can be implemented trivially in C or C++ as follows, where it is assumed the calling code will keep track of the last frame's filtered input for use on the subsequent frame. For more information, see http://en.wikipedia.org/wiki/Low-pass_filter.

```
F32 lowPassFilter(F32 unfilteredInput,
                  F32 lastFramesFilteredInput,
                  F32 rc, F32 dt)
{
    F32 a = dt / (rc + dt);

    return (1 - a) * lastFramesFilteredInput
        + a * unfilteredInput;
}
```

Another way to filter HID input data is to calculate a simple moving average. For example, if we wish to average the input data over a 3/30 second (3 frame) interval, we simply store the raw input values in a 3-element circular

buffer. The filtered input value is then the sum of the values in this array at any moment, divided by 3. There are a few minor details to account for when implementing such a filter. For example, we need to properly handle the first two frames of input, during which the 3-element array has not yet been filled with valid data. However, the implementation is not particularly complicated. The code below shows one way to properly implement an N -element moving average.

```
template< typename TYPE, int SIZE >
class MovingAverage
{
    TYPE    m_samples[SIZE];
    TYPE    m_sum;
    U32     m_curSample;
    U32     m_sampleCount;

public:
    MovingAverage() :
        m_sum(static_cast<TYPE>(0)),
        m_curSample(0),
        m_sampleCount(0)
    {
    }

    void addSample(TYPE data)
    {
        if (m_sampleCount == SIZE)
        {
            m_sum -= m_samples[m_curSample];
        }
        else
        {
            m_sampleCount++;
        }

        m_samples[m_curSample] = data;
        m_sum += data;
        m_curSample++;

        if (m_curSample >= SIZE)
        {
            m_curSample = 0;
        }
    }

    F32 getCurrentAverage() const
    {
```

```

        if (m_sampleCount != 0)
        {
            return static_cast<F32>(m_sum)
                / static_cast<F32>(m_sampleCount);
        }
        return 0.0f;
    }
};

```

9.5.4 Detecting Input Events

The low-level HID interface typically provides the game with the current states of the device's various inputs. However, games are often interested in detecting *events*, such as changes in state, rather than just inspecting the current state each frame. The most common HID events are probably button-down (pressed) and button-up (released), but of course we can detect other kinds of events as well.

9.5.4.1 Button Up and Button Down

Let's assume for the moment that our buttons' input bits are 0 when not pressed and 1 when pressed. The easiest way to detect a change in button state is to keep track of the buttons' state bits as observed last frame and compare them to the state bits observed this frame. If they differ, we know an event occurred. The current state of each button tells us whether the event is a button-up or a button-down.

We can use simple bit-wise operators to detect button-down and button-up events. Given a 32-bit word `buttonStates` containing the current state bits of up to 32 buttons, we want to generate two new 32-bit words: one for button-down events, which we'll call `buttonDowns`, and one for button-up events, which we'll call `buttonUps`. In both cases, the bit corresponding to each button will be 0 if the event has not occurred this frame and 1 if it has. To implement this, we also need the last frame's button states, `prevButtonStates`.

The exclusive OR (XOR) operator produces a 0 if its two inputs are identical and a 1 if they differ. So if we apply the XOR operator to the previous and current button state words, we'll get 1's only for buttons whose states have changed between the last frame and this frame. To determine whether the event is a button-up or a button-down, we need to look at the current state of each button. Any button whose state has changed that is currently down generates a button-down event, and vice versa for button-up events. The following code applies these ideas in order to generate our two button event words:

```

class ButtonState
{
    U32 m_buttonStates;          // current frame's button
                                // states
    U32 m_prevButtonStates;      // previous frame's states
    U32 m_buttonDowns;          // 1 = button pressed this
                                // frame
    U32 m_buttonUps;             // 1 = button released this
                                // frame

    void DetectButtonUpDownEvents()
    {
        // Assuming that m_buttonStates and
        // m_prevButtonStates are valid, generate
        // m_buttonDowns and m_buttonUps.

        // First determine which bits have changed via
        // XOR.
        U32 buttonChanges = m_buttonStates
                            ^ m_prevButtonStates;

        // Now use AND to mask off only the bits that
        // are DOWN.
        m_buttonDowns = buttonChanges & m_buttonStates;

        // Use AND-NOT to mask off only the bits that
        // are UP.
        m_buttonUps = buttonChanges & (~m_buttonStates);
    }

    // ...
};

```

9.5.4.2 Chords

A *chord* is a group of buttons that, when pressed at the same time, produce a unique behavior in the game. Here are a few examples:

- *Super Mario Galaxy*'s start-up screen requires you to press the A and B buttons on the Wiimote together in order to start a new game.
- Pressing the 1 and 2 buttons on the Wiimote at the same time puts it into Bluetooth discovery mode (no matter what game you're playing).

- The “grapple” move in many fighting games is triggered by a two-button combination.
- For development purposes, holding down both the left and right triggers on the DualShock 3 in *Uncharted* allows the player character to fly anywhere in the game world, with collisions turned off. (Sorry, this doesn’t work in the shipping game!) Many games have a cheat like this to make development easier. (It may or may not be triggered by a chord, of course.) It is called *no-clip mode* in the Quake engine, because the character’s collision volume is not *clipped* to the valid playable area of the world. Other engines use different terminology.

Detecting chords is quite simple in principle: We merely watch the states of two or more buttons and only perform the requested operation when *all* of them are down.

There are some subtleties to account for, however. For one thing, if the chord includes a button or buttons that have other purposes in the game, we must take care not to perform *both* the actions of the individual buttons and the action of chord when it is pressed. This is usually done by including a check that the other buttons in the chord are *not* down when detecting the individual button presses.

Another fly in the ointment is that humans aren’t perfect, and they often press one or more of the buttons in the chord slightly earlier than the rest. So our chord-detection code must be robust to the possibility that we’ll observe one or more individual buttons on frame i and the rest of the chord on frame $i + 1$ (or even multiple frames later). There are a number of ways to handle this:

- You can design your button inputs such that a chord always does the actions of the individual buttons *plus* some additional action. For example, if pressing L1 fires the primary weapon and L2 lob a grenade, perhaps the L1 + L2 chord could fire the primary weapon, lob a grenade, *and* send out an energy wave that doubles the damage done by these weapons. That way, whether or not the individual buttons are detected before the chord or not, the behavior will be identical from the point of view of the player.
- You can introduce a delay between when an individual button-down event is seen and when it “counts” as a valid game event. During the delay period (say 2 or 3 frames), if a chord is detected, then it takes precedence over the individual button-down events. This gives the human player some leeway in performing the chord.
- You can detect the chord when the buttons are pressed, but wait to trigger the effect until the buttons are released again.

- You can begin the single-button move immediately and allow it to be preempted by the chord move.

9.5.4.3 Sequences and Gesture Detection

The idea of introducing a delay between when a button actually goes down and when it really “counts” as down is a special case of *gesture detection*. A gesture is a sequence of actions performed via a HID by the human player over a period of time. For example, in a fighting game or brawler, we might want to detect a *sequence* of button presses, such as A-B-A. We can extend this idea to non-button inputs as well. For example, A-B-A-Left-Right-Left, where the latter three actions are side-to-side motions of one of the thumb sticks on the game pad. Usually a sequence or gesture is only considered to be valid if it is performed within some maximum time frame. So a rapid A-B-A within a quarter of a second might “count,” but a slow A-B-A performed over a second or two might not.

Gesture detection is generally implemented by keeping a brief history of the HID actions performed by the player. When the first component of the gesture is detected, it is stored in the history buffer, along with a time stamp indicating when it occurred. As each subsequent component is detected, the time between it and the previous component is checked. If it is within the allowable time window, it too is added to the history buffer. If the entire sequence is completed within the allotted time (i.e., the history buffer is filled), an event is generated telling the rest of the game engine that the gesture has occurred. However, if any non-valid intervening inputs are detected, or if any component of the gesture occurs outside of its valid time window, the entire history buffer is reset and the player must start the gesture over again.

Let’s look at three concrete examples, so we can really understand how this works.

Rapid Button Tapping

Many games require the user to tap a button rapidly in order to perform an action. The frequency of the button presses may or may not translate into some quantity in the game, such as the speed with which the player character runs or performs some other action. The frequency is usually also used to define the validity of the gesture—if the frequency drops below some minimum value, the gesture is no longer considered valid.

We can detect the frequency of a button press by simply keeping track of the last time we saw a button-down event for the button in question. We’ll call this T_{last} . The frequency f is then just the inverse of the time interval between

presses $\Delta T = T_{\text{cur}} - T_{\text{last}}$ and $f = 1/\Delta T$. Every time we detect a new button-down event, we calculate a new frequency f . To implement a minimum valid frequency, we simply check f against the minimum frequency f_{min} (or we can just check ΔT against the maximum period $\Delta T_{\text{max}} = 1/f_{\text{min}}$ directly). If this threshold is satisfied, we update the value of T_{last} , and the gesture is considered to be on-going. If the threshold is not satisfied, we simply don't update T_{last} . The gesture will be considered invalid until a new pair of rapid-enough button-down events occurs. This is illustrated by the following pseudocode:

```
class ButtonTapDetector
{
    U32  m_buttonMask; // which button to observe (bit
                       // mask)
    F32  m_dtMax;      // max allowed time between
                       // presses
    F32  m_tLast;      // last button-down event, in
                       // seconds

public:

    // Construct an object that detects rapid tapping of
    // the given button (identified by an index).
    ButtonTapDetector(U32 buttonId, F32 dtMax) :
        m_buttonMask(1U << buttonId),
        m_dtMax(dtMax),
        m_tLast(CurrentTime() - dtMax) // start out
                                       // invalid
    {
    }

    // Call this at any time to query whether or not
    // the gesture is currently being performed.
    bool IsGestureValid() const
    {
        F32 t = CurrentTime();
        F32 dt = t - m_tLast;
        return (dt < m_dtMax);
    }

    // Call this once per frame.
    void Update()
    {
        if (ButtonsJustWentDown(m_buttonMask))
        {
            m_tLast = CurrentTime();
        }
    }
}
```



```
    }
};
```

In the above code excerpt, we assume that each button is identified by a unique id. The id is really just an index, ranging from 0 to $N - 1$ (where N is the number of buttons on the HID in question). We convert the button id to a bitmask by shifting an unsigned 1 bit to the left by an amount equaling the button's index ($1U \ll \text{buttonId}$). The function `ButtonsJustWentDown()` returns a nonzero value if *any one* of the buttons specified by the given bitmask just went down this frame. Here, we're only checking for a single button-down event, but we can and will use this same function later to check for multiple simultaneous button-down events.

Multibutton Sequence

Let's say we want to detect the sequence A-B-A, performed within at most one second. We can detect this button sequence as follows: We maintain a variable that tracks which button in the sequence we're currently looking for. If we define the sequence with an array of button ids (e.g., `aButtons[3] = {A, B, A}`), then our variable is just an index i into this array. It starts out initialized to the first button in the sequence, $i = 0$. We also maintain a start time for the entire sequence, T_{start} , much as we did in the rapid button-pressing example.

The logic goes like this: Whenever we see a button-down event that matches the button we're currently looking for, we check its time stamp against the start time of the entire sequence, T_{start} . If it occurred within the valid time window, we advance the current button to the next button in the sequence; for the first button in the sequence only ($i = 0$), we also update T_{start} . If we see a button-down event that doesn't match the next button in the sequence, or if the time delta has grown too large, we reset the button index i back to the beginning of the sequence and set T_{start} to some invalid value (such as 0). This is illustrated by the code below.

```
class ButtonSequenceDetector
{
    U32* m_aButtonIds; // sequence of buttons to watch for
    U32 m_buttonCount; // number of buttons in sequence
    F32 m_dtMax;       // max time for entire sequence
    U32 m_iButton;     // next button to watch for in seq.
    F32 m_tStart;      // start time of sequence, in
                      // seconds
}
```

```

public:

    // Construct an object that detects the given button
    // sequence. When the sequence is successfully
    // detected, the given event is broadcast so that the
    // rest of the game can respond in an appropriate way.

    ButtonSequenceDetector(U32* aButtonIds,
                           U32 buttonCount,
                           F32 dtMax,
                           EventId eventIdToSend) :
        m_aButtonIds(aButtonIds),
        m_buttonCount(buttonCount),
        m_dtMax(dtMax),
        m_eventId(eventIdToSend), // event to send when
                                   // complete
        m_iButton(0),             // start of sequence
        m_tStart(0)               // initial value
                                   // irrelevant
    {
    }

    // Call this once per frame.
    void Update()
    {
        ASSERT(m_iButton < m_buttonCount);

        // Determine which button we're expecting next, as
        // a bitmask (shift a 1 up to the correct bit
        // index).

        U32 buttonMask = (1U << m_aButtonId[m_iButton]);

        // If any button OTHER than the expected button
        // just went down, invalidate the sequence. (Use
        // the bitwise NOT operator to check for all other
        // buttons.)

        if (ButtonsJustWentDown(~buttonMask))
        {
            m_iButton = 0; // reset
        }

        // Otherwise, if the expected button just went
        // down, check dt and update our state appropriately.

        else if (ButtonsJustWentDown(buttonMask))
        {

```

```

    if (m_iButton == 0)
    {
        // This is the first button in the
        // sequence.
        m_tStart = CurrentTime();
        m_iButton++; // advance to next button
    }
    else
    {
        F32 dt = CurrentTime() - m_tStart;

        if (dt < m_dtMax)
        {
            // Sequence is still valid.

            m_iButton++; // advance to next button

            // Is the sequence complete?
            if (m_iButton == m_buttonCount)
            {
                BroadcastEvent(m_eventId);
                m_iButton = 0; // reset
            }
        }
        else
        {
            // Sorry, not fast enough.
            m_iButton = 0; // reset
        }
    }
}
};

```

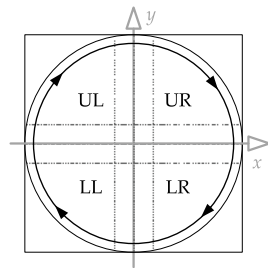


Figure 9.14. Detecting circular rotations of the stick by dividing the 2D range of stick inputs into quadrants.

Thumb Stick Rotation

As an example of a more complex gesture, let's see how we might detect when the player is rotating the left thumb stick in a clockwise circle. We can detect this quite easily by dividing the two-dimensional range of possible stick positions into quadrants, as shown in Figure 9.14. In a clockwise rotation, the stick passes through the upper-left quadrant, then the upper-right, then the lower-right and finally the lower-left. We can treat each of these cases like a button press and detect a full rotation with a slightly modified version of the sequence detection code shown above. We'll leave this one as an exercise for the reader. Try it!

9.5.5 Managing Multiple HID's for Multiple Players

Most game machines allow two or more HID's to be attached for multiplayer games. The engine must keep track of which devices are currently attached and route each one's inputs to the appropriate player in the game. This implies that we need some way of mapping controllers to players. This might be as simple as a one-to-one mapping between controller index and player index, or it might be something more sophisticated, such as assigning controllers to players at the time the user hits the Start button.

Even in a single-player game with only one HID, the engine needs to be robust to various exceptional conditions, such as the controller being accidentally unplugged or running out of batteries. When a controller's connection is lost, most games pause gameplay, display a message and wait for the controller to be reconnected. Some multiplayer games suspend or temporarily remove the avatar corresponding to a removed controller, but allow the other players to continue playing the game; the removed/suspended avatar might reactivate when the controller is reconnected.

On systems with battery-operated HID's, the game or the operating system is responsible for detecting low-battery conditions. In response, the player is usually warned in some way, for example via an unobtrusive on-screen message and/or a sound effect.

9.5.6 Cross-Platform HID Systems

Many game engines are cross-platform. One way to handle HID inputs and outputs in such an engine would be to sprinkle conditional compilation directives all over the code, wherever interactions with the HID take place, as shown below. This is clearly not an ideal solution, but it does work.

```
#if TARGET_XBOX360
if (ButtonsJustWentDown(XB360_BUTTONMASK_A))
```

```

#elif TARGET_PS3
if (ButtonsJustWentDown(PS3_BUTTONMASK_TRIANGLE))
#elif TARGET_WII
if (ButtonsJustWentDown(WII_BUTTONMASK_A))
#endif
{
    // do something...
}

```

A better solution is to provide some kind of hardware abstraction layer, thereby insulating the game code from hardware-specific details.

If we're lucky, we can abstract most of the differences between the HIDs on the different platforms by a judicious choice of abstract button and axis ids. For example, if our game is to ship on Xbox 360 and PS3, the layout of the controls (buttons, axes and triggers) on these two joypads are almost identical. The controls have different ids on each platform, but we can come up with generic control ids that cover both types of joystick quite easily. For example:

```

enum AbstractControlIndex
{
    // Start and back buttons
    AINDEX_START,           // Xbox 360 Start, PS3 Start
    AINDEX_BACK_SELECT,    // Xbox 360 Back, PS3 Select

    // Left D-pad
    AINDEX_LPAD_DOWN,
    AINDEX_LPAD_UP,
    AINDEX_LPAD_LEFT,
    AINDEX_LPAD_RIGHT,

    // Right "pad" of four buttons
    AINDEX_RPAD_DOWN,      // Xbox 360 A, PS3 X
    AINDEX_RPAD_UP,       // Xbox 360 Y, PS3 Triangle
    AINDEX_RPAD_LEFT,     // Xbox 360 X, PS3 Square
    AINDEX_RPAD_RIGHT,    // Xbox 360 B, PS3 Circle

    // Left and right thumb stick buttons
    AINDEX_LSTICK_BUTTON,  // Xbox 360 LThumb, PS3 L3,
                          // Xbox white
    AINDEX_RSTICK_BUTTON,  // Xbox 360 RThumb, PS3 R3,
                          // Xbox black

    // Left and right shoulder buttons
    AINDEX_LSHOULDER,      // Xbox 360 L shoulder, PS3 L1
    AINDEX_RSHOULDER,      // Xbox 360 R shoulder, PS3 R1
}

```

```

        // Left thumb stick axes
        AINDEX_LSTICK_X,
        AINDEX_LSTICK_Y,

        // Right thumb stick axes
        AINDEX_RSTICK_X,
        AINDEX_RSTICK_Y,

        // Left and right trigger axes
        AINDEX_LTRIGGER,      // Xbox 360 -Z, PS3 L2
        AINDEX_RTRIGGER,      // Xbox 360 +Z, PS3 R2
    };

```

Our abstraction layer can translate between the raw control ids on the current target hardware into our abstract control indices. For example, whenever we read the state of the buttons into a 32-bit word, we can perform a bit-swizzling operation that rearranges the bits into the proper order to correspond to our abstract indices. Analog inputs can likewise be shuffled around into the proper order.

In performing the mapping between physical and abstract controls, we'll sometimes need to get a bit clever. For example, on the Xbox, the left and right triggers act as a single axis, producing negative values when the left trigger is pressed, zero when neither is trigger is pressed, and positive values when the right trigger is pressed. To match the behavior of the PlayStation's DualShock controller, we might want to separate this axis into two distinct axes on the Xbox, scaling the values appropriately so the range of valid values is the same on all platforms.

This is certainly not the only way to handle HID I/O in a multiplatform engine. We might want to take a more functional approach, for example, by naming our abstract controls according to their function in the game, rather than their physical locations on the joypad. We might introduce higher-level functions that detect abstract gestures, with custom detection code on each platform, or we might just bite the bullet and write platform-specific versions of all of the game code that requires HID I/O. The possibilities are numerous, but virtually all cross-platform game engines insulate the game from hardware details in *some* manner.

9.5.7 Input Remapping

Many games allow the player some degree of choice with regard to the functionality of the various controls on the physical HID. A common option is the sense of the vertical axis of the right thumb stick for camera control in a console game. Some folks like to push forward on the stick to angle the camera

up, while others like an inverted control scheme, where pulling back on the stick angles the camera up (much like an airplane control stick). Other games allow the player to select between two or more predefined button mappings. Some PC games allow the user full control over the functions of individual keys on the keyboard, the mouse buttons and the mouse wheel, plus a choice between various control schemes for the two mouse axes.

To implement this, we turn to a favorite saying of an old professor of mine, Professor Jay Black of the University of Waterloo, “Every problem in computer science can be solved with a level of indirection.” We assign each function in the game a unique id and then provide a simple table, which maps each physical or abstract control index to a logical function in the game. Whenever the game wishes to determine whether a particular logical game function should be activated, it looks up the corresponding abstract or physical control id in the table and then reads the state of that control. To change the mapping, we can either swap out the entire table wholesale, or we can allow the user to edit individual entries in the table.

We’re glossing over a few details here. For one thing, different controls produce different kinds of inputs. Analog axes may produce values ranging from $-32,768$ to $32,767$, or from 0 to 255 , or some other range. The states of all the digital buttons on a HID are usually packed into a single machine word. Therefore, we must be careful to only permit control mappings that make sense. We cannot use a button as the control for a logical game function that requires an axis, for example. One way around this problem is to normalize all of the inputs. For example, we could re-scale the inputs from all analog axes and buttons into the range $[0, 1]$. This isn’t quite as helpful as you might at first think, because some axes are inherently bidirectional (like a joy stick) while others are unidirectional (like a trigger). But, if we group our controls into a few classes, we can normalize the inputs within those classes and permit remapping only within compatible classes. A reasonable set of classes for a standard console joystick and their normalized input values might be:

- *Digital buttons*. States are packed into a 32-bit word, one bit per button.
- *Unidirectional absolute axes* (e.g., triggers, analog buttons). Produce floating-point input values in the range $[0, 1]$.
- *Bidirectional absolute axes* (e.g., joy sticks). Produce floating-point input values in the range $[-1, 1]$.
- *Relative axes* (e.g., mouse axes, wheels, track balls). Produce floating-point input values in the range $[-1, 1]$, where ± 1 represents the maximum relative offset possible within a single game frame (i.e., during a period of $1/30$ or $1/60$ of a second).

9.5.8 Context-Sensitive Controls

In many games, a single physical control can have different functions, depending on context. A simple example is the ubiquitous “use” button. If pressed while standing in front of a door, the “use” button might cause the character to open the door. If it is pressed while standing near an object, it might cause the player character to pick up the object and so on. Another common example is a modal control scheme. When the player is walking around, the controls are used to navigate and control the camera. When the player is riding a vehicle, the controls are used to steer the vehicle, and the camera controls might be different as well.

Context-sensitive controls are reasonably straightforward to implement via a state machine. Depending on what state we’re in, a particular HID control may have a different purpose. The tricky part is deciding what state to be in. For example, when the context-sensitive “use” button is pressed, the player might be standing at a point equidistant between a weapon and a health pack, facing the center point between them. Which object do we use in this case? Some games implement a priority system to break ties like this. Perhaps the weapon has a higher weight than the health pack, so it would “win” in this example. Implementing context-sensitive controls isn’t rocket science, but it invariably requires lots of trial and error to get it feeling and behaving just right. Plan on lots of iteration and focus testing!

Another related concept is that of *control ownership*. Certain controls on the HID might be “owned” by different parts of the game. For example, some inputs are for player control, some for camera control and still others are for use by the game’s wrapper and menu system (pausing the game, and so on). Some game engines introduce the concept of a logical device, which is composed of only a subset of the inputs on the physical device. One logical device might be used for player control, while another is used by the camera system, and another by the menu system.

9.5.9 Disabling Inputs

In most games, it is sometimes necessary to disallow the player from controlling his or her character. For example, when the player character is involved in an in-game cinematic, we might want to disable all player controls temporarily; or when the player is walking through a narrow doorway, we might want to temporarily disable free camera rotation.

One rather heavy-handed approach is to use a bitmask to disable individual controls on the input device itself. Whenever the control is read, the disable mask is checked, and if the corresponding bit is set, a neutral or zero value is

returned instead of the actual value read from the device. We must be particularly cautious when disabling controls, however. If we forget to reset the disable mask, the game can get itself into a state where the player loses all control forever and must restart the game. It's important to check our logic carefully, and it's also a good idea to put in some fail-safe mechanisms to ensure that the disable mask is cleared at certain key times, such as whenever the player dies and re-spawns.

Disabling a HID input masks it for all possible clients, which can be overly limiting. A better approach is probably to put the logic for disabling specific player actions or camera behaviors directly into the player or camera code itself. That way, if the camera decides to ignore the deflection of the right thumb stick, for example, other game engine systems still have the freedom to read the state of that stick for other purposes.

9.6 Human Interface Devices in Practice

Correct and smooth handling of human interface devices is an important part of any good game. Conceptually speaking, HIDs may seem quite straightforward. However, there can be quite a few “gotchas” to deal with, including variations between different physical input devices, proper implementation of low-pass filtering, bug-free handling of control scheme mappings, achieving just the right “feel” in your joystick rumble, limitations imposed by console manufacturers via their technical requirements checklists (TRCs), and the list goes on. A game team should expect to devote a nontrivial amount of time and engineering bandwidth to a careful and complete implementation of the human interface device system. This is extremely important because the HID system forms the underpinnings of your game's most precious resource—its player mechanics.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

10

Tools for Debugging and Development

Developing game software is a complex, intricate, math-intensive and error-prone business. So it should be no surprise that virtually every professional game team builds a suite of tools for themselves, in order to make the game development process easier and less error-prone. In this chapter, we'll take a look at the development and debugging tools most often found in professional-grade game engines.

10.1 Logging and Tracing

Remember when you wrote your first program in BASIC or Pascal? (OK, maybe you don't. If you're significantly younger than me—and there's a pretty good chance of that—you probably wrote your first program in Java, or maybe Python or Lua.) In any case, you probably remember how you debugged your programs back then. You know, back when you thought a *debugger* was one of those glowing blue insect zapper things? You probably used *print statements* to dump out the internal state of your program. C/C++ programmers call this *printf debugging* (after the C standard library function, `printf()`).

It turns out that *printf debugging* is still a perfectly valid thing to do—even if you know that a debugger isn't a device for frying hapless insects at

night. Especially in real-time programming, it can be difficult to trace certain kinds of bugs using breakpoints and watch windows. Some bugs are timing-dependent: they only happen when the program is running at full speed. Other bugs are caused by a complex sequence of events too long and intricate to trace manually one-by-one. In these situations, the most powerful debugging tool is often a sequence of print statements.

Every game platform has some kind of console or teletype (TTY) output device. Here are some examples:

- In a console application written in C/C++, running under Linux or Win32, you can produce output in the console by printing to `stdout` or `stderr` via `printf()`, `fprintf()` or the C++ standard library's `iostream` interface.
- Unfortunately, `printf()` and `iostream` don't work if your game is built as a windowed application under Win32, because there's no console in which to display the output. However, if you're running under the Visual Studio debugger, it provides a debug console to which you can print via the Win32 function `OutputDebugString()`.
- On the PlayStation 3 and PlayStation 4, an application known as the Target Manager (or PlayStation Neighborhood on the PS4) runs on your PC and allows you to launch programs on the console. The Target Manager includes a set of TTY output windows to which messages can be printed by the game engine.

So printing out information for debugging purposes is almost always as easy as adding calls to `printf()` throughout your code. However, most game engines go a bit farther than this. In the following sections, we'll investigate the kinds of printing facilities most game engines provide.

10.1.1 Formatted Output with `OutputDebugString()`

The Windows SDK function `OutputDebugString()` is great for printing debugging information to Visual Studio's Debug Output window. However, unlike `printf()`, `OutputDebugString()` does not support *formatted* output—it can only print raw strings in the form of arrays. For this reason, most Windows game engines wrap it in a custom function, like this:

```
#include <stdio.h>          // for va_list et al

#ifdef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN 1
#endif
#include <windows.h>        // for OutputDebugString()
```

```

int VDebugPrintF(const char* format, va_list argList)
{
    const U32 MAX_CHARS = 1024;
    static char s_buffer[MAX_CHARS];

    int charsWritten
        = vsnprintf(s_buffer, MAX_CHARS, format, argList);

    // Now that we have a formatted string, call the
    // Win32 API.
    OutputDebugString(s_buffer);

    return charsWritten;
}

int DebugPrintF(const char* format, ...)
{
    va_list argList;
    va_start(argList, format);

    int charsWritten = VDebugPrintF(format, argList);

    va_end(argList);
    return charsWritten;
}

```

Notice that two functions are implemented: `DebugPrintF()` takes a variable-length argument list (specified via the ellipsis, `...`), while `VDebugPrintF()` takes a `va_list` argument. This is done so that programmers can build additional printing functions in terms of `VDebugPrintF()`. (It's impossible to pass ellipses from one function to another, but it *is* possible to pass `va_lists` around.)

10.1.2 Verbosity

Once you've gone to the trouble of adding a bunch of print statements to your code in strategically chosen locations, it's nice to be able to leave them there, in case they're needed again later. To permit this, most engines provide some kind of mechanism for controlling the level of *verbosity* via the command line, or dynamically at runtime. When the verbosity level is at its minimum value (usually zero), only critical error messages are printed. When the verbosity is higher, more of the print statements embedded in the code start to contribute to the output.

The simplest way to implement this is to store the current verbosity level in a global integer variable, perhaps called `g_verbosity`. We then provide a `VerboseDebugPrintf()` function whose first argument is the verbosity level at or above which the message will be printed. This function could be implemented as follows:

```
int g_verbosity = 0;

void VerboseDebugPrintf(int verbosity,
                        const char* format, ...)
{
    // Only print when the global verbosity level is
    // high enough.
    if (g_verbosity >= verbosity)
    {
        va_list argList;
        va_start(argList, format);

        VDebugPrintf(format, argList);

        va_end(argList);
    }
}
```

10.1.3 Channels

It's also extremely useful to be able to categorize your debug output into *channels*. One channel might contain messages from the animation system, while another might be used to print messages from the physics system, for example.

On some platforms, like the PlayStation 3, debug output can be directed to one of 14 distinct TTY windows. In addition, messages are mirrored to a special TTY window that contains the output from all of the other 14 windows. This makes it very easy for a developer to focus in on only the messages he or she wants to see. When working on an animation problem, one can simply flip to the animation TTY and ignore all the other output. When working on a general problem of unknown origin, the "all" TTY can be consulted for clues.

Other platforms like Windows provide only a single debug output console. However, even on these systems it can be helpful to divide your output into channels. The output from each channel might be assigned a different color. You might also implement *filters*, which can be turned on and off at runtime, and restrict output to only a specified channel or set of channels. In this model, if a developer is debugging an animation-related problem, for example, he or she can simply filter out all of the channels except the animation channel.

A channel-based debug output system can be implemented quite easily by adding an additional channel argument to our debug printing function. Channels might be numbered, or better, assigned symbolic values via a C/C++ enum declaration. Or channels might be named using a string or hashed string id. The printing function can simply consult the list of active channels and only print the message if the specified channel is among them.

If you don't have more than 32 or 64 channels, it can be helpful to identify the channels via a 32- or 64-bit mask. This makes implementing a channel filter as easy as specifying a single integer. When a bit in the mask is 1, the corresponding channel is active; when the bit is 0, the channel is muted.

10.1.3.1 Using Redis to Manage TTY Channels

The developers at Naughty Dog use a web-based interface called *Connector* as their window into the various streams of debugging information that are emitted by the game engine at runtime. The game spits out its debug text into various named channels, each associated with a different engine system (animation, rendering, AI, sound, etc.) These data streams are collected by a lightweight Redis key-value store (see <http://redis.io> for more information on Redis). The Connector interface allows users to view and filter this Redis data easily from any web browser.

10.1.4 Mirroring Output to a File

It's a good idea to mirror all debug output to one or more log files (e.g., one file per channel). This permits problems to be diagnosed after the fact. Ideally the log file(s) should contain *all* of the debug output, independent of the current verbosity level and active channels mask. This allows *unexpected* problems to be caught and tracked down by simply inspecting the most-recent log files.

You may want to consider *flushing* your log file(s) after every call to your debug output function to ensure that if the game crashes, the log file(s) won't be missing the last buffer-full of output. The last data printed are usually the most useful for determining the cause of a crash, so we want to be sure that the log file always contains the most up-to-date output. Of course, flushing the output buffer can be expensive. So you should only flush buffers after every debug output call if either (a) you are not doing a lot of logging, or (b) you discover that it is truly necessary on your particular platform. If flushing is deemed to be necessary, you can always provide an engine configuration option to turn it on and off.

10.1.5 Crash Reports

Some game engines produce special text output and/or log files when the game crashes. In most operating systems, a top-level exception handler can be installed that will catch most crashes. In this function, you could print out all sorts of useful information. You could even consider emailing the crash report to the entire programming team. This can be incredibly enlightening for the programmers: When they see just how often the art and design teams are crashing, they may discover a renewed sense of urgency in their debugging tasks!

Here are just a few examples of the kinds of information you can include in a crash report:

- Current level(s) being played at the time of the crash.
- World-space location of the player character when the crash occurred.
- Animation/action state of the player when the game crashed.
- Gameplay script(s) that were running at the time of the crash. (This can be especially helpful if the script is the cause of the crash!)
- Stack trace. Most operating systems provide a mechanism for walking the call stack (although they are nonstandard and highly platform specific). With such a facility, you can print out the symbolic names of all non-inline functions on the stack at the time the crash occurred.
- State of all memory allocators in the engine (amount of memory free, degree of fragmentation, etc.). This kind of data can be helpful when bugs are caused by low-memory conditions, for example.
- Any other information you think might be relevant when tracking down the cause of a crash.
- A screenshot of the game at the moment it crashed.

10.2 Debug Drawing Facilities

Modern interactive games are driven almost entirely by math. We use math to position and orient objects in the game world, move them around, test for collisions, and cast rays to determine lines of sight, and of course we use matrix multiplication to transform objects from object space to world space and eventually into screen space for rendering. Almost all modern games are three-dimensional, but even in a two-dimensional game it can be very difficult to mentally visualize the results of all these mathematical calculations. For this reason, most good game engines provide an API for drawing colored lines,

simple shapes and 3D text. We call this a *debug drawing* facility, because the lines, shapes and text that are drawn with it are intended for visualization during development and debugging and are removed prior to shipping the game.

A *debug drawing* API can save you huge amounts of time. For example, if you are trying to figure out why your projectiles are not hitting the enemy characters, which is easier? Deciphering a bunch of numbers in the debugger? Or drawing a line showing the trajectory of the projectile in three dimensions within your game? With a debug drawing API, logical and mathematical errors become immediately obvious. One might say that a picture is worth a thousand minutes of debugging.

Here are some examples of debug drawing in action within Naughty Dog's engine. The following screenshots were all taken within our *play-test* level, one of many special levels we use for testing out new features and debugging problems in the game.

- Figure 10.1 shows a visualization of an enemy NPC's perception of the player. The little "stick man" figure represents the location of the player as perceived by the NPC. When the player has broken the line of sight between himself and the NPC, the "stick man" will remain at the player's last known location, even after the player sneaks away.
- Figure 10.2 shows how a wireframe sphere can be used to visualize the dynamically expanding blast radius of an explosion.



Figure 10.1. Visualizing the line of sight from an NPC to the player in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

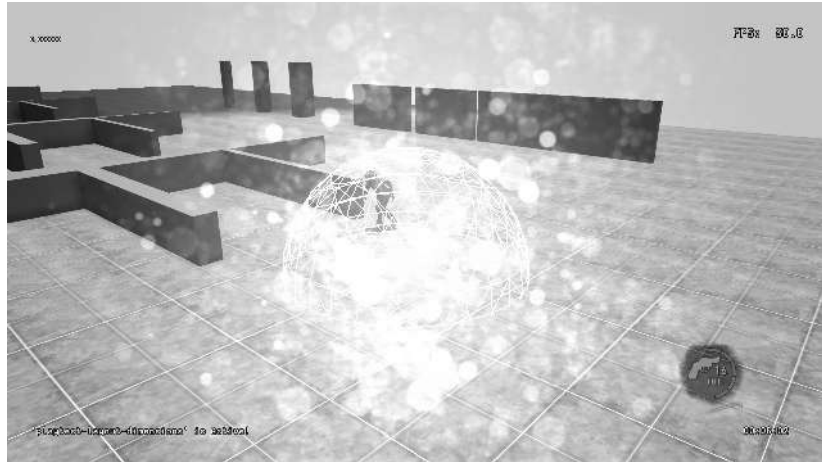


Figure 10.2. Visualizing the expanding blast sphere of an explosion in the Naughty Dog engine.

- Figure 10.3 shows how circles can be used to visualize the radii used by Drake when searching for ledges to hang from in the game. A line shows the ledge he is currently hanging from.
- Figure 10.4 shows an AI character that has been placed in a special debugging mode. In this mode, the character's brain is effectively turned off, and the developer is given full control over the character's movements and actions via a simple heads-up menu. The developer can paint

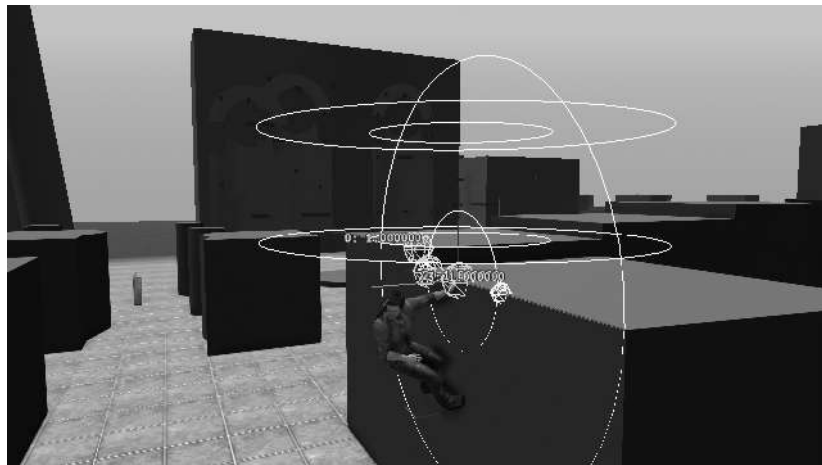


Figure 10.3. Spheres and vectors used in Drake's ledge hang and shimmy system in the *Uncharted* series (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 3).

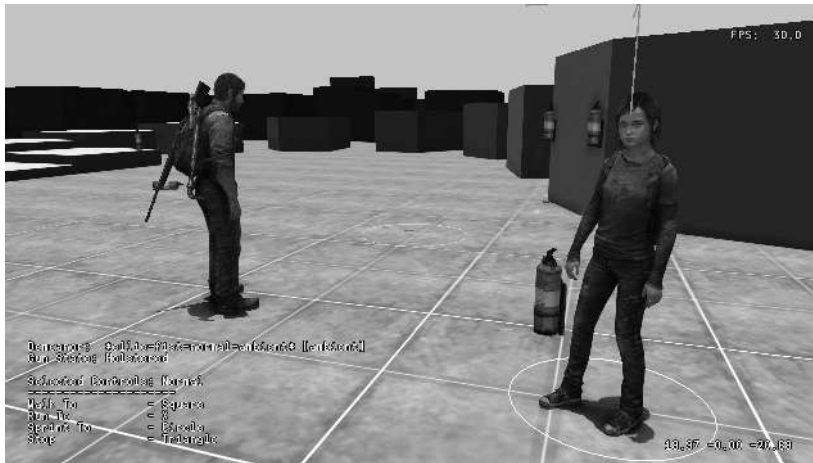


Figure 10.4. Manually controlling an NPC's actions for debugging purposes in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

target points in the game world by simply aiming the camera and can then instruct the character to walk, run or sprint to the specified points. The user can also tell the character to enter or leave nearby cover, fire its weapon and so on.

10.2.1 Debug Drawing API

A debug drawing API generally needs to satisfy the following requirements:

- The API should be simple and easy to use.
- It should support a useful set of primitives, including (but not limited to):
 - lines,
 - spheres,
 - points (usually represented as small crosses or spheres, because a single pixel is very difficult to see),
 - coordinate axes (typically, the x -axis is drawn in red, y in green and z in blue),
 - bounding boxes, and
 - formatted text.
- It should provide a good deal of flexibility in controlling how primitives are drawn, including:

- color,
 - line width,
 - sphere radii,
 - the size of points, lengths of coordinate axes, and dimensions of other “canned” primitives.
- It should be possible to draw primitives in world space (full 3D, using the game camera’s perspective projection matrix) or in screen space (either using an orthographic projection, or possibly a perspective projection). World-space primitives are useful for annotating objects in the 3D scene. Screen-space primitives are helpful for displaying debugging information in the form of a heads-up display that is independent of camera position or orientation.
- It should be possible to draw primitives with or without *depth testing* enabled.
 - When depth testing is enabled, the primitives will be occluded by real objects in your scene. This makes their depth easy to visualize, but it also means that the primitives may sometimes be difficult to see or totally hidden by the geometry of your scene.
 - With depth testing disabled, the primitives will “hover” over the real objects in the scene. This makes it harder to gauge their real depth, but it also ensures that no primitive is ever hidden from view.
- It should be possible to make calls to the drawing API from anywhere in your code. Most rendering engines require that geometry be submitted for rendering during a specific phase of the game loop, usually at the end of each frame. This requirement implies that the system must queue up all incoming debug drawing requests, so that they may be submitted at the proper time later on.
- Ideally, every debug primitive should have a *lifetime* associated with it. The lifetime controls how long the primitive will remain on-screen after having been requested. If the code that is drawing the primitive is called every frame, the lifetime can be one frame—the primitive will remain on-screen because it will be refreshed every frame. However, if the code that draws the primitive is called rarely or intermittently (e.g., a function that calculates the initial velocity of a projectile), then you do not want the primitive to flicker on-screen for just one frame and then disappear. In such situations, the programmer should be able to give his or her debug primitives a longer lifetime, on the order of a few seconds.

- It's also important that the debug drawing system be capable of handling a large number of debug primitives efficiently. When you're drawing debug information for 1,000 game objects, the number of primitives can really add up, and you don't want your game to be unusable when debug drawing is turned on.

The debug drawing API in Naughty Dog's engine looks something like this:

```
class DebugDrawManager
{
public:
    // Adds a line segment to the debug drawing queue.
    void AddLine(const Point& fromPosition,
                  const Point& toPosition,
                  Color color,
                  float lineWidth = 1.0f,
                  float duration = 0.0f,
                  bool depthEnabled = true);

    // Adds an axis-aligned cross (3 lines converging at
    // a point) to the debug drawing queue.
    void AddCross(const Point& position,
                   Color color,
                   float size,
                   float duration = 0.0f,
                   bool depthEnabled = true);

    // Adds a wireframe sphere to the debug drawing queue.
    void AddSphere(const Point& centerPosition,
                    float radius,
                    Color color,
                    float duration = 0.0f,
                    bool depthEnabled = true);

    // Adds a circle to the debug drawing queue.
    void AddCircle(const Point& centerPosition,
                    const Vector& planeNormal,
                    float radius,
                    Color color,
                    float duration = 0.0f,
                    bool depthEnabled = true);

    // Adds a set of coordinate axes depicting the
    // position and orientation of the given
    // transformation to the debug drawing queue.
    void AddAxes(const Transform& xfm,
                  Color color,
                  float size,
```

```

        float duration = 0.0f,
        bool depthEnabled = true);

// Adds a wireframe triangle to the debug drawing
// queue.
void AddTriangle(const Point& vertex0,
                  const Point& vertex1,
                  const Point& vertex2,
                  Color color,
                  float lineWidth = 1.0f,
                  float duration = 0.0f,
                  bool depthEnabled = true);

// Adds an axis-aligned bounding box to the debug
// queue.
void AddAABB(const Point& minCoords,
              const Point& maxCoords,
              Color color,
              float lineWidth = 1.0f,
              float duration = 0.0f,
              bool depthEnabled = true);

// Adds an oriented bounding box to the debug queue.
void AddOBB(const Mat44& centerTransform,
              const Vector& scaleXYZ,
              Color color,
              float lineWidth = 1.0f,
              float duration = 0.0f,
              bool depthEnabled = true);

// Adds a text string to the debug drawing queue.
void AddString(const Point& pos,
                 const char* text,
                 Color color,
                 float duration = 0.0f,
                 bool depthEnabled = true);
};

// This global debug drawing manager is configured for
// drawing in full 3D with a perspective projection.
extern DebugDrawManager g_debugDrawMgr;

// This global debug drawing manager draws its
// primitives in 2D screen space. The (x,y) coordinates
// of a point specify a 2D location on-screen, and the
// z coordinate contains a special code that indicates
// whether the (x,y) coordinates are measured in absolute

```

```
// pixels or in normalized coordinates that range from
// 0.0 to 1.0. (The latter mode allows drawing to be
// independent of the actual resolution of the screen.)
extern DebugDrawManager g_debugDrawMgr2D;
```

Here's an example of this API being used within game code:

```
void Vehicle::Update()
{
    // Do some calculations...

    // Debug-draw my velocity vector.
    const Point& start = GetWorldSpacePosition();
    Point end = start + GetVelocity();
    g_debugDrawMgr.AddLine(start, end, kColorRed);

    // Do some other calculations...

    // Debug-draw my name and number of passengers.
    {
        char buffer[128];
        sprintf(buffer, "Vehicle %s: %d passengers",
                GetName(), GetNumPassengers());

        const Point& pos = GetWorldSpacePosition();
        g_debugDrawMgr.AddString(pos,
                                buffer, kColorWhite, 0.0f, false);
    }
}
```

You'll notice that the names of the drawing functions use the verb "add" rather than "draw." This is because the debug primitives are typically not drawn immediately when the drawing function is called. Instead, they are added to a list of visual elements that will be drawn at a later time. Most high-speed 3D rendering engines require that all visual elements be maintained in a *scene* data structure so that they can be drawn efficiently, usually at the end of the game loop. We'll learn a lot more about how rendering engines work in Chapter 11.

10.3 In-Game Menus

Every game engine has a large number of configuration options and features. In fact, each major subsystem, including rendering, animation, collision, physics, audio, networking, player mechanics, AI and so on, exposes its own specialized configuration options. It is highly useful to programmers, artists

and game designers alike to be able to configure these options while the game is running, without having to change the source code, recompile and relink the game executable, and then rerun the game. This can greatly reduce the amount of time the game development team spends on debugging problems and setting up new levels or game mechanics.

One simple and convenient way to permit this kind of thing is to provide a system of *in-game menus*. Items on an in-game menu can do any number of things, including (but certainly not limited to):

- toggling global Boolean settings,
- adjusting global integer and floating-point values,
- calling arbitrary functions, which can perform literally any task within the engine, and
- bringing up submenus, allowing the menu system to be organized hierarchically for easy navigation.

An in-game menu should be easy and convenient to bring up, perhaps via a simple button press on the joypad. (Of course, you'll want to choose a button combination that doesn't occur during normal gameplay.) Bringing up the menus usually pauses the game. This allows the developer to play the game until the moment just before a problem occurs, then pause the game by bringing up the menus, adjust engine settings in order to visualize the

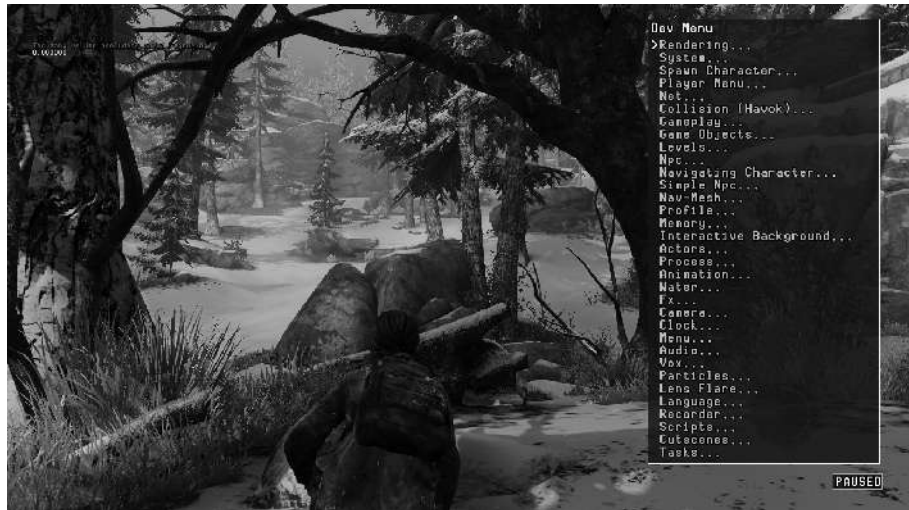


Figure 10.5. Main development menu in *The Last of Us: Remastered* (© 2014™ SIE. Created and developed by Naughty Dog, PlayStation 4).

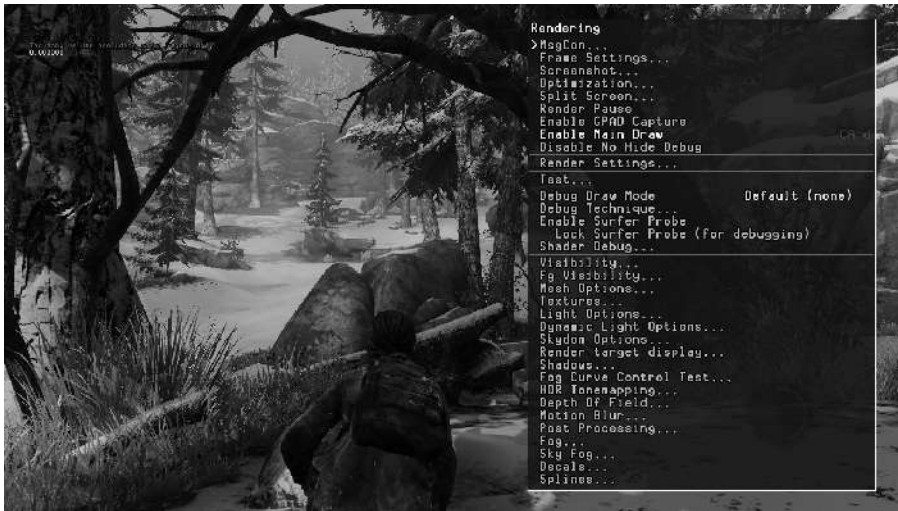


Figure 10.6. Rendering submenu in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

problem more clearly, and then un-pause the game to inspect the problem in depth.

Let's take a brief look at how the menu system works in the Naughty Dog engine. Figure 10.5 shows the top-level menu. It contains submenus for each major subsystem in the engine. In Figure 10.6, we've drilled down one level

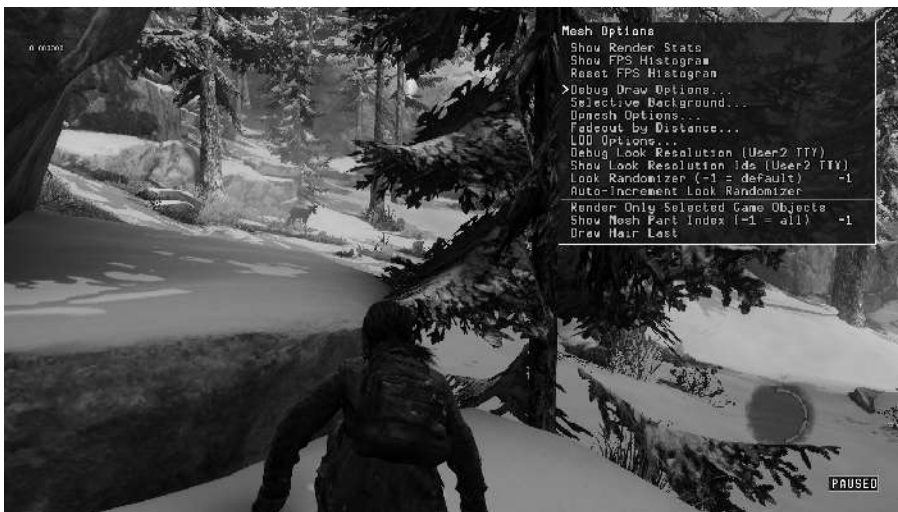


Figure 10.7. Mesh options subsubmenu in *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).



Figure 10.8. Background meshes turned off (*The Last of Us: Remastered* © 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

into the *Rendering...* submenu. Since the rendering engine is a highly complex system, its menu contains many submenus controlling various aspects of rendering. To control the way in which 3D meshes are rendered, we drill down further into the *Mesh Options...* submenu, shown in Figure 10.7. On this menu, we can turn off rendering of all static background meshes, leaving only the dynamic foreground meshes visible. This is shown in Figure 10.8. (Ah ha, there's that pesky deer!)

10.4 In-Game Console

Some engines provide an in-game console, either in lieu of or in addition to an in-game menu system. An in-game console provides a command-line interface to the game engine's features, much as a DOS *command prompt* provides users with access to various features of the Windows operating system, or a *csh*, *tcsh*, *ksh* or *bash shell prompt* provides users with access to the features of UNIX-like operating systems. Much like a menu system, the game engine console can provide commands allowing a developer to view and manipulate global engine settings, as well as running arbitrary commands.

A console is somewhat less convenient than a menu system, especially for those who aren't very fast typists. However, a console can be much more powerful than a menu. Some in-game consoles provide only a rudimentary set of hard-coded commands, making them about as flexible as a menu system.



Figure 10.9. The in-game console in *Minecraft*, overlaid on top of the main game screen and displaying a list of valid commands.

But others provide a rich interface to virtually every feature of the engine. A screenshot of the in-game console in *Minecraft* is shown in Figure 10.9.

Some game engines provide a powerful scripting language that can be used by programmers and game designers to extend the functionality of the engine, or even build entirely new games. If the in-game console “speaks” this same scripting language, then anything you can do in script can also be done interactively via the console. We’ll explore scripting languages in depth in Section 16.9.

10.5 Debug Cameras and Pausing the Game

An in-game menu or console system is best accompanied by two other crucial features: (a) the ability to detach the camera from the player character and fly it around the game world in order to scrutinize any aspect of the scene, and (b) the ability to pause, un-pause and single-step the game (see Section 8.5.5). When the game is paused, it is still important to be able to control the camera. To support this, we can simply keep the rendering engine and camera controls running, even when the game’s logical clock is paused.

Slow motion mode is another incredibly useful feature for scrutinizing animations, particle effects, physics and collision behaviors, AI behaviors, and the list goes on. This feature is easy to implement. Presuming we’ve taken care to update all gameplay elements using a clock that is distinct from the real-time clock, we can put the game into slo-mo by simply updating the gameplay clock at a rate that is slower than usual. This approach can also be used to implement a fast-motion mode, which can be useful for moving rapidly through time-consuming portions of gameplay in order to get to an area of interest (not to mention being a great source of laughs, especially when accompanied by a bad vocal rendition of Benny Hill music...).

10.6 Cheats

When developing or debugging a game, it's important to allow the user to break the rules of the game in the name of expediency. Such features are aptly named *cheats*. For example, many engines allow you to “pick up” the player character and fly him or her around in the game world, with collisions disabled so he or she can pass through all obstacles. This can be incredibly helpful for testing out gameplay. Rather than taking the time to actually play the game in an attempt to get the player character into some desirable location, you can simply pick him up, fly him over to where you want him to be, and then drop him back into his regular gameplay mode.

Other useful cheats include, but are certainly not limited to:

- *Invincible player.* As a developer, you often don't want to be bothered having to defend yourself from enemy characters, or worrying about falling from too high a height, as you test out a feature or track down a bug.
- *Give player weapon.* It's often useful to be able to give the player any weapon in the game for testing purposes.
- *Infinite ammo.* When you're trying to kill bad guys to test out the weapon system or AI hit reactions, you don't want to be scrounging for clips!
- *Select player mesh.* If the player character has more than one “costume,” it can be useful to be able to select any of them for testing purposes.

Obviously this list could go on for pages. The sky's the limit—you can add whatever cheats you need in order to develop or debug the game. You might even want to expose some of your favorite cheats to the players of the final shipping game. Players can usually activate cheats by entering unpublished *cheat codes* on the joypad or keyboard and/or by accomplishing certain objectives in the game.

10.7 Screenshots and Movie Capture

Another extremely useful facility is the ability to capture screenshots and write them to disk in a suitable image format such as Windows Bitmap files (.bmp), JPEG (.jpg) or Targa (.tga). The details of how to capture a screenshot vary from platform to platform, but they typically involve making a call to the graphics API that allows the contents of the frame buffer to be transferred from video RAM to main RAM, where it can be scanned and converted into the image file format of your choice. The image files are typically written to a predefined

folder on disk and named using a date and time stamp to guarantee unique file names.

You may want to provide your users with various options controlling how screenshots are to be captured. Some common examples include:

- Whether or not to include debug primitives and text in the screenshot.
- Whether or not to include heads-up display (HUD) elements in the screenshot.
- The resolution at which to capture. Some engines allow high-resolution screenshots to be captured, perhaps by modifying the projection matrix so that separate screenshots can be taken of the four quadrants of the screen at normal resolution and then combined into the final high-res image.
- Simple camera animations. For example, you could allow the user to mark the starting and ending positions and orientations of the camera. A sequence of screenshots could then be taken while gradually interpolating the camera from the starting location to the ending location.

Some engines also provide a full-fledged movie capture mode. Such a system captures a sequence of screenshots at the target frame rate of the game, which are processed either offline or at runtime to generate a movie file in a suitable format such as MPEG-2 (H.262) or MPEG-4 Part 10 (H.264). But even if your engine doesn't support real-time video capture, external hardware like *Roxio Game Capture HD Pro* can always be used to capture the output from your game console or PC. And for PC and Mac games, a great many software video capture tools are available, including *Fraps* by Beepa, *Camtasia* by Camtasia Software, *Dxtory* by ExKode, *Debut* by NCH Software and *Action!* by Mirillis.

The PlayStation 4 has built-in support for sharing screenshots and video clips taken from within the game. During gameplay, the PS4 is continually capturing video of the most-recent 15 minutes of the user's in-game experience. At any moment, the user can hit the Share button on the controller and opt to share a screenshot or the recorded video in various ways—by saving it to the PS4's HDD or a thumb drive, or by uploading it to one of a number of online services. At Naughty Dog, we use these facilities to capture video and a screenshot of the game any time it crashes, thereby allowing us to see what situation led up to the crash.

PlayStation 4 users can also live-stream video of their playthroughs. For development purposes, it's possible to use a PC to connect to a remote PS4 (perhaps sitting on another developer's desk within or outside your studio), to

including raw numbers of cycles, execution times in microseconds, and percentages relative to the execution time of the entire frame.



Figure 10.11. The timeline mode in *Uncharted: The Lost Legacy* (© 2017/™ SIE. Created and developed by Naughty Dog, PlayStation 4) shows exactly when various operations are performed across a single frame on the PS4's seven CPU cores.

10.8.1 Hierarchical Profiling

Computer programs written in an imperative language are inherently *hierarchical*—a function calls other functions, which in turn call still more functions. For example, let's imagine that function `a()` calls functions `b()` and `c()`, and function `b()` in turn calls functions `d()`, `e()` and `f()`. The pseudocode for this is shown below.

```
void a()
{
    b();
    c();
}

void b()
{
    d();
    e();
    f();
}

void c() { ... }
```

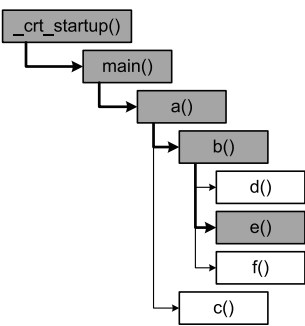


Figure 10.13. Call stack resulting from setting a breakpoint in function `e()`.

```
void d() { ... }  
  
void e() { ... }  
  
void f() { ... }
```

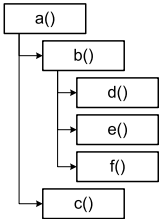


Figure 10.12. A hypothetical function call hierarchy.

Assuming function `a()` is called directly from `main()`, this function call hierarchy is shown in Figure 10.12.

When debugging a program, the *call stack* shows only a snapshot of this tree. Specifically, it shows us the path from whichever function in the hierarchy is *currently executing* all the way to the root function in the tree. In C/C++, the root function is usually `main()` or `WinMain()`, although technically this function is called by a start-up function that is part of the standard C runtime library (CRT), so that function is the true root of the hierarchy. If we set a breakpoint in function `e()`, for example, the call stack would look something like this:

```
e()           ← The currently executing function.  
b()  
a()  
main()  
_crt_startup() ← Root of the call hierarchy.
```

This call stack is depicted in Figure 10.13 as a pathway from function `e()` to the root of the function call tree.

10.8.1.1 Measuring Execution Times Hierarchically

If we measure the execution time of a single function, the time we measure includes the execution time of any the child functions called and all of their

grandchildren, great-grandchildren and so on as well. To properly interpret any profiling data we might collect, we must be sure to take the function call hierarchy into account.

Many commercial profilers can *automatically* instrument every single function in your program. This permits them to measure both the *inclusive* and *exclusive* execution times of every function that is called during a profiling session. As the name implies, inclusive times measure the execution time of the function including all of its children, while exclusive times measure only the time spent in the function itself. (The exclusive time of a function can be calculated by subtracting the inclusive times of all its immediate children from the inclusive time of the function in question.) In addition, some profilers record how many times each function is called. This is an important piece of information to have when optimizing a program, because it allows you to differentiate between functions that eat up a lot of time internally and functions that eat up time because they are called a very large number of times.

In contrast, in-game profiling tools are not so sophisticated and usually rely on *manual* instrumentation of the code. If our game engine's main loop is structured simply enough, we may be able to obtain valid data at a coarse level without thinking much about the function call hierarchy. For example, a typical game loop might look roughly like this:

```
while (!quitGame)
{
    PollJoypad();
    UpdateGameObjects();
    UpdateAllAnimations();
    PostProcessJoints();
    DetectCollisions();
    RunPhysics();
    GenerateFinalAnimationPoses();
    UpdateCameras();
    RenderScene();
    UpdateAudio();
}
```

We could profile this game at a very coarse level by measuring the execution times of each major phase of the game loop:

```
while (!quitGame)
{
    {
        PROFILE(SID("Poll Joypad"));
        PollJoypad();
    }
}
```

```

    {
        PROFILE(SID("Game Object Update"));
        UpdateGameObjects();
    }
    {
        PROFILE(SID("Animation"));
        UpdateAllAnimations();
    }
    {
        PROFILE(SID("Joint Post-Processing"));
        PostProcessJoints();
    }
    {
        PROFILE(SID("Collision"));
        DetectCollisions();
    }
    {
        PROFILE(SID("Physics"));
        RunPhysics();
    }
    {
        PROFILE(SID("Animation Finaling"));
        GenerateFinalAnimationPoses();
    }
    {
        PROFILE(SID("Cameras"));
        UpdateCameras();
    }
    {
        PROFILE(SID("Rendering"));
        RenderScene();
    }
    {
        PROFILE(SID("Audio"));
        UpdateAudio();
    }
}

```

The `PROFILE()` macro shown above would probably be implemented as a class whose constructor starts the timer and whose destructor stops the timer and records the execution time under the given name. Thus, it only times the code within its containing block, by nature of the way C++ automatically constructs and destroys objects as they go in and out of scope.

```

struct AutoProfile
{
    AutoProfile(const char* name)

```

```

    {
        m_name = name;
        m_startTime = QueryPerformanceCounter();
    }

    ~AutoProfile()
    {
        std::int64_t endTime = QueryPerformanceCounter();
        std::int64_t elapsedTime = endTime - m_startTime;

        g_profileManager.storeSample(m_name, elapsedTime);
    }

    const char*    m_name;
    std::int64_t    m_startTime;
};

#define PROFILE(name) AutoProfile p(name)

```

The problem with this simplistic approach is that it breaks down when used within deeper levels of function call nesting. For example, if we embed additional `PROFILE()` annotations within the `RenderScene()` function, we need to understand the function call hierarchy in order to properly interpret those measurements.

One solution to this problem is to allow the programmer who is annotating the code to indicate the hierarchical interrelationships between profiling samples. For example, any `PROFILE(...)` samples taken within the `RenderScene()` function could be declared to be children of the `PROFILE(SID("Rendering"))` sample. These relationships are usually set up separately from the annotations themselves, by predeclaring all of the sample bins. For example, we might set up the in-game profiler during engine initialization as follows:

```

// This code declares various profile sample "bins",
// listing the name of the bin and the name of its
// parent bin, if any.

ProfilerDeclareSampleBin(SID("Rendering"), nullptr);
ProfilerDeclareSampleBin(SID("Visibility"), SID("Rendering"));
ProfilerDeclareSampleBin(SID("Shaders"), SID("Rendering"));
    ProfilerDeclareSampleBin(SID("Materials"), SID("Shaders"));

    ProfilerDeclareSampleBin(SID("SubmitGeo"), SID("Rendering"));

ProfilerDeclareSampleBin(SID("Audio"), nullptr);

// ...

```

This approach still has its problems. Specifically, it works well when every function in the call hierarchy has only one parent, but it breaks down when we try to profile a function that is called by more than one parent function. The reason for this should be pretty obvious. We're statically declaring our sample bins *as if* every function can only appear once in the function call hierarchy, but actually the same function can reappear many times in the tree, each time with a different parent. The result can be misleading data, because a function's time will be included in one of the parent bins, but really should be distributed across all of its parents' bins. Most game engines don't make an attempt to remedy this problem, since they are primarily interested in profiling coarse-grained functions that are only called from one specific location in the function call hierarchy. But this limitation is something to be aware of when profiling your code with a simple in-engine profile of the sort found in most game engines.

Of course, it is also possible to write a much more sophisticated profiling system that handles nested instances of AutoProfile properly. This is an example of the many trade-offs one makes when designing a game engine. Do we invest the engineering time to create a fully hierarchical profiler? Or, do we make do with something simpler and invest those programming resources elsewhere? Ultimately, it's up to you.

We would also like to account for how many *times* a given function is called. In the example above, we know that each of the functions we profiled is called exactly once per frame. But other functions, deeper in the function call hierarchy, may be called more than once per frame. If we measure function `x()` to take 2 ms to execute, it's important to know whether it takes 2 ms to execute on its own, or whether its execution time is 2 ms but it was called 1,000 times during the frame. Keeping track of the number of times a function is called per frame is quite simple—the profiling system can simply increment a counter each time a sample is received and reset the counters at the start of each frame.

10.8.2 Exporting to Excel

Some game engines permit the data captured by the in-game profiler to be dumped to a text file for subsequent analysis. I find that a comma-separated values (CSV) format is best, because such files can be loaded easily into a Microsoft Excel spreadsheet, where the data can be manipulated and analyzed in myriad ways. I wrote such an exporter for the *Medal of Honor: Pacific Assault* engine. The columns corresponded to the various annotated blocks, and each row represented the profiling sample taken during one frame of the game's

execution. The first column contained frame numbers and the second actual game time measured in seconds. This allowed the team to graph how the performance statistics varied over time and to determine how long each frame actually took to execute. By adding some simple formulae to the exported spreadsheet, we could calculate frame rates, execution time percentages and so on.

10.9 In-Game Memory Stats and Leak Detection

In addition to runtime performance (i.e., frame rate), most game engines are also constrained by the amount of memory available on the target hardware. PC games are least affected by such constraints, because modern PCs have sophisticated virtual memory managers. But even PC games are constrained by the memory limitations of their so-called “min spec” machine—the least-powerful machine on which the game is guaranteed to run, as promised by the publisher and stated on the game’s packaging.

For this reason, most game engines implement custom memory-tracking tools. These tools allow the developers to see how much memory is being used by each engine subsystem and whether or not any memory is leaking (i.e., memory is allocated but never freed). It’s important to have this information so that you can make informed decisions when trying to cut back the memory usage of your game so that it will fit onto the console or type of PC you are targeting.

Keeping track of how much memory a game actually uses can be a surprisingly tricky job. You’d think you could simply wrap `malloc()`/`free()` or `new/delete` in a pair of functions or macros that keep track of the amount of memory that is allocated and freed. However, it’s never that simple for a few reasons:

1. *You often can’t control the allocation behavior of other people’s code.* Unless you write the operating system, drivers and the game engine entirely from scratch, there’s a good chance you’re going to end up linking your game with at least some third-party libraries. Most good libraries provide *memory allocation hooks* so that you can replace their allocators with your own. But some do not. It’s often difficult to keep track of the memory allocated by each and every third-party library you use in your game engine—but it usually *can* be done if you’re thorough and selective in your choice of third-party libraries.
2. *Memory comes in different flavors.* For example, a PC has two kinds of RAM: main RAM and video RAM (the memory residing on your graph-

ics card, which is used primarily for geometry and texture data). Even if you manage to track all of the memory allocations and deallocations occurring within main RAM, it can be well nigh impossible to track video RAM usage. This is because graphics APIs like DirectX actually hide the details of how video RAM is being allocated and used from the developer. On a console, life is a bit easier only because you often end up having to write a video RAM manager yourself. This is more difficult than using DirectX, but at least you have complete knowledge of what's going on.

3. *Allocators come in different flavors.* Many games make use of specialized allocators for various purposes. For example, the Naughty Dog engine has a *global heap* for general-purpose allocations, a special heap for managing the memory created by *game objects* as they spawn into the game world and are destroyed, a *level-loading heap* for data that is *streamed* into memory during gameplay, a stack allocator for *single-frame allocations* (the stack is cleared automatically every frame), an allocator for *video RAM*, and a *debug memory heap* used only for allocations that will not be needed in the final shipping game. Each of these allocators grabs a large hunk of memory when the game starts up and then manages that memory block itself. If we were to track all the calls to new and delete, we'd see one new for each of these six allocators and that's all. To get any useful information, we really need to track all of the allocations *within* each of these allocators' memory blocks.

Most professional game teams expend a significant amount of effort on creating in-engine memory-tracking tools that provide accurate and detailed information. The resulting tools usually provide their output in a variety of forms. For example, the engine might produce a detailed dump of all memory allocations made by the game during a specific period of time. The data might include high water marks for each memory allocator or each game system, indicating the maximum amount of physical RAM required by each. Some engines also provide heads-up displays of memory usage while the game is running. This data might be tabular, as shown in Figure 10.14, or graphical as shown in Figure 10.15.

In addition, when low-memory or out-of-memory conditions arise, a good engine will provide this information in as helpful a way as possible. When PC games are developed, the game team usually works on high-powered PCs with more RAM than the min-spec machine being targeted. Likewise, console games are developed on special *development kits* that have more memory than a retail console. So in both cases, the game can continue to run even when it



Figure 10.14. Tabular memory statistics from Naughty Dog's engine.

Figure 10.15. A graphical memory usage display, also from *The Last of Us: Remastered* (© 2014/™ SIE. Created and developed by Naughty Dog, PlayStation 4).

technically has run out of memory (i.e., would no longer fit on a retail console or min-spec PC). When this kind of out-of-memory condition arises, the game engine can display a message saying something like, “Out of memory—this level will not run on a retail system.”

There are lots of other ways in which a game engine’s memory tracking system can aid developers in pinpointing problems as early and as conveniently

as possible. Here are just a few examples:

- If a model fails to load, a bright red text string could be displayed in 3D hovering in the game world where that object would have been.
- If a texture fails to load, the object could be drawn with an ugly pink texture that is very obviously not part of the final game.
- If an animation fails to load, the character could assume a special (possibly humorous) pose that indicates a missing animation, and the name of the missing asset could hover over the character's head.

The key to providing good memory analysis tools is (a) to provide accurate information, (b) to present the data in a way that is convenient and that makes problems obvious and (c) to provide contextual information to aid the team in tracking down the root cause of problems when they occur.